Fragmentos de Luz

Documentación de ingeniería de software y ejecutable del videojuego en Godot 4.5

Parcial – Grupo 2

Integrantes

Jaider Villamizar – Product Owner y Developer

Camilo Vargas – Scrum Master y Developer

Kevin Yaruro – Developer

Diego Villamizar – Developer

Docente

Yeison Sanabria Ascanio

Programa de Ingeniería de Software, semestre IX

Diseño Multimedia y Videojuegos

Repositorio: https://github.com/jaider23100/FragmentosDeLuz

Versión publicada v1.0.0: https://github.com/jaider23100/FragmentosDeLuz/releases/tag/v1.0.0

Video demo: https://github.com/jaider23100/FragmentosDeLuz/releases/download/v1.0.0/FragmentosDeLuz-v1.0.0-demo.mp4

Fundación de Estudios Superiores Comfanorte – FESC

Cúcuta, 23 de octubre de 2026

Tabla de contenido

Glosario de términos y siglas....13
1. Inicio y planeación....16
1.1 Acta de constitución del proyecto....16
1.1.1 Objetivo general....16
1.1.2 Objetivos específicos....16
1.1.3 Justificación....16
1.1.4 Alcance de la versión 1.0.0....17
1.1.5 Fuera de alcance....17
1.1.6 Supuestos....18
1.1.7 Restricciones....18
1.1.8 Criterios de éxito....18
1.1.9 Integrantes y aprobación....19
1.2 Interesados....19
1.3 Metodología, roles e iteraciones....20
1.3.1 Marco de trabajo....20
1.3.2 Roles....21
1.3.3 Forma de trabajo....21
1.3.4 Definición de listo....23
1.3.5 Definición de terminado....23
1.3.6 Iteraciones....24
1.4 Estructura de desglose del trabajo (EDT)....24
1.5 Estimación y cronograma....28
1.5.1 Técnica de estimación....28
1.5.2 Velocidad prevista....28
1.5.3 Cronograma....29
1.6 Matriz de riesgos....30
1.7 Matriz RACI....34
2. Diseño del juego (GDD)....35
2.1 Concepto....35
2.1.1 Referentes....35

2.1.2 Propuesta de valor....36
2.2 Historia y personajes....36
2.2.1 Fichas de personajes....37
2.3 Mecánicas, ciclo central de juego y reglas....39
2.3.1 Mecánicas principales....39
2.3.2 Ciclo central de juego....40
2.3.3 Reglas del juego....42
2.3.4 Controles....42
2.4 Movimientos especiales....43
2.4.1 Comportamiento detallado....43
2.5 Niveles y progresión....44
2.5.1 Zonas del recorrido....44
2.5.2 Curva de dificultad....45
2.6 Interfaz y experiencia de usuario....46
2.6.1 Bocetos de pantalla....46
2.6.2 Flujo de pantallas....48
2.6.3 Decisiones de usabilidad....49
2.7 Arte, audio y licencias....49
2.7.1 Estilo visual....49
2.7.2 Audio....51
2.7.3 Recursos y licencias....51
3. Requisitos....53
3.1 Marco de referencia....53
3.2 Requisitos funcionales....54
3.3 Requisitos no funcionales....57
3.4 Reglas de negocio....59
3.5 Pila del producto e historias de usuario....59
3.6 Casos de uso....60
3.6.1 Especificaciones de casos de uso....62
4. Análisis y diseño....67
4.1 Arquitectura (modelo C4)....67

4.1.1 Nivel 1: contexto del sistema....67
4.1.2 Nivel 2: contenedores....68
4.1.3 Nivel 3: componentes del ejecutable....68
4.2 Patrones de diseño....72
4.2.1 Fragmentos de código....74
4.3 Modelos UML....75
4.3.1 Diagrama de clases....75
4.3.2 Diagramas de secuencia....78
4.3.3 Diagramas de estados....82
4.3.4 Diagrama de actividades....84
4.4 Escenas, señales, entradas y colisiones....85
4.4.1 Árbol de nodos de las escenas principales....85
4.4.2 Diagrama de escenas....87
4.4.3 Mapa de señales....88
4.4.4 Mapa de entradas....90
4.4.5 Capas y máscaras de colisión....91
4.5 Modelo de datos y persistencia....93
4.5.1 Datos del juego....93
4.5.2 Persistencia....94
4.6 Registros de decisiones de arquitectura (ADR)....95
5. Construcción....98
5.1 Estándar de codificación....98
5.2 Estructura de carpetas y nombres....100
5.3 Control de versiones....101
5.3.1 Estrategia de ramas....102
5.3.2 Convención de confirmaciones....103
5.3.3 Solicitudes de integración revisadas....104
5.3.4 Versiones publicadas y etiquetas....108
5.3.5 Archivos de Godot en el repositorio....108
5.3.6 Tablero....108
5.4 Gestión de configuración....108

5.5 Deuda técnica.....110
6. Calidad y pruebas.....112
6.1 Plan de pruebas.....112
6.2 Pruebas unitarias automatizadas.....113
6.3 Casos de prueba funcionales.....116
6.4 Pruebas de rendimiento.....117
6.5 Usabilidad y pruebas de juego.....119
6.6 Registro de defectos.....122
6.7 Matriz de trazabilidad.....124
7. Despliegue y cierre.....128
7.1 Manual de usuario.....128
7.1.1 Qué necesitas.....128
7.1.2 Descargar y abrir el juego.....128
7.1.3 Controles.....128
7.1.4 Objetivo y reglas.....129
7.1.5 Pantallas.....129
7.1.6 Guardado.....133
7.1.7 Solución de problemas.....134
7.1.8 Créditos.....134
7.2 Manual técnico.....134
7.2.1 Obtener el proyecto.....134
7.2.2 Instalar Godot y las plantillas de exportación.....134
7.2.3 Abrir el proyecto.....135
7.2.4 Complementos.....136
7.2.5 Ejecutar las pruebas.....136
7.2.6 Exportar el ejecutable.....137
7.2.7 Publicar la versión.....138
7.2.8 Regenerar el arte y el nivel (opcional).....138
7.3 Notas de versión.....138
7.4 Métricas de proceso.....139
7.4.1 Velocidad por iteración.....139

7.4.2 Gráfico de trabajo pendiente....139
7.4.3 Contribuciones por integrante....140
7.5 Retrospectiva y lecciones aprendidas....141
7.5.1 Trabajo futuro....142
Referencias....144
Anexo A. Historias de usuario....147
Anexo B. Casos de prueba funcionales....155
Anexo C. Datos de las sesiones de pruebas de juego....179
Anexo D. Evidencias de los eventos Scrum....181

Lista de tablas

Tabla 1 *Integrantes y aprobación del acta* 19
Tabla 2 *Registro de interesados* 20
Tabla 3 *Roles de Scrum y responsabilidades técnicas* 21
Tabla 4 *Registro de iteraciones* 24
Tabla 5 *Diccionario de la EDT* 26
Tabla 6 *Velocidad prevista por sprint* 28
Tabla 7 *Hitos del proyecto* 30
Tabla 8 *Matriz de riesgos* 31
Tabla 9 *Mapa de calor de los riesgos (probabilidad × impacto)* 32
Tabla 10 *Seguimiento de los riesgos por iteración* 33
Tabla 11 *Matriz RACI* 34
Tabla 12 *Juegos de referencia* 36
Tabla 13 *Estructura narrativa* 37
Tabla 14 *Fichas de personajes: Sir Aldric y Morvak* 38
Tabla 15 *Fichas de personajes: enemigos* 38
Tabla 16 *Mecánicas principales* 39
Tabla 17 *Mecánicas secundarias* 40
Tabla 18 *Controles* 42
Tabla 19 *Especificación de movimientos especiales* 43
Tabla 20 *Zonas del nivel Bosque Marchito* 44
Tabla 21 *Dificultad y peligros por zona* 46
Tabla 22 *Transiciones entre pantallas* 48
Tabla 23 *Paleta de colores* 50
Tabla 24 *Música y efectos de sonido* 51
Tabla 25 *Recursos y licencias* 52
Tabla 26 *Requisitos funcionales* 55
Tabla 27 *Requisitos no funcionales* 58
Tabla 28 *Reglas de negocio* 59
Tabla 29 *Resumen de la pila del producto* 60
Tabla 30 *Casos de uso y sus relaciones* 62

Tabla 31 *Especificación de CU-01: Iniciar partida (flujo entre pantallas)*.....62
Tabla 32 *Especificación de CU-02: Ejecutar movimiento especial*.....63
Tabla 33 *Especificación de CU-03: Combatir enemigo*.....64
Tabla 34 *Especificación de CU-04: Completar el nivel*.....65
Tabla 35 *Especificación de CU-05: Ajustar volumen (persistencia)*.....66
Tabla 36 *Elementos del diagrama de contexto*.....67
Tabla 37 *Contenedores del sistema*.....68
Tabla 38 *Componentes: autoloads*.....70
Tabla 39 *Componentes: escenas principales*.....72
Tabla 40 *Patrones de diseño aplicados*.....73
Tabla 41 *Mapa de señales*.....89
Tabla 42 *Mapa de entradas*.....90
Tabla 43 *Capas y máscaras de colisión*.....92
Tabla 44 *Datos del juego*.....93
Tabla 45 *Decisiones de persistencia*.....94
Tabla 46 *Esquema del archivo de configuración*.....94
Tabla 47 *Manejo de archivos ausentes o dañados*.....95
Tabla 48 *ADR-001: Gestión del estado global y de los servicios mediante autoloads*.....96
Tabla 49 *ADR-002: Formato de persistencia ConfigFile en user://config.cfg*.....96
Tabla 50 *ADR-003: Uso del renderizador Compatibility*.....97
Tabla 51 *ADR-004: Uso de GUT 9.5.0 para las pruebas unitarias*.....97
Tabla 52 *Convenciones de nombres*.....98
Tabla 53 *Estrategia de ramas*.....102
Tabla 54 *Tipos de confirmación*.....103
Tabla 55 *Rotación de revisores por sprint*.....104
Tabla 56 *Solicitudes de integración revisadas*.....105
Tabla 57 *Versiones publicadas*.....108
Tabla 58 *Elementos de configuración*.....109
Tabla 59 *Registro de deuda técnica*.....111
Tabla 60 *Tipos de prueba*.....112
Tabla 61 *Entorno de prueba*.....113

Tabla 62 *Pruebas unitarias*.....114
Tabla 63 *Resumen de los casos de prueba funcionales*.....117
Tabla 64 *Resultados de rendimiento*.....118
Tabla 65 *Resultados de las sesiones de pruebas de juego*.....120
Tabla 66 *Hallazgos de las pruebas de juego*.....122
Tabla 67 *Registro de defectos*.....123
Tabla 68 *Matriz de trazabilidad*.....125
Tabla 69 *Controles del juego*.....128
Tabla 70 *Problemas frecuentes*.....134
Tabla 71 *Velocidad por iteración*.....139
Tabla 72 *Contribuciones por integrante*.....141
Tabla 73 *Lecciones aprendidas*.....142
Tabla 74 *Historia de usuario HU-01: Moverse y saltar*.....147
Tabla 75 *Historia de usuario HU-02: Doble salto*.....147
Tabla 76 *Historia de usuario HU-03: Impulso*.....147
Tabla 77 *Historia de usuario HU-04: Slime patrullero*.....148
Tabla 78 *Historia de usuario HU-05: Puntos de control y caída al vacío*.....148
Tabla 79 *Historia de usuario HU-06: Menú principal*.....148
Tabla 80 *Historia de usuario HU-07: Pruebas automatizadas*.....149
Tabla 81 *Historia de usuario HU-08: Atacar con la espada*.....149
Tabla 82 *Historia de usuario HU-09: Vida, daño e invulnerabilidad*.....149
Tabla 83 *Historia de usuario HU-10: Murciélago perseguidor*.....150
Tabla 84 *Historia de usuario HU-11: Roca empujable*.....150
Tabla 85 *Historia de usuario HU-12: Corriente de viento*.....150
Tabla 86 *Historia de usuario HU-13: Fragmentos y puntaje*.....151
Tabla 87 *Historia de usuario HU-14: Monedas*.....151
Tabla 88 *Historia de usuario HU-15: Historia en el juego*.....151
Tabla 89 *Historia de usuario HU-16: Música y efectos de sonido*.....152
Tabla 90 *Historia de usuario HU-17: HUD*.....152
Tabla 91 *Historia de usuario HU-18: Victoria y derrota*.....152
Tabla 92 *Historia de usuario HU-19: Pausa y reinicio*.....153

Tabla 93 *Historia de usuario HU-20: Opciones de volumen*.....153
Tabla 94 *Historia de usuario HU-21: Guardar mejor puntaje y volumen*.....153
Tabla 95 *Historia de usuario HU-22: Créditos*.....154
Tabla 96 *Historia de usuario HU-23: Jugar sin instalar Godot*.....154
Tabla 97 *Caso de prueba CP-01: Moverse y saltar*.....155
Tabla 98 *Caso de prueba CP-02: Doble salto*.....156
Tabla 99 *Caso de prueba CP-03: Impulso con enfriamiento*.....157
Tabla 100 *Caso de prueba CP-04: Atacar con la espada*.....158
Tabla 101 *Caso de prueba CP-05: Daño, invulnerabilidad y derrota*.....159
Tabla 102 *Caso de prueba CP-06: Caída al vacío y punto de control*.....160
Tabla 103 *Caso de prueba CP-07: Slime patrullero*.....161
Tabla 104 *Caso de prueba CP-08: Murciélago perseguidor*.....162
Tabla 105 *Caso de prueba CP-09: Roca empujable*.....163
Tabla 106 *Caso de prueba CP-10: Corriente de viento*.....165
Tabla 107 *Caso de prueba CP-11: Recoger fragmentos*.....166
Tabla 108 *Caso de prueba CP-12: Recoger monedas*.....167
Tabla 109 *Caso de prueba CP-13: Santuario y victoria*.....168
Tabla 110 *Caso de prueba CP-14: HUD*.....170
Tabla 111 *Caso de prueba CP-15: Pausa y reinicio*.....171
Tabla 112 *Caso de prueba CP-16: Historia en el juego*.....173
Tabla 113 *Caso de prueba CP-17: Música y efectos*.....174
Tabla 114 *Caso de prueba CP-18: Menú, opciones y créditos*.....175
Tabla 115 *Caso de prueba CP-19: Persistencia del récord y del volumen*.....176
Tabla 116 *Caso de prueba CP-20: Ejecutable sin Godot*.....178
Tabla 117 *Sesión 1 de pruebas de juego*.....179
Tabla 118 *Sesión 2 de pruebas de juego*.....179
Tabla 119 *Sesión 3 de pruebas de juego*.....180
Tabla 120 *Sesión 4 de pruebas de juego*.....180
Tabla 121 *Eventos Scrum realizados*.....181

Lista de figuras

Figura 1 *Tablero de GitHub Projects al inicio del Sprint 1* 22
Figura 2 *Tablero de GitHub Projects al cierre del Sprint 3* 23
Figura 3 *Estructura de desglose del trabajo del proyecto* 25
Figura 4 *Cronograma del proyecto del 19 al 23 de octubre de 2026* 29
Figura 5 *La historia dentro del juego: introducción, cartel y final* 37
Figura 6 *Imágenes de los personajes* 39
Figura 7 *Diagrama de actividades del ciclo central de juego* 41
Figura 8 *Mapa del nivel Bosque Marchito con sus zonas y objetos* 45
Figura 9 *Curva de dificultad por zona* 46
Figura 10 *Bocetos de las pantallas del juego* 47
Figura 11 *Diagrama de flujo de pantallas* 48
Figura 12 *Tablero de referencias visuales y paleta de colores* 50
Figura 13 *Diagrama de casos de uso* 61
Figura 14 *Diagrama C4 de contexto* 67
Figura 15 *Diagrama C4 de contenedores* 68
Figura 16 *Diagrama C4 de componentes* 69
Figura 17 *Autoloads en Proyecto › Configuración del Proyecto › Globales* 71
Figura 18 *Diagrama de clases: autoloads* 76
Figura 19 *Diagrama de clases: clases de escena* 77
Figura 20 *Diagrama de secuencia DS-01: ejecución del impulso* 79
Figura 21 *Diagrama de secuencia DS-02: daño y derrota del jugador* 80
Figura 22 *Diagrama de secuencia DS-03: recolección de un fragmento y actualización del HUD* 81
Figura 23 *Diagrama de secuencia DS-04: completar el nivel y guardar el mejor puntaje* 82
Figura 24 *Diagrama de estados DE-01: caballero* 83
Figura 25 *Diagrama de estados DE-02: murciélago* 83
Figura 26 *Diagrama de estados slime* 84
Figura 27 *Diagrama de estados DE-03: ciclo del juego* 84
Figura 28 *Árbol de nodos de las escenas principales* 86
Figura 29 *Diagrama de escenas (composición)* 87

Figura 30 *Mapa de entrada (primera parte)*.....90
Figura 31 *Mapa de entrada (segunda parte)*.....91
Figura 32 *Nombres de las capas de física 2D*.....93
Figura 33 *Editor sin errores ni advertencias después de ejecutar el juego*.....100
Figura 34 *Regla de protección de la rama main en GitHub*.....103
Figura 35 *Solicitud de integración #33 aprobada por el revisor*.....107
Figura 36 *Gestor de plantillas de exportación con 4.5.stable instaladas*.....110
Figura 37 *Ejecución de las 42 pruebas GUT por línea de comandos*.....116
Figura 38 *Panel Depurador › Monitores con FPS, tiempo de proceso y de física*.....118
Figura 39 *Panel Monitores durante el escenario 2 (zona 5)*.....119
Figura 40 *Perfilador con el tiempo de cuadro, de proceso y de física*.....119
Figura 41 *Pantalla de victoria del Usuario 4 (sesión 4)*.....121
Figura 42 *Evidencia del defecto DF-05*.....124
Figura 43 *Pantalla: menú principal*.....129
Figura 44 *Pantalla: opciones*.....130
Figura 45 *Pantalla: introducción*.....130
Figura 46 *Pantalla: juego y HUD*.....131
Figura 47 *Pantalla: pausa*.....131
Figura 48 *Pantalla: victoria*.....132
Figura 49 *Pantalla: derrota*.....132
Figura 50 *Pantalla: créditos*.....133
Figura 51 *Administrador de proyectos de Godot con el botón Importar y el proyecto abierto*..135
Figura 52 *Complemento GUT 9.5.0 activado en la pestaña Plugins*.....136
Figura 53 *Diálogo Exportar con la configuración «Windows Desktop»*.....137
Figura 54 *Gráfico de trabajo pendiente del proyecto*.....140
Figura 55 *Evidencia del caso CP-01 (E-01.png)*.....155
Figura 56 *Evidencia del caso CP-02 (E-02.png)*.....156
Figura 57 *Evidencia del caso CP-03 (E-03.png)*.....157
Figura 58 *Evidencia del caso CP-04 (E-04_E-05.png)*.....158
Figura 59 *Evidencia del caso CP-05 (E-04_E-05.png)*.....159
Figura 60 *Evidencia del caso CP-05 (E-05_derrota.png)*.....160

Figura 61 *Evidencia del caso CP-06 (E-06.png)*.....161
Figura 62 *Evidencia del caso CP-07 (E-07.png)*.....162
Figura 63 *Evidencia del caso CP-08 (E-08.png)*.....163
Figura 64 *Evidencia del caso CP-09 (E-09_reintento_empujando.png)*.....164
Figura 65 *Evidencia del caso CP-09 (E-09_sobre_la_roca.png)*.....164
Figura 66 *Evidencia del caso CP-09 (E-09_en_la_cornisa.png)*.....165
Figura 67 *Evidencia del caso CP-10 (E-10.png)*.....166
Figura 68 *Evidencia del caso CP-11 (E-11.png)*.....167
Figura 69 *Evidencia del caso CP-12 (E-12.png)*.....168
Figura 70 *Evidencia del caso CP-13 (E-13_3de5.png)*.....169
Figura 71 *Evidencia del caso CP-13 (E-13_final.png)*.....169
Figura 72 *Evidencia del caso CP-13 (E-13_victoria.png)*.....170
Figura 73 *Evidencia del caso CP-14 (E-04_E-05.png)*.....171
Figura 74 *Evidencia del caso CP-15 (E-15_pausa.png)*.....172
Figura 75 *Evidencia del caso CP-15 (E-15_reinicio.png)*.....172
Figura 76 *Evidencia del caso CP-16 (E-16_intro.png)*.....173
Figura 77 *Evidencia del caso CP-16 (E-07.png)*.....174
Figura 78 *Evidencia del caso CP-18 (E-18_opciones.png)*.....175
Figura 79 *Evidencia del caso CP-18 (E-18_creditos.png)*.....176
Figura 80 *Evidencia del caso CP-19 (E-13_victoria.png)*.....177
Figura 81 *Evidencia del caso CP-19 (E-19_menu_record.png)*.....177
Figura 82 *Evidencia del caso CP-20 (E-20_carpeta_aparte.png)*.....178
Figura 83 *Llamada de la retrospectiva del 22 de octubre*.....182

Glosario de términos y siglas

Siglas

ADR. Registro de decisión de arquitectura (Architecture Decision Record).

APA. American Psychological Association; normas de citación y presentación usadas en este documento (7.ª edición).

CU. Caso de uso.

C4. Modelo de diagramas de arquitectura en cuatro niveles: contexto, contenedores, componentes y código.

DE. Diagrama de estados.

DS. Diagrama de secuencia.

EDT. Estructura de desglose del trabajo.

FESC. Fundación de Estudios Superiores Comfanorte.

FPS. Fotogramas por segundo; medida de la fluidez con que se dibuja el juego.

GDD. Documento de diseño del juego (Game Design Document).

GUT. Godot Unit Test; complemento de pruebas unitarias para Godot, versión 9.5.0.

HU. Historia de usuario.

HUD. Interfaz superpuesta durante la partida que muestra corazones, fragmentos y puntaje.

MoSCoW. Técnica de priorización con cuatro niveles: obligatoria, importante, deseable y excluida.

RACI. Matriz de responsabilidades: responsable, aprobador, consultado e informado.

RF. Requisito funcional.

RN. Regla de negocio.

RNF. Requisito no funcional.

SUS. Escala de usabilidad del sistema (System Usability Scale).

UML. Lenguaje unificado de modelado, versión 2.5.1.

Términos

Autoload. Script de Godot que se carga una sola vez al iniciar el juego y queda disponible para todas las escenas; en el proyecto se usan cuatro: EstadoJuego, GestorEscenas, GestorAudio y GestorGuardado.

Capa de colisión. Grupo al que pertenece un cuerpo físico en Godot; define qué objetos pueden chocar entre sí. El proyecto usa ocho capas con nombre.

Ciclo central de juego. Secuencia de acciones que el jugador repite durante toda la partida: explorar, enfrentar enemigos, recoger fragmentos y avanzar.

Confirmación. Registro de un conjunto de cambios en el historial de Git, con un mensaje que describe el cambio.

Conjunto de casillas. Imagen dividida en piezas cuadradas con las que se dibuja el escenario del nivel.

Definición de terminado. Lista de condiciones que debe cumplir una historia de usuario para considerarse completa.

Developer. Rol de Scrum: integrante que construye el incremento del producto en cada sprint.

Ejecutable. Versión del juego exportada desde Godot que se abre sin tener instalado el editor.

Escena. Archivo de Godot (.tscn) que agrupa nodos y puede reutilizarse dentro de otras escenas.

Etiqueta. Marca con nombre (por ejemplo, v1.0.0) que identifica una versión concreta en el historial de Git.

Fusión. Integración de los cambios de una rama en otra, normalmente en la rama principal main.

Historia de usuario. Descripción breve de una funcionalidad desde el punto de vista del jugador, con sus criterios de aceptación.

Imagen del personaje. Dibujo o secuencia de dibujos que representa a un personaje u objeto en pantalla.

Impulso. Movimiento especial del caballero que lo desplaza rápidamente en horizontal durante un instante.

Maqueta gris. Versión preliminar del nivel construida con formas simples para probar la jugabilidad antes del arte final.

Matriz de trazabilidad. Tabla que relaciona cada requisito con su diseño, su código y sus pruebas, para comprobar que todo lo documentado está en el juego.

Máquina de estados. Patrón de diseño en el que un objeto se comporta según el estado en que se encuentra y cambia de estado ante eventos.

Nodo. Elemento básico de Godot; las escenas se construyen como árboles de nodos.

Pila del producto. Lista priorizada de todo el trabajo que se hará en el producto.

Pila del sprint. Historias de usuario seleccionadas para un sprint, con sus tareas.

Plataformero. Género de videojuego en el que el personaje salta entre plataformas y evita obstáculos.

Product Owner. Rol de Scrum responsable de maximizar el valor del producto y de ordenar la pila del producto.

Prueba unitaria. Prueba automatizada que verifica una parte pequeña del código de forma aislada.

Pruebas de juego. Sesiones en las que personas ajenas al equipo juegan para detectar problemas de jugabilidad y usabilidad.

Punto de control. Estandarte del nivel que, al tocarlo, guarda la posición a la que vuelve el caballero tras caer al vacío.

Puntos de historia. Unidad relativa con la que el equipo estima el esfuerzo de una historia de usuario.

Rama. Línea de desarrollo independiente en Git donde se trabaja un cambio sin afectar la rama principal.

Retroalimentación. Respuesta del juego o de los usuarios que informa sobre el resultado de una acción.

Scrum Master. Rol de Scrum que facilita los eventos, elimina impedimentos y vela por que el equipo aplique el marco.

Señal. Mecanismo de Godot para avisar que algo ocurrió sin que el emisor conozca a sus receptores (patrón observador).

Solicitud de integración. Petición en GitHub para fusionar una rama en main, revisada y aprobada por otro integrante.

Sprint. Iteración de Scrum de duración fija en la que se construye un incremento del producto.

Versión publicada. Entrega oficial del juego en GitHub, asociada a una etiqueta y con el ejecutable adjunto.

1. Inicio y planeación

1.1 Acta de constitución del proyecto

Proyecto: Desarrollo y documentación del videojuego *Fragmentos de Luz*
Juego: *Fragmentos de Luz*, plataformero de acción 2D
Asignatura: Diseño Multimedia y Videojuegos, Ingeniería de Software, semestre IX
Patrocinador y cliente: Yeison Sanabria Ascanio (docente)
Fecha de inicio: 19 de octubre de 2026
Fecha de entrega: 23 de octubre de 2026, hasta las 5:00 p. m.

1.1.1 Objetivo general

Desarrollar el videojuego 2D *Fragmentos de Luz* en Godot 4.5, con un nivel completo y su documentación de ingeniería de software trazable al ejecutable de la versión 1.0.0.

1.1.2 Objetivos específicos

1. Documentar la planeación, el diseño del juego, los requisitos y la arquitectura del proyecto, con los mínimos exigidos por el parcial para un grupo de cuatro integrantes.
2. Implementar un nivel completo que integre música, física, doble salto, impulso, dinámica de juego e historia, con el flujo completo de pantallas y el código integrado mediante solicitudes de integración revisadas.
3. Verificar el ejecutable de la versión 1.0.0 con pruebas unitarias, casos funcionales, pruebas de rendimiento y pruebas de juego con usuarios externos, hasta alcanzar el 100 % de cobertura de requisitos en la matriz de trazabilidad.

1.1.3 Justificación

El proyecto permite aplicar, sobre un producto real, el ciclo de vida de la ingeniería de software (planeación, requisitos, diseño, construcción, pruebas y despliegue) y demostrar que la documentación y el software pueden mantenerse coherentes y verificables. Un plataformero de acción permite trabajar física, control de personaje, inteligencia básica de enemigos, interfaz, audio y narrativa en un juego pequeño, que se puede terminar en cinco días, y el equipo ya tiene experiencia previa con Godot 2D en los talleres del curso.

1.1.4 Alcance de la versión 1.0.0

1. Menú principal con las opciones Jugar, Opciones, Créditos y Salir, y el mejor puntaje guardado.
2. Introducción narrativa en texto antes del nivel.
3. Un nivel completo (Bosque Marchito) construido con `TileMapLayer`.
4. Caballero con caminar, salto, doble salto, impulso y ataque con espada.
5. Sistema de vida de tres corazones con invulnerabilidad temporal.
6. Puntos de control de reaparición.
7. Enemigo slime patrullero y enemigo murciélago perseguidor.
8. Obstáculos: púas y caídas al vacío.
9. Roca empujable (`RigidBody2D`) y corriente de viento (`Area2D`).
10. Cinco fragmentos obligatorios y monedas opcionales.
11. Sistema de puntaje con bono por corazones restantes.
12. Santuario con condición de victoria.
13. Cinemática de cierre.
14. HUD con corazones, fragmentos y puntaje.
15. Menú de pausa (continuar, reiniciar, opciones y salir) y panel de opciones con el volumen de la música y de los efectos, accesible desde el menú principal y desde la pausa.
16. Pantallas de victoria y derrota (reintentar y menú principal).
17. Pantalla de créditos con los recursos de terceros y sus licencias.
18. Música de fondo en el menú y en el nivel, y efectos de sonido de las acciones principales.
19. Guardado del mejor puntaje y del volumen de la música y de los efectos.
20. Ejecutable exportado para Windows (x86_64).

1.1.5 Fuera de alcance

Combate con Morvak como jefe final; más de un nivel; guardado de la partida a mitad del nivel; reasignación de teclas; soporte de mando; versiones web, Linux y móviles; multijugador; y doblaje o narración por voz.

1.1.6 Supuestos

Los supuestos son condiciones que el equipo da por ciertas al iniciar el proyecto y de las que depende el plan; si alguna no se cumple, se convierte en un riesgo (sección 1.6).

- Los cuatro integrantes disponen de un computador con Windows 10 o posterior capaz de ejecutar Godot 4.5.
- Los cuatro integrantes tienen cuenta de GitHub.
- Existen recursos de arte y audio con licencia libre (CC0 o CC-BY) suficientes para la ambientación de fantasía.
- Es posible conseguir cinco usuarios externos para las pruebas de juego entre el 22 y el 23 de octubre.

1.1.7 Restricciones

- Motor Godot 4.5-stable con GDScript.
- Plazo de cinco días calendario.
- Equipo de cuatro personas.
- Solo recursos con licencia libre o propios, con el uso de inteligencia artificial declarado.
- Repositorio público.
- Plantillas y normas del documento del parcial; documento en normas APA 7 (American Psychological Association, 2020).

1.1.8 Criterios de éxito

1. El ejecutable de la versión 1.0.0 se abre en un equipo sin Godot y permite terminar el nivel sin errores.
2. Los cinco elementos obligatorios del nivel (música, física, movimientos especiales, dinámica de juego e historia) están presentes y son demostrables.
3. El 100 % de los requisitos tiene una prueba con resultado en la matriz de trazabilidad.
4. El 100 % de las pruebas unitarias se aprueba sobre la versión 1.0.0.
5. Se cumplen todos los mínimos del parcial para un grupo de cuatro integrantes.
6. La entrega se realiza antes de la hora límite.

1.1.9 Integrantes y aprobación

La Tabla 1 registra a las personas que aprueban el acta.

Tabla 1

Integrantes y aprobación del acta

Nombre	Rol	Firma
Jaider Villamizar	Product Owner y Developer	
Camilo Vargas	Scrum Master y Developer	
Kevin Yaruro	Developer	
Diego Villamizar	Developer	
Yeison Sanabria Ascanio	Patrocinador y cliente	

Nota. Elaboración propia.

Fecha de aprobación: 19 de octubre de 2026

1.2 Interesados

La Tabla 2 identifica a los interesados del proyecto, su influencia (alta, media o baja) y la estrategia con que se gestiona cada uno.

Tabla 2

Registro de interesados

Interesado	Rol en el proyecto	Interés o expectativa	Influencia	Estrategia de gestión
Yeison Sanabria Ascanio	Docente, cliente y patrocinador	Recibir un juego funcional y una documentación coherente con el ejecutable que cumpla las condiciones del parcial	Alta	Gestionar de cerca: validar el acta, consultar dudas del enunciado y presentarle las versiones publicadas intermedias
Jaider Villamizar	Product Owner y Developer (sistema del jugador)	Que el producto cumpla el alcance y los requisitos aprobados y que el jugador responda con precisión	Alta	Gestionar de cerca: ordena la pila del producto, acepta las historias en la revisión del sprint y participa en todos los eventos
Camilo Vargas	Scrum Master y Developer (nivel, narrativa y audio)	Que el equipo cumpla los sprints sin bloqueos y que el nivel y la historia queden completos	Alta	Gestionar de cerca: facilita los eventos, mantiene el tablero y registra las evidencias
Kevin Yaruro	Developer (enemigos y física)	Entregar enemigos y física estables y bien probados, y aprobar su defensa técnica	Media	Mantener informado y comprometido: reuniones diarias, revisión cruzada de solicitudes de integración y tareas asignadas en el tablero
Diego Villamizar	Developer (interfaz y persistencia)	Entregar un flujo de pantallas y un guardado sin errores, y aprobar su defensa técnica	Media	Mantener informado y comprometido: reuniones diarias, revisión cruzada de solicitudes de integración y tareas asignadas en el tablero
Jugadores casuales de 10 años en adelante	Público objetivo	Un juego corto, entretenido, con controles claros y sin contenido inapropiado	Media	Mantener satisfechos: sus necesidades guían el GDD y se representan con los usuarios de las pruebas de juego
Usuarios externos de prueba (cinco personas ajenas al grupo)	Participantes de las pruebas de juego	Probar un juego que funcione, con instrucciones claras y en una sesión corta	Media	Mantener informados: protocolo con consentimiento, sesiones de 15 minutos y comunicación de los cambios hechos con su retroalimentación
Autores de los recursos de terceros	Proveedores de arte, música y efectos con licencia libre	Que se respeten sus licencias y se les dé la atribución exigida	Baja	Monitorear: registrar cada recurso en la tabla de licencias, el README y la pantalla de créditos

Nota. Elaboración propia.

1.3 Metodología, roles e iteraciones

1.3.1 Marco de trabajo

El proyecto se gestiona con Scrum (Schwaber y Sutherland, 2020). Se eligió frente a Kanban por tres razones: el plazo de cinco días se divide naturalmente en iteraciones cortas con un objetivo cada una; el curso pide medir la velocidad por iteración y el trabajo pendiente, que son métricas propias de Scrum; y un equipo de cuatro personas permite asignar los roles sin sobrecargar a nadie.

1.3.2 Roles

La Tabla 3 asigna los roles de Scrum y la responsabilidad técnica de cada integrante.

Tabla 3

Roles de Scrum y responsabilidades técnicas

Integrante	Rol en Scrum	Responsabilidad técnica principal
Jaider Villamizar	Product Owner y Developer	Sistema del jugador
Camilo Vargas	Scrum Master y Developer	Nivel, narrativa y audio
Kevin Yaruro	Developer	Enemigos y física
Diego Villamizar	Developer	Interfaz y persistencia
Yeison Sanabria Ascanio (docente)	Cliente y patrocinador	Aprueba el acta y define las restricciones

Nota. Elaboración propia.

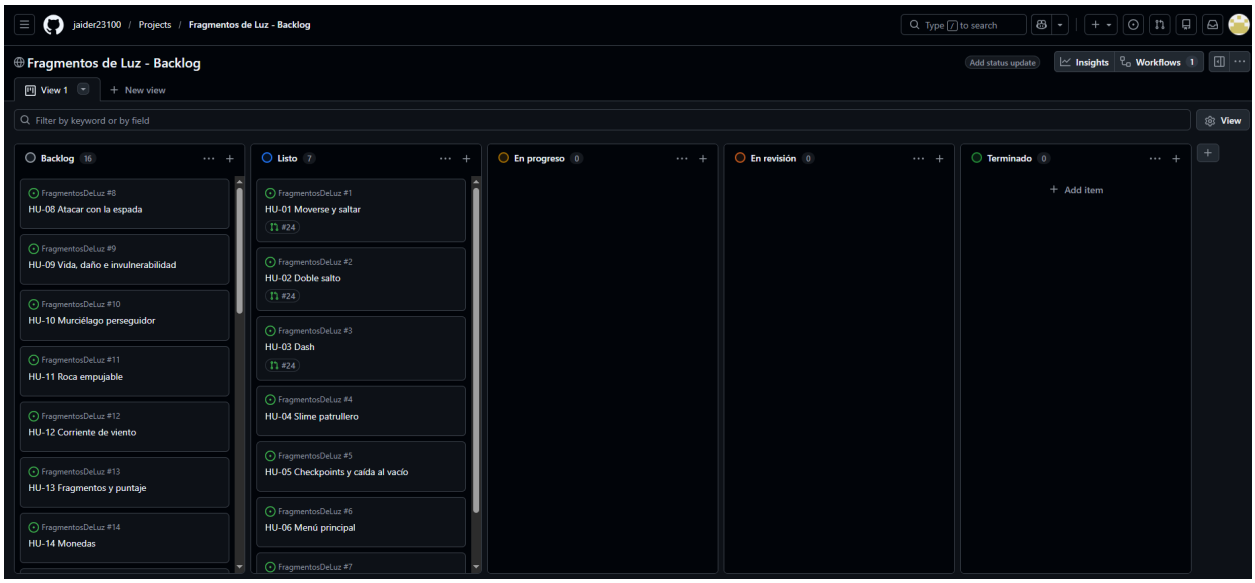
1.3.3 Forma de trabajo

- **Duración de los sprints:** de 1,5 a 2 días calendario; tres sprints entre el 19 y el 23 de octubre de 2026.
- **Eventos:** planificación del sprint de 15 minutos al inicio de cada sprint; reunión diaria de 10 minutos por la mañana y por la tarde; revisión del sprint y retrospectiva de 15 minutos cada una al cierre. De cada evento queda un acta breve (fecha, asistentes y decisiones) en el Anexo D.
- **Tablero:** GitHub Projects del repositorio, con las columnas Pila del producto, Listo, En progreso, En revisión y Terminado.
- **Comunicación:** grupo de WhatsApp y reuniones presenciales o por Google Meet.

La Figura 1 muestra el tablero al inicio del Sprint 1, con las historias del sprint en Listo y el resto en la pila del producto.

Figura 1

Tablero de GitHub Projects al inicio del Sprint 1

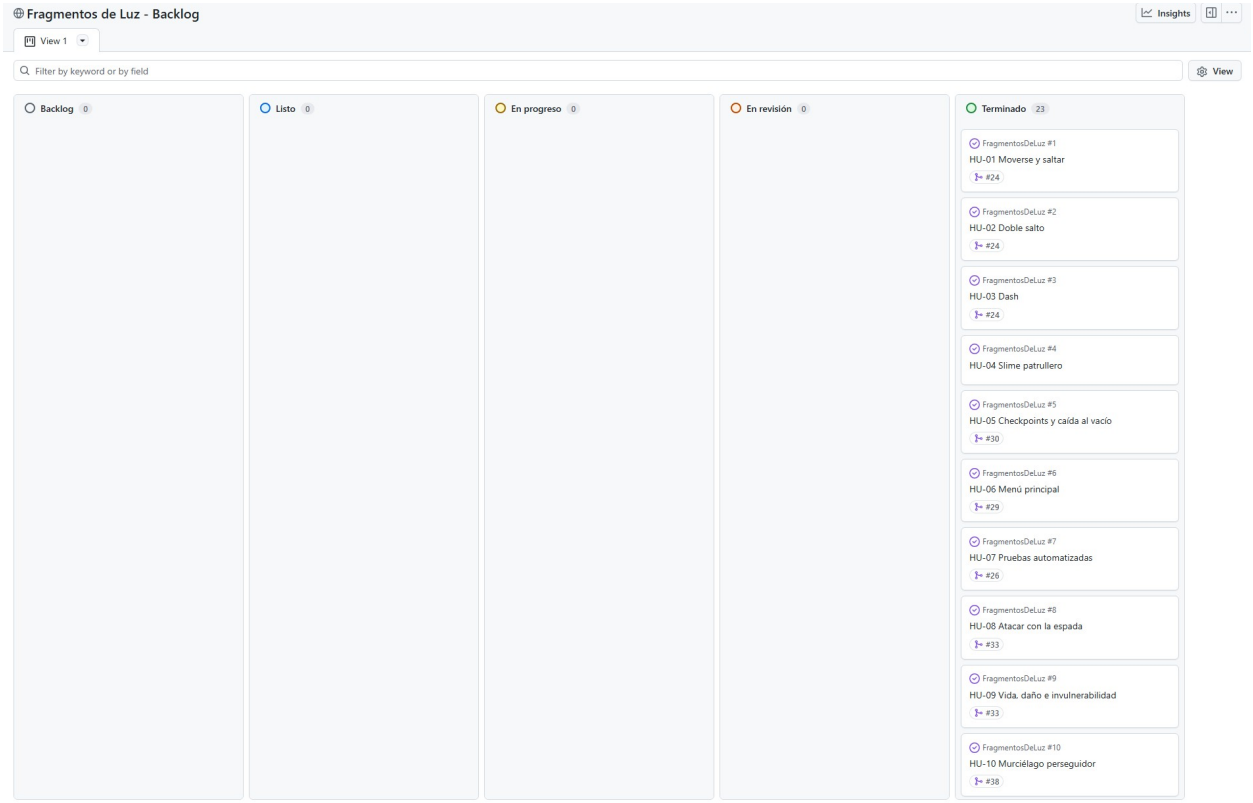


Nota. Captura de GitHub Projects, 19 oct. 2026.

La Figura 2 muestra el tablero al cierre del proyecto: las 23 historias en Terminado.

Figura 2

Tablero de GitHub Projects al cierre del Sprint 3



Nota. Captura de GitHub Projects, 23 oct. 2026.

1.3.4 Definición de listo

Una historia puede entrar al sprint cuando: (1) está escrita en el formato «Como…, quiero…, para…»; (2) tiene al menos un criterio de aceptación Dado / Cuando / Entonces; (3) está estimada en puntos de historia; (4) tiene responsable; y (5) sus dependencias están resueltas.

1.3.5 Definición de terminado

Una historia está terminada cuando: (1) su código está en `main` mediante una solicitud de integración aprobada por otro integrante; (2) sigue la guía de estilo de GDScript con tipado estático; (3) el proyecto se ejecuta con F5 sin errores en el panel Depurador; (4) sus pruebas unitarias o su caso funcional están aprobados; (5) está documentada en el capítulo correspondiente y en la matriz de trazabilidad; y (6) el Product Owner la aceptó en la revisión del sprint.

1.3.6 Iteraciones

La Tabla 4 resume el objetivo de cada sprint, las historias comprometidas y la versión publicada con que cierra.

Tabla 4

Registro de iteraciones

Iteración	Fechas	Objetivo	HU comprometidas (puntos)	HU terminadas (puntos)	Versión publicada
Sprint 1	19 y 20 oct. 2026	Base jugable: repositorio, GUT, caballero con doble salto e impulso, nivel en maqueta gris, menú y gestor de escenas	HU-01 a HU-07 (25)	HU-01 a HU-07 (25)	v0.1.0
Sprint 2	21 y 22 oct. 2026 (hasta el mediodía)	Juego completo: enemigos, física, fragmentos, HUD, pausa, victoria y derrota, audio y narrativa	HU-08 a HU-19 (37)	HU-08 a HU-19 (37)	v0.2.0
Sprint 3	22 (tarde) y 23 oct. 2026	Calidad y entrega: pruebas de juego, pruebas, rendimiento, opciones, persistencia, créditos, exportación, video y documento final	HU-20 a HU-23 (10)	HU-20 a HU-23 (10)	v1.0.0

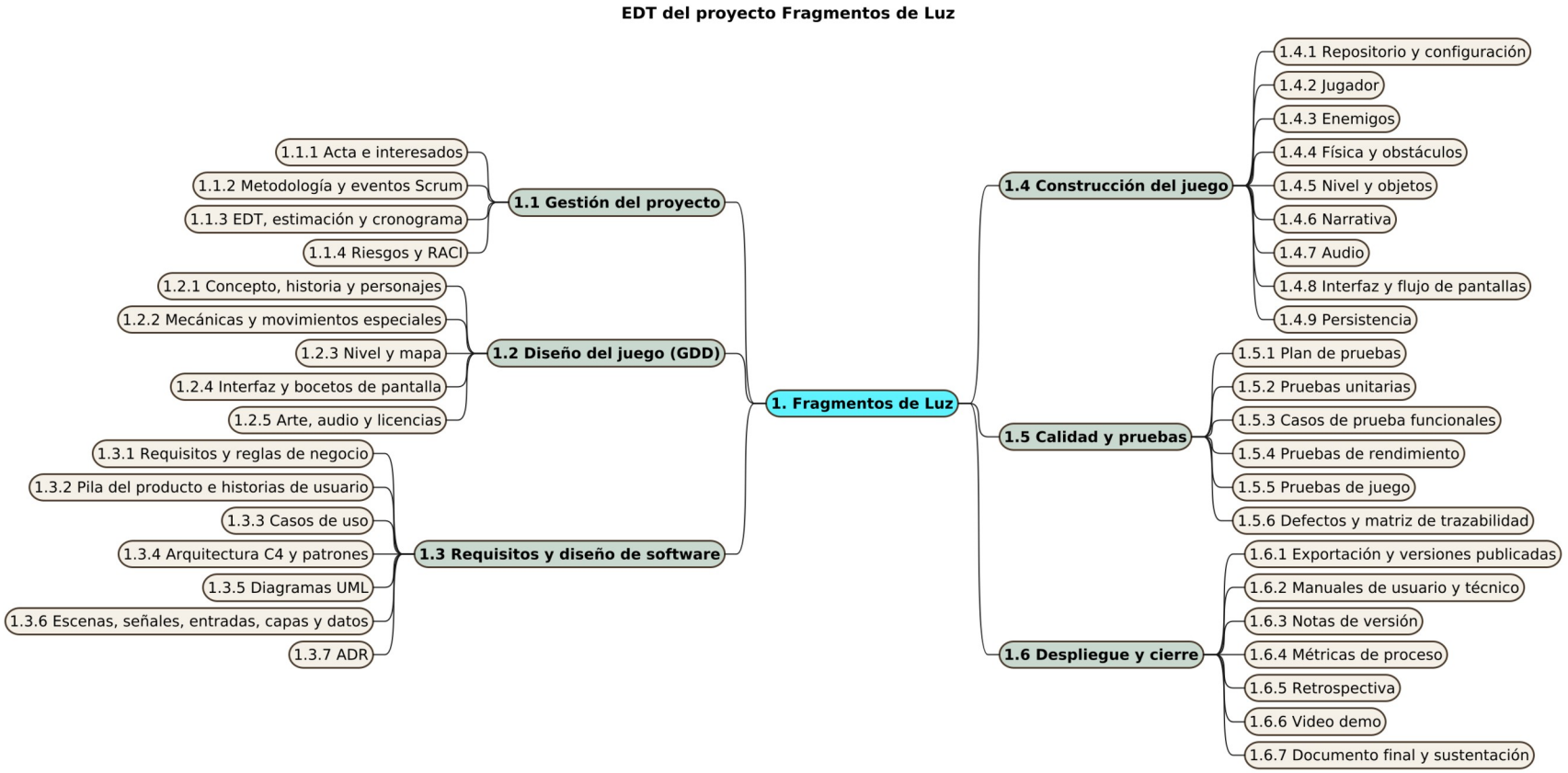
Nota. Elaboración propia.

1.4 Estructura de desglose del trabajo (EDT)

La Figura 3 descompone el proyecto en seis paquetes de trabajo de primer nivel y 38 paquetes de segundo nivel. La Tabla 5 describe cada paquete, el entregable que produce y su responsable. La fuente del diagrama está en `docs/diagramas/edt.puml`.

Figura 3

Estructura de desglose del trabajo del proyecto



Nota. Elaboración propia.

Tabla 5

Diccionario de la EDT

Código	Paquete	Descripción	Entregable que produce	Responsable
1.1.1	Acta e interesados	Redactar el acta de constitución y el registro de interesados	Capítulo 1 (1.1 y 1.2)	Jaider
1.1.2	Metodología y eventos Scrum	Definir los roles y las definiciones de listo y de terminado, y registrar las actas de planificación, revisión y retrospectiva	Capítulo 1 (1.3), anexo de evidencias	Camilo
1.1.3	EDT, estimación y cronograma	Construir la EDT, estimar con póquer de planificación y elaborar el cronograma con hitos	Capítulo 1 (1.4 y 1.5)	Camilo
1.1.4	Riesgos y RACI	Identificar los riesgos, darles seguimiento y asignar responsabilidades	Capítulo 1 (1.6 y 1.7)	Camilo
1.2.1	Concepto, historia y personajes	Concepto, referentes, narrativa y fichas de personajes	Capítulo 2 (2.1 y 2.2)	Camilo
1.2.2	Mecánicas y movimientos especiales	Ciclo central de juego, reglas, controles y especificación del impulso y del doble salto	Capítulo 2 (2.3 y 2.4)	Jaider
1.2.3	Nivel y mapa	Mapa del Bosque Marchito con enemigos, objetos y puntos de control, y su curva de dificultad	Capítulo 2 (2.5)	Camilo
1.2.4	Interfaz y bocetos de pantalla	Bocetos de cada pantalla y flujo de pantallas	Capítulo 2 (2.6)	Diego
1.2.5	Arte, audio y licencias	Estilo, paleta, lista de audio y tabla de licencias	Capítulo 2 (2.7)	Camilo
1.3.1	Requisitos y reglas de negocio	19 RF, 8 RNF medibles y 10 RN en formato de especificación de requisitos	Capítulo 3 (3.1 a 3.4)	Jaider
1.3.2	Pila del producto e historias de usuario	23 HU con criterios Dado / Cuando / Entonces y estimación	Capítulo 3 (3.5), tablero	Jaider
1.3.3	Casos de uso	Diagrama de casos de uso y 5 especificaciones completas	Capítulo 3 (3.6)	Jaider
1.3.4	Arquitectura C4 y patrones	Niveles 1, 2 y 3 de C4 y los 4 patrones con su código	Capítulo 4 (4.1 y 4.2)	Diego
1.3.5	Diagramas UML	Clases, 4 secuencias, 4 estados y actividades	Capítulo 4 (4.3)	Jaider
1.3.6	Escenas, señales, entradas, capas y datos	Árbol de escenas, mapa de señales, mapa de entradas, capas y persistencia	Capítulo 4 (4.4 y 4.5)	Diego
1.3.7	ADR	4 registros de decisiones de arquitectura	Capítulo 4 (4.6), `docs/adr/`	Kevin
1.4.1	Repositorio y configuración	Estructura, `.gitignore`, ramas, protección de `main`, GUT y configuración de exportación	Repositorio, capítulo 5	Diego
1.4.2	Jugador	Caballero con movimiento, doble salto, impulso, espada, vida y máquina de estados	`caballero.tscn`, `caballero.gd`	Jaider
1.4.3	Enemigos	Slime patrullero y murciélago perseguidor con sus estados	Escenas y scripts de los enemigos	Kevin
1.4.4	Física y obstáculos	Roca `RigidBody2D`, viento `Area2D`, púas y capas de colisión	Escenas de física, configuración de capas	Kevin
1.4.5	Nivel y objetos	`TileMapLayer`, fragmentos, monedas, puntos de control y santuario	`nivel_bosque.tscn` y escenas de objetos	Camilo
1.4.6	Narrativa	Introducción, carteles y cinemática de cierre	Escenas de introducción y final	Camilo
1.4.7	Audio	Música, efectos de sonido y buses de audio	`assets/audio/`, `assets/musica/`, `GestorAudio`	Camilo

Código	Paquete	Descripción	Entregable que produce	Responsable
1.4.8	Interfaz y flujo de pantallas	Menú, HUD, pausa, opciones, victoria, derrota, créditos y gestor de escenas	Escenas de interfaz, autoloads	Diego
1.4.9	Persistencia	Guardado del mejor puntaje y del volumen en `user://config.cfg`	`GestorGuardado`	Diego
1.5.1	Plan de pruebas	Plan según ISO/IEC/IEEE 29119-3, con criterios de entrada y salida	Capítulo 6 (6.1)	Kevin
1.5.2	Pruebas unitarias	42 pruebas con GUT y su reporte XML	`test/unit/`, `docs/reporte_gut.xml`	Todos (coordina Diego)
1.5.3	Casos de prueba funcionales	20 casos ejecutados sobre el ejecutable	Capítulo 6 (6.3)	Kevin
1.5.4	Pruebas de rendimiento	2 escenarios con 3 mediciones cada uno, con los paneles Monitores y Perfilador	Capítulo 6 (6.4)	Diego
1.5.5	Pruebas de juego	Sesiones con 5 usuarios externos, cuestionario SUS y hallazgos	Capítulo 6 (6.5), anexos	Camilo
1.5.6	Defectos y matriz de trazabilidad	Registro de defectos y matriz de trazabilidad con su cobertura	Capítulo 6 (6.6 y 6.7)	Jaider
1.6.1	Exportación y versiones publicadas	Ejecutables de Windows y versiones publicadas v0.1.0, v0.2.0 y v1.0.0	Versiones publicadas en GitHub	Diego
1.6.2	Manuales de usuario y técnico	Manual de usuario y manual técnico	Capítulo 7 (7.1 y 7.2)	Kevin
1.6.3	Notas de versión	`CHANGELOG.md` en formato Keep a Changelog	Repositorio, capítulo 7 (7.3)	Diego
1.6.4	Métricas de proceso	Velocidad, trabajo pendiente y contribuciones por integrante	Capítulo 7 (7.4)	Camilo
1.6.5	Retrospectiva	Retrospectiva final y 5 lecciones aprendidas	Capítulo 7 (7.5)	Camilo
1.6.6	Video demo	Video de 3 a 5 minutos del nivel completo	Enlace en la portada y en la versión publicada	Jaider
1.6.7	Documento final y sustentación	Ensamblar el PDF en APA 7 y preparar la sustentación	`Parcial_Grupo2_FragmentosDeLuz.pdf`	Jaider

Nota. Elaboración propia.

1.5 Estimación y cronograma

1.5.1 Técnica de estimación

Las historias de usuario se estiman en puntos de historia con la escala de Fibonacci modificada (1, 2, 3, 5, 8 y 13) mediante póquer de planificación (Cohn, 2004): (1) el Product Owner lee la historia y sus criterios de aceptación; (2) cada integrante elige su estimación en secreto, con cartas o con un mensaje simultáneo por WhatsApp; (3) si hay diferencias, explican su voto quienes dieron el valor más alto y el más bajo; y (4) se repite hasta llegar a un consenso, con un máximo de tres rondas; si no lo hay, se toma la mediana.

Como referencia de la escala, un punto equivale a una tarea trivial de menos de una hora, por ejemplo la pantalla de créditos. Una historia de 13 puntos es demasiado grande y se divide antes de entrar al sprint.

1.5.2 Velocidad prevista

La capacidad estimada es de cuatro integrantes con unas seis horas efectivas por día cada uno, sin contar el tiempo de documentación. La Tabla 6 distribuye los 72 puntos de las 23 historias de la pila del producto (sección 3.5) entre los tres sprints.

Tabla 6

Velocidad prevista por sprint

Sprint	Días	Velocidad prevista (puntos)	Razón
Sprint 1	2	25	Incluye la configuración inicial (repositorio, GUT y estructura), que no produce funcionalidad visible
Sprint 2	1,5	37	Es el sprint con más desarrollo: la versión 0.2.0 debe contener los cinco elementos obligatorios. Lista de recorte disponible: HU-14 (1 punto) y HU-10 (5 puntos)
Sprint 3	1,5	10	La mayor parte del tiempo va a pruebas, pruebas de juego, documentación y entrega
Total	5	72	

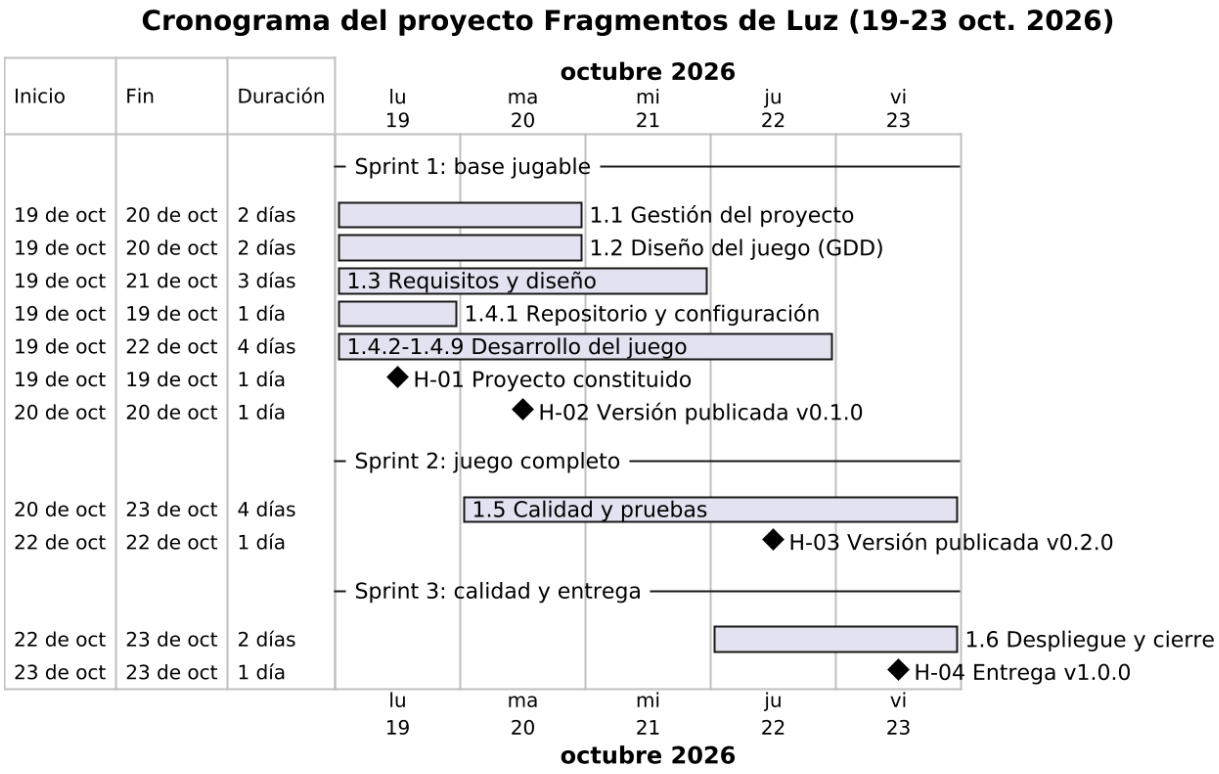
Nota. Elaboración propia.

1.5.3 Cronograma

La Figura 4 muestra el cronograma por día del proyecto, con los paquetes de la EDT, los tres sprints y los hitos, que se detallan en la Tabla 7. La fuente del diagrama está en `docs/diagramas/cronograma_gantt.puml`.

Figura 4

Cronograma del proyecto del 19 al 23 de octubre de 2026



Nota. Elaboración propia.

Tabla 7

Hitos del proyecto

ID	Hito	Fecha prevista	Fecha real	Criterio de cumplimiento (evidencia)
H-01	Proyecto constituido	Día 1 (19 oct.)	20 oct.	Acta aprobada por el docente; repositorio público creado con los cuatro integrantes como colaboradores; tablero de GitHub Projects con la pila del producto
H-02	Base jugable (fin del Sprint 1)	Día 2 (20 oct.)	20 oct.	Etiqueta y versión publicada **v0.1.0**: el caballero camina, salta, hace doble salto e impulso en el nivel en maqueta gris, y GUT se ejecuta
H-03	Juego completo (fin del Sprint 2)	Día 4 (22 oct., 12:00 m.)	22 oct. (4:58 p. m.)	Etiqueta y versión publicada **v0.2.0**: el nivel tiene los cinco elementos obligatorios y el flujo de pantallas completo
H-04	Entrega final (fin del Sprint 3)	Día 5 (23 oct., 5:00 p. m.)	23 oct.	Versión publicada **v1.0.0** con el ejecutable de Windows; PDF en la plataforma y en `docs/`; video publicado; 100 % de las pruebas unitarias aprobadas

Nota. Elaboración propia.

1.6 Matriz de riesgos

La Tabla 8 registra los diez riesgos identificados. La probabilidad (P) y el impacto (I) se califican de 1 a 5 y su producto es la exposición (P × I). Los riesgos con exposición de 12 o más (R-01 a R-07) tienen un plan de mitigación activo.

Tabla 8

Matriz de riesgos

ID	Riesgo (causa → efecto)	Tipo	P	I	P × I	Respuesta y mitigación	Disparador	Responsable	Estado
R-01	El plazo es de solo cinco días → funcionalidades sin terminar al cierre	Calendario	4	5	**20**	**Mitigar:** sprints cortos, priorizar las historias obligatorias y lista de recorte definida (primero las monedas, luego el murciélago)	Al cierre del Sprint 1 hay menos del 80 % de los puntos comprometidos terminados	Camilo	Activo
R-02	Se documenta algo que no se implementa, o se implementa algo no documentado → entrega inválida	Alcance	3	5	**15**	**Mitigar:** la trazabilidad es parte de la definición de terminado; el Product Owner revisa la matriz en cada revisión del sprint; lo recortado pasa a «Fuera de alcance»	Una funcionalidad del ejecutable no tiene fila en la matriz, o al revés	Jaider	Activo
R-03	Faltan las plantillas de exportación o el ejecutable falla fuera del editor → 0,0 en el componente del ejecutable	Técnico	3	5	**15**	**Mitigar:** exportar desde el Sprint 1 (v0.1.0) con las plantillas 4.5-stable y probar el ejecutable en un equipo sin Godot	El ejecutable exportado se cierra o muestra errores	Diego	Activo
R-04	Varias personas editan el mismo `.tscn` → conflictos de fusión y pérdida de trabajo	Técnico	4	3	**12**	**Mitigar:** cada escena tiene un solo responsable; composición de escenas pequeñas; solicitudes de integración cortas e integración frecuente con `main`	Una solicitud de integración tiene conflictos en un archivo `.tscn`	Diego	Activo
R-05	No se consiguen cinco usuarios externos a tiempo → incumplimiento de las pruebas de juego	Equipo	3	4	**12**	**Mitigar:** contactar desde el día 1 a estudiantes de otros grupos y a conocidos; agendar las sesiones el jueves 22 por la tarde	El miércoles 21 hay menos de cinco usuarios confirmados	Camilo	Activo
R-06	Errores de física (la roca atraviesa el suelo, el jugador se atasca o el viento se comporta mal) → fallas en la demostración	Técnico	3	4	**12**	**Mitigar:** implementar la física al inicio del Sprint 2, documentar capas y máscaras, y crear casos funcionales específicos	Un caso funcional de física falla	Kevin	Activo
R-07	Un integrante no domina el código de otros → baja nota en la defensa técnica	Equipo	3	4	**12**	**Mitigar:** revisión cruzada rotativa de las solicitudes de integración y ensayo de la sustentación el jueves por la noche	En el ensayo, un integrante no logra explicar un sistema ajeno	Camilo	Activo
R-08	Se usan recursos sin licencia clara → falta de integridad académica	Alcance	2	5	**10**	**Evitar:** solo recursos CC0, CC-BY o propios, registrados en la tabla de licencias al momento de descargarlos	Aparece un recurso en el proyecto sin fila en la tabla de licencias	Camilo	Monitoreado
R-09	Poca experiencia con GUT y con la ejecución por línea de comandos → pruebas atrasadas o sin reporte	Técnico	3	3	**9**	**Mitigar:** instalar GUT 9.5.0 y una prueba de ejemplo en el Sprint 1	Al cierre del Sprint 1, GUT no se ejecuta por consola	Diego	Monitoreado
R-10	Un integrante se enferma o no está disponible → retraso y menos contribuciones registradas	Equipo	2	4	**8**	**Aceptar y monitorear:** conocimiento compartido por la revisión cruzada; redistribuir tareas en la reunión diaria	Un integrante falta a dos reuniones diarias seguidas	Camilo	Monitoreado

Nota. Elaboración propia.

La Tabla 9 ubica cada riesgo según su probabilidad e impacto. El color indica la exposición: verde de 1 a 6, amarillo de 7 a 11 y rojo de 12 a 25.

Tabla 9

Mapa de calor de los riesgos (probabilidad × impacto)

P \ I	1	2	3	4	5
5					
4			R-04		R-01
3			R-09	R-05, R-06, R-07	R-02, R-03
2				R-10	R-08
1					

Nota. Elaboración propia. Filas: probabilidad (P); columnas: impacto (I).

La Tabla 10 registra el estado de cada riesgo al cierre de cada sprint.

Tabla 10

Seguimiento de los riesgos por iteración

Riesgo	Al cierre del Sprint 1	Al cierre del Sprint 2	Al cierre del Sprint 3
R-01	**Controlado:** 25 de 25 puntos terminados (100 %); no se activó el disparador	**Controlado:** 37 de 37 puntos terminados (100 %); v0.2.0 salió a las 4:58 p. m., unas horas después del mediodía previsto	**Cerrado:** 72 de 72 puntos y 23 de 23 historias terminadas
R-02	**Activo:** el documento se escribe con los valores reales del código; la matriz se revisa en cada revisión del sprint	**Activo:** documento con los valores reales del código; la matriz tiene resultado en los 27 requisitos	**Cerrado:** los 27 requisitos tienen prueba y resultado en la matriz de trazabilidad
R-03	**Mitigado en parte:** v0.1.0 exportada con las plantillas 4.5-stable y publicada el 20 oct.; falta probarla en un equipo sin Godot (CP-20)	**Controlado:** v0.2.0 exportada y publicada (#43); el ejecutable corre desde una carpeta aparte sin abrir Godot (CP-20)	**Cerrado:** v1.0.0 exportada con las plantillas 4.5-stable y adjunta a la versión publicada
R-04	**Controlado:** las 11 solicitudes de integración se fusionaron sin conflictos	**Controlado:** las solicitudes del Sprint 2 (#32 a #43) se fusionaron sin conflictos	**Cerrado:** ninguna solicitud tuvo conflictos de fusión en todo el proyecto
R-05	**Controlado:** cinco usuarios externos confirmados para las pruebas de juego	**Materializado en parte:** se hicieron 4 de las 5 pruebas de juego (la quinta persona no pudo asistir)	**Cerrado con impacto:** 4 de 5 pruebas de juego; RNF-03 y RNF-04 se evaluaron con 4 personas
R-06	**Activo:** roca, viento y púas fusionados el 20 oct. (#34); sus casos funcionales se ejecutan en el Sprint 3	**Materializado y corregido:** la roca no se movía (DF-06), encontrado en las pruebas de juego y corregido en #39	**Cerrado:** DF-06 corregido y verificado (CP-09 y sesión 3)
R-07	**Mitigado:** los cuatro integrantes revisaron solicitudes de otros (Kevin 4, Jaider 3, Camilo 2 y Diego 2)	**Mitigado:** la revisión cruzada siguió en todas las solicitudes; ensayo de la sustentación con guías por integrante	**Mitigado:** guías de sustentación por integrante con sus archivos, funciones y preguntas probables
R-08	**Monitoreado:** los 41 recursos de `assets/` están registrados; el ícono propio `icon.svg` se registra en el Sprint 2	**Cerrado:** el ícono propio `icon.svg` quedó registrado (#37); todos los recursos están en `LICENCIAS.md`	**Cerrado:** todos los recursos registrados en `LICENCIAS.md` y en los créditos
R-09	**Cerrado:** GUT 9.5.0 se ejecuta por consola desde la HU-07 (#26)	**Cerrado:** las pruebas GUT se ejecutan por consola en cada versión	**Cerrado:** 42 de 42 pruebas en la versión final (#46)
R-10	**Monitoreado:** ningún integrante faltó a dos reuniones diarias seguidas	**Monitoreado:** nadie faltó a las reuniones	**Cerrado:** nadie faltó a las reuniones

Nota. Elaboración propia.

1.7 Matriz RACI

La Tabla 11 asigna las responsabilidades sobre cada entregable: R, realiza; A, rinde cuentas; C, es consultado; I, es informado; y A/R, rinde cuentas y realiza. Cada actividad tiene un único responsable de rendir cuentas.

Tabla 11

Matriz RACI

Actividad o entregable	Jaider	Camilo	Kevin	Diego	Docente
Documento					
Acta de constitución	R	C	C	C	**A**
Capítulo 1. Inicio y planeación (resto)	C	**A/R**	C	C	I
Capítulo 2. Diseño del juego (GDD)	R	**A/R**	C	R	I
Capítulo 3. Requisitos	**A/R**	C	R	C	I
Capítulo 4. Análisis y diseño	C	I	R	**A/R**	I
Capítulo 5. Construcción	C	C	C	**A/R**	I
Capítulo 6. Calidad y pruebas	R	R	**A/R**	R	I
Capítulo 7. Despliegue y cierre	C	**A/R**	R	R	I
Ensamblaje del PDF final (APA 7)	**A/R**	C	C	C	I
Sistemas del juego					
Jugador (movimiento, doble salto, impulso y espada)	**A/R**	I	C	C	
Enemigos (slime y murciélago)	**A**	C	R	I	
Física y obstáculos (roca, viento, púas y capas)	**A**	C	R	I	
Nivel y objetos (`TileMapLayer`, fragmentos, puntos de control y santuario)	**A**	R	C	C	
Narrativa (introducción, carteles y cinemática final)	**A**	R	I	C	
Audio (música, efectos y buses)	**A**	R	C	C	
Interfaz y flujo de pantallas	**A**	C	I	R	
Persistencia (mejor puntaje y volumen)	**A**	I	I	R	
Construcción y calidad					
Repositorio, ramas y configuración	C	I	C	**A/R**	
Pruebas unitarias (GUT)	R	R	R	**A/R**	
Casos funcionales y rendimiento	C	C	**A/R**	R	
Pruebas de juego con usuarios externos	C	**A/R**	C	C	I
Matriz de trazabilidad	**A/R**	C	C	C	I
Entrega					
Exportación y versiones publicadas (v0.1.0, v0.2.0 y v1.0.0)	C	I	I	**A/R**	I
Video demo	**A/R**	C	C	C	I
Sustentación	**A**	R	R	R	I

Nota. Elaboración propia.

2. Diseño del juego (GDD)

Este capítulo describe el juego desde la perspectiva del jugador y es la fuente de los requisitos del capítulo 3. Todos los valores numéricos son los del código de la versión 1.0.0 y pueden verificarse en el Inspector de Godot o en el script indicado.

2.1 Concepto

Título: *Fragmentos de Luz*

Concepto en una frase: Morvak, el Hechicero de Sombra, robó el Cristal de Luz de Valdoria y lo rompió en cinco fragmentos. Como Sir Aldric, el jugador recorre el Bosque Marchito saltando, embistiendo y combatiendo para recuperarlos y devolver la luz al reino.

Género y subgénero: plataformas 2D; subgénero de acción y plataformas, con combate cuerpo a cuerpo y recolección.

Público objetivo: jugadores casuales de 10 años en adelante que disfrutan de partidas cortas (de 5 a 10 minutos por nivel) y de controles precisos.

Clasificación estimada: ESRB E10+ (todos, a partir de 10 años), con el descriptor de violencia fantástica (Entertainment Software Rating Board, s. f.), y PEGI 7 (PEGI, s. f.).

La clasificación se justifica porque la única violencia es el caballero golpeando con su espada a criaturas fantásticas (slimes y murciélagos), que desaparecen sin sangre ni restos, en un contexto claramente irreal. No hay lenguaje ofensivo, contenido sexual, apuestas ni compras dentro del juego.

Plataforma y dispositivo de entrada: PC con Windows 10 o posterior (x86_64), con teclado y ratón para los menús. El mando queda fuera de alcance.

2.1.1 Referentes

La Tabla 12 indica qué se toma de cada juego de referencia.

Tabla 12

Juegos de referencia

Juego	Qué se toma
Shovel Knight **(Yacht Club Games, 2014)**	Un caballero protagonista en arte de píxeles y un ataque cuerpo a cuerpo como forma principal de enfrentar a los enemigos
Celeste **(Maddy Makes Games, 2018)**	El impulso como movimiento central, el control preciso y los puntos de control frecuentes que reducen la frustración
Hollow Knight **(Team Cherry, 2017)**	La atmósfera de un mundo oscuro y decadente, y la vida representada con íconos en el HUD
Super Mario Bros. **(Nintendo, 1985)**	Enemigos que patrullan, monedas como puntaje opcional y una meta clara al final del nivel

Nota. Elaboración propia.

2.1.2 Propuesta de valor

Fragmentos de Luz reúne en un nivel corto lo que esos referentes reparten en horas de juego: la movilidad de *Celeste* (impulso y doble salto), el combate con espada de *Shovel Knight* y la ambientación de *Hollow Knight*. Suma dos interacciones físicas que cambian la forma de moverse: una corriente de viento que eleva al caballero y una roca que hay que empujar. El jugador ve su avance de forma directa, porque cada fragmento recogido es a la vez el objetivo, el puntaje y la historia.

2.2 Historia y personajes

Premisa: el reino de Valdoria vive gracias a la luz del Cristal de Luz, guardado en el santuario del bosque. Morvak, el Hechicero de Sombra, lo roba y lo rompe en cinco fragmentos que esparce por el bosque. Sin el Cristal, el bosque se marchita y sus criaturas se corrompen.
Conflicto: Sir Aldric, el último caballero guardián del santuario, debe atravesar el Bosque Marchito, enfrentar a las criaturas corrompidas y recuperar los cinco fragmentos antes de que la oscuridad consuma el reino.

La Tabla 13 resume la estructura narrativa y la escena de Godot donde se implementa cada parte; la Figura 5 muestra cómo se ve en el juego.

Tabla 13

Estructura narrativa

Parte	Qué ocurre	Cómo se presenta en el juego	Escena de Godot
Inicio	Morvak roba el Cristal y lo rompe; Aldric jura recuperarlo	Introducción de 4 pantallas de texto sobre una ilustración; el texto aparece letra por letra, Enter avanza y Esc la salta	`scenes/narrativa/intro.tscn`
Desarrollo	Aldric atraviesa el bosque, aprende a moverse y recupera los fragmentos	3 carteles de piedra en el nivel: al acercarse, el HUD muestra un texto corto que mezcla la historia con instrucciones	`nivel_bosque.tscn` con instancias de `cartel.tscn`
Cierre	Aldric coloca los 5 fragmentos en el santuario, el Cristal se restaura y la luz vuelve; Morvak sigue libre, como gancho para una continuación	Cinemática final de 3 pantallas de texto, seguida de la pantalla de victoria	`scenes/narrativa/final.tscn`

Nota. Elaboración propia.

Figura 5

La historia dentro del juego: introducción, cartel y final



Nota. Capturas del juego, versión 1.0.0.

Los carteles también funcionan como tutorial integrado, base del requisito de usabilidad RNF-03.

2.2.1 Fichas de personajes

Las Tablas 14 y 15 describen a cada personaje y la Figura 6 muestra sus imágenes del juego.

Tabla 14

Fichas de personajes: Sir Aldric y Morvak

Campo	Sir Aldric (jugable)	Morvak, el Hechicero de Sombra (no jugable)
Rol	Protagonista, último guardián del santuario	Antagonista
Motivación	Cumplir su juramento y devolver la luz a Valdoria	Sumir el reino en la oscuridad para gobernarlo
Habilidades	Caminar, salto, doble salto («salto bendecido»), impulso («embestida de escudo») y ataque con espada	Corrompe a las criaturas del bosque; no combate en la versión 1.0.0
Aspecto	Caballero de armadura plateada con capa y penacho azules, y ojos cian (la luz del Cristal) dentro del visor; imagen propia de 24 × 24 px con 9 animaciones	Figura encapuchada de tonos morados y negros; solo aparece en las ilustraciones de la introducción
Comportamiento	Lo controla el jugador; escena `caballero.tscn`	Elemento narrativo sin comportamiento en el juego

Nota. Elaboración propia.

Tabla 15

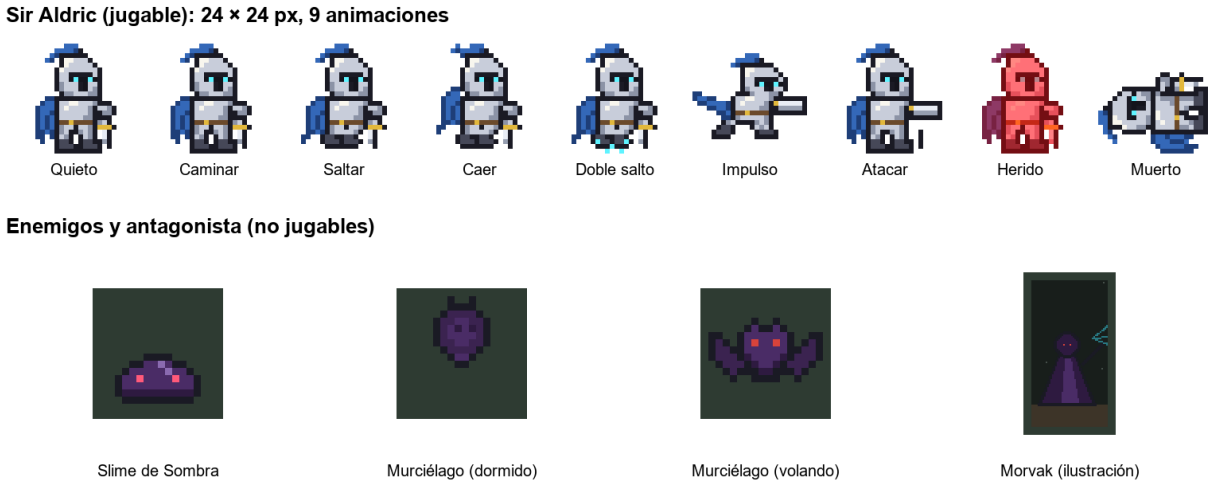
Fichas de personajes: enemigos

Campo	Slime de Sombra (no jugable)	Murciélago de Sombra (no jugable)
Rol	Enemigo terrestre básico	Enemigo aéreo
Motivación	Criatura del bosque corrompida que defiende su territorio	Criatura corrompida que ataca a los intrusos
Habilidades	Daño por contacto (1 corazón); muere de 1 golpe	Vuelo y persecución; daño por contacto (1 corazón); muere de 2 golpes
Aspecto	Gota gelatinosa morada con ojos rojos; imagen propia de 16 × 16 px	Murciélago morado con ojos rojos; imagen propia de 16 × 16 px con animaciones de dormir y volar
Comportamiento	Patrulla de lado a lado a 40 px/s y gira al chocar con una pared o al llegar a un borde; escena `slime.tscn`	Duerme hasta que el caballero se acerca a menos de 80 px; lo persigue a 60 px/s, vuelve a su sitio si se aleja a más de 140 px y se duerme otra vez; escena `murcielago.tscn`

Nota. Elaboración propia.

Figura 6

Imágenes de los personajes



Nota. Imágenes propias del equipo, ampliadas sin suavizado.

2.3 Mecánicas, ciclo central de juego y reglas

2.3.1 Mecánicas principales

Las Tablas 16 y 17 describen las mecánicas, su propósito en la experiencia y los parámetros que las gobiernan.

Tabla 16

Mecánicas principales

Mecánica	Descripción	Propósito en la experiencia	Parámetros
Movimiento horizontal	El caballero camina a izquierda y derecha	Explorar el nivel y esquivar	Velocidad: 100 px/s
Salto	Salto desde el suelo, más alto si se mantiene la tecla	Superar obstáculos y alcanzar plataformas	Velocidad inicial: 260 px/s; gravedad: 800 px/s²; al soltar la tecla en la subida la velocidad vertical se reduce a la mitad; altura máxima de unos 42 px (2,6 casillas)
Doble salto (especial)	Un segundo salto en el aire	Llegar a plataformas altas y corregir saltos	Ver la sección 2.4
Impulso (especial)	Desplazamiento horizontal rápido	Cruzar huecos largos y esquivar enemigos	Ver la sección 2.4
Ataque con espada	Golpe corto hacia donde mira el caballero	Derrotar enemigos y ganar puntos	Daño: 1; zona de golpe de 16 × 12 px delante del caballero; duración: 0,25 s; enfriamiento: 0,35 s
Recolección de fragmentos	Al tocar un fragmento se suma al contador	Es el objetivo del nivel y el motor de la historia	5 en el nivel; 100 puntos cada uno

Nota. Elaboración propia.

Tabla 17

Mecánicas secundarias

Mecánica	Descripción	Propósito en la experiencia	Parámetros
Monedas	Objetos opcionales en zonas de riesgo	Premiar la exploración	20 en el nivel; 10 puntos cada una
Vida e invulnerabilidad	Recibir daño quita un corazón, empuja al caballero hacia atrás y lo hace parpadear	Castigar los errores sin frustrar	3 corazones; 1,0 s de invulnerabilidad; retroceso de 0,3 s
Puntos de control	Estandartes que cambian de color al tocarlos	Reducir la repetición tras un error	2 en el nivel; se reaparece en el último activado
Roca empujable	`RigidBody2D` que el caballero mueve al empujarla	Acertijo físico para llegar a un fragmento alto	Masa: 3 kg; fuerza de empuje: 900; velocidad máxima: 45 px/s
Corriente de viento	`Area2D` que empuja hacia arriba al caballero mientras está dentro	Zona de vuelo que se combina con el doble salto	Área de 40 × 220 px; fuerza hacia arriba: 1100 px/s²; velocidad máxima de subida: 120 px/s
Púas	`Area2D` que hace daño al tocarla	Castigar los saltos imprecisos	7 tramos en el nivel; 1 corazón por contacto
Carteles de piedra	Muestran un texto en el HUD al acercarse	Narrativa y tutorial	3 en el nivel
Pausa	Detiene el juego y abre un menú	Control del jugador	Tecla Esc; la música baja 12 dB

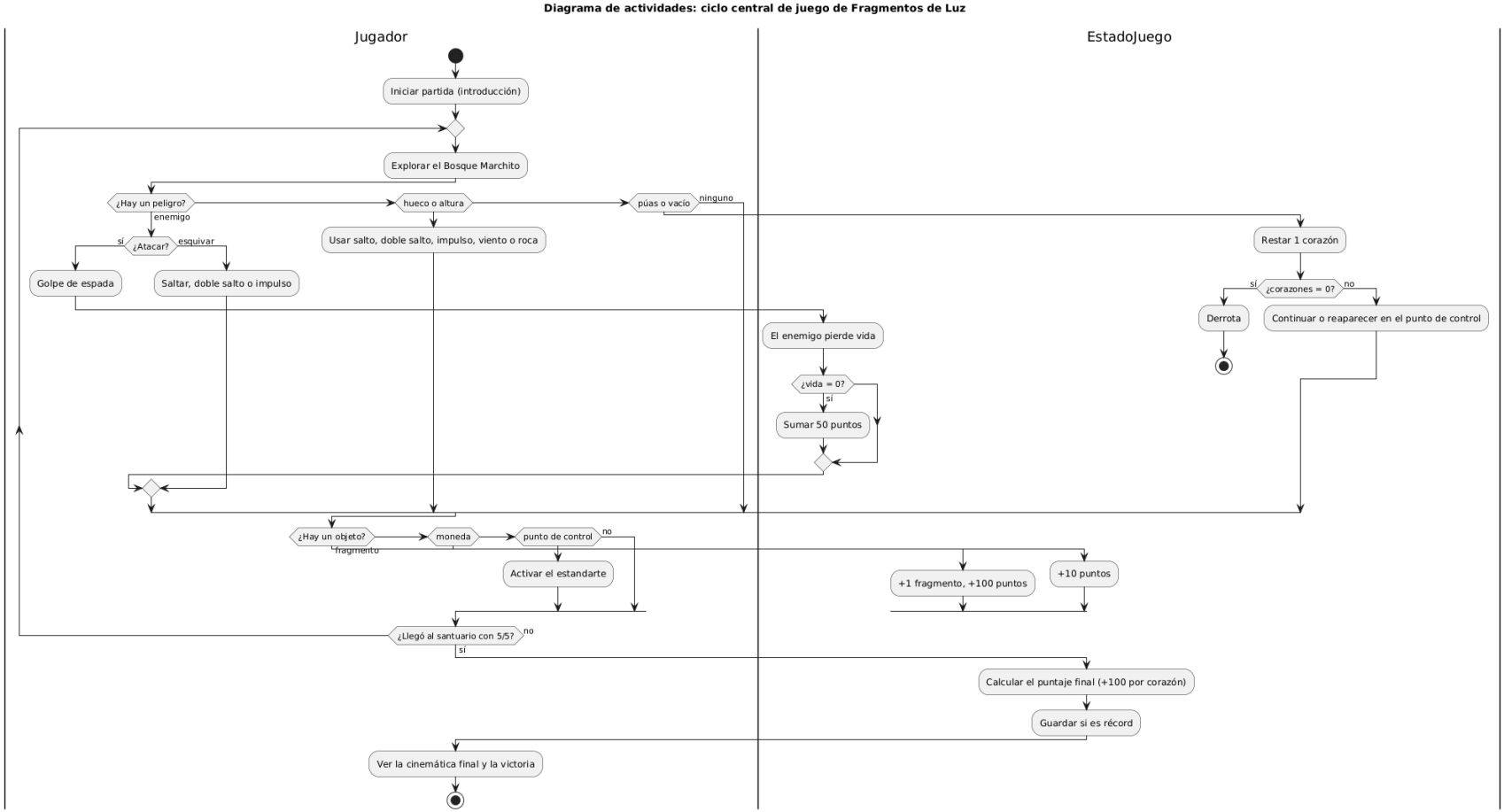
Nota. Elaboración propia.

2.3.2 Ciclo central de juego

La Figura 7 muestra el ciclo que el jugador repite durante la partida: explorar, superar un obstáculo o enemigo con sus movimientos o con la física, recoger fragmentos y monedas y, con los cinco fragmentos, ir al santuario. Si recibe daño y le quedan corazones, sigue jugando o reaparece en el último punto de control; si no, pierde. La fuente está en `docs/diagramas/actividades_y_flujo.puml`.

Figura 7

Diagrama de actividades del ciclo central de juego



Nota. Elaboración propia.

2.3.3 Reglas del juego

1. El caballero empieza con **3 corazones**, 0 puntos y 0/5 fragmentos.
2. El contacto con un enemigo o con púas quita **1 corazón** y activa **1,0 s de invulnerabilidad**.
3. Caer al vacío quita **1 corazón** y el caballero reaparece en el **último punto de control activado**, o en el inicio si no hay ninguno.
4. **Derrota:** quedarse con 0 corazones abre la pantalla de derrota.
5. El slime muere con 1 golpe y el murciélago con 2. Cada enemigo derrotado da **50 puntos**.
6. Cada fragmento da **100 puntos** y cada moneda, **10 puntos**.
7. **Victoria:** tocar el santuario con **5/5 fragmentos**. Si faltan, se muestra «Falta 1 fragmento» o «Faltan X fragmentos».
8. **Puntaje final** = puntos acumulados + **100 por cada corazón restante**.
9. Si el puntaje final supera **estrictamente** al mejor guardado, se guarda como nuevo **mejor puntaje**.

2.3.4 Controles

La Tabla 18 lista los controles. Los nombres de las acciones coinciden con el mapa de entradas del proyecto (capítulo 4).

Tabla 18

Controles

Acción (mapa de entradas)	Tecla	Efecto
`mover_izquierda`	A o flecha izquierda	Camina a la izquierda
`mover_derecha`	D o flecha derecha	Camina a la derecha
`saltar`	Espacio, W o flecha arriba	Salta; en el aire, hace el doble salto
`dash`	Mayúsculas o K	Hace el impulso hacia donde mira el caballero
`atacar`	J o clic izquierdo	Golpe de espada
`pausa`	Esc	Pausa y reanuda el juego; salta la introducción y el final
`ui_accept` (de Godot)	Enter o Espacio	Avanza el texto de la introducción y del final; confirma en los menús

Nota. Elaboración propia.

2.4 Movimientos especiales

La Tabla 19 especifica los dos movimientos especiales con los valores del código entregado; los parámetros son variables `@export` de `scripts/jugador/caballero.gd` y se pueden verificar en el Inspector.

Tabla 19

Especificación de movimientos especiales

Movimiento	Acción (mapa de entradas)	Condición de activación	Parámetros (valor y unidad)	Estado del jugador	Script y función
Doble salto	`saltar`	Presionar `saltar` en el aire, con `saltos_aereos_restantes` > 0	`velocidad_doble_salto` = 220 px/s hacia arriba; `max_saltos_aereos` = 1; se recarga al tocar el suelo	`DOBLE_SALTO`	`scripts/jugador/caballero.gd` → `doble_salto()`
Impulso	`dash`	Presionar `dash` con el enfriamiento terminado (`puede_dash`); en el aire, un solo impulso hasta volver a tocar el suelo	`velocidad_dash` = 260 px/s; `duracion_dash` = 0,15 s (unos 39 px); `enfriamiento_dash` = 0,5 s; sin gravedad durante el impulso; hacia donde mira el caballero	`DASH`	`scripts/jugador/caballero.gd` → `dash()`

Nota. Elaboración propia.

2.4.1 Comportamiento detallado

- **Doble salto:** la velocidad vertical se reemplaza por −220 px/s aunque el caballero esté cayendo, para que el salto sea predecible, y suena el efecto `doble_salto.wav`. Al tocar el suelo, `saltos_aereos_restantes` vuelve a 1.
- **Impulso:** durante 0,15 s el caballero se mueve en horizontal a velocidad constante, sin gravedad y sin poder cambiar de dirección, y suena `dash.wav`. Al terminar pasa a `CAYENDO` o a `QUIETO`, según esté en el aire o en el suelo. El enfriamiento empieza al terminar el impulso.
- **Combinación:** se pueden encadenar salto, doble salto e impulso, que es el movimiento de máximo alcance. El nivel lo exige en el hueco largo de la zona 5, antes del santuario.
- **Dentro de la corriente de viento:** ambos movimientos funcionan y el viento suma su fuerza hacia arriba.

Los estados del jugador, detallados en el diagrama de estados del capítulo 4, son `QUIETO`, `CORRIENDO`, `SALTANDO`, `CAYENDO`, `DOBLE_SALTO`, `DASH`, `ATACANDO`, `HERIDO` y `MUERTO`.

2.5 Niveles y progresión

El juego tiene un único nivel, el *Bosque Marchito* (`scenes/niveles/nivel_bosque.tscn`), que es el nivel completo con los cinco elementos obligatorios: música, física, movimientos especiales, dinámica de juego e historia. Como hay un solo nivel, no hay progresión entre niveles; las mecánicas nuevas se introducen por zonas dentro del mismo nivel.

Objetivo: recuperar los 5 fragmentos del Cristal y llegar al santuario.
Tamaño: 3840 × 440 px (240 casillas de 16 px de ancho), con recorrido de izquierda a derecha en cinco zonas de 768 px.
Duración estimada: de 5 a 8 minutos para un jugador nuevo.
Contenido: 5 fragmentos, 4 slimes, 3 murciélagos, 3 carteles, 2 puntos de control, 1 roca, 1 corriente de viento, 7 tramos de púas, 20 monedas y 1 santuario.

2.5.1 Zonas del recorrido

La Tabla 20 describe cada zona y la Figura 8 muestra el nivel completo con la ubicación de los enemigos, obstáculos, objetos y puntos de control.

Tabla 20

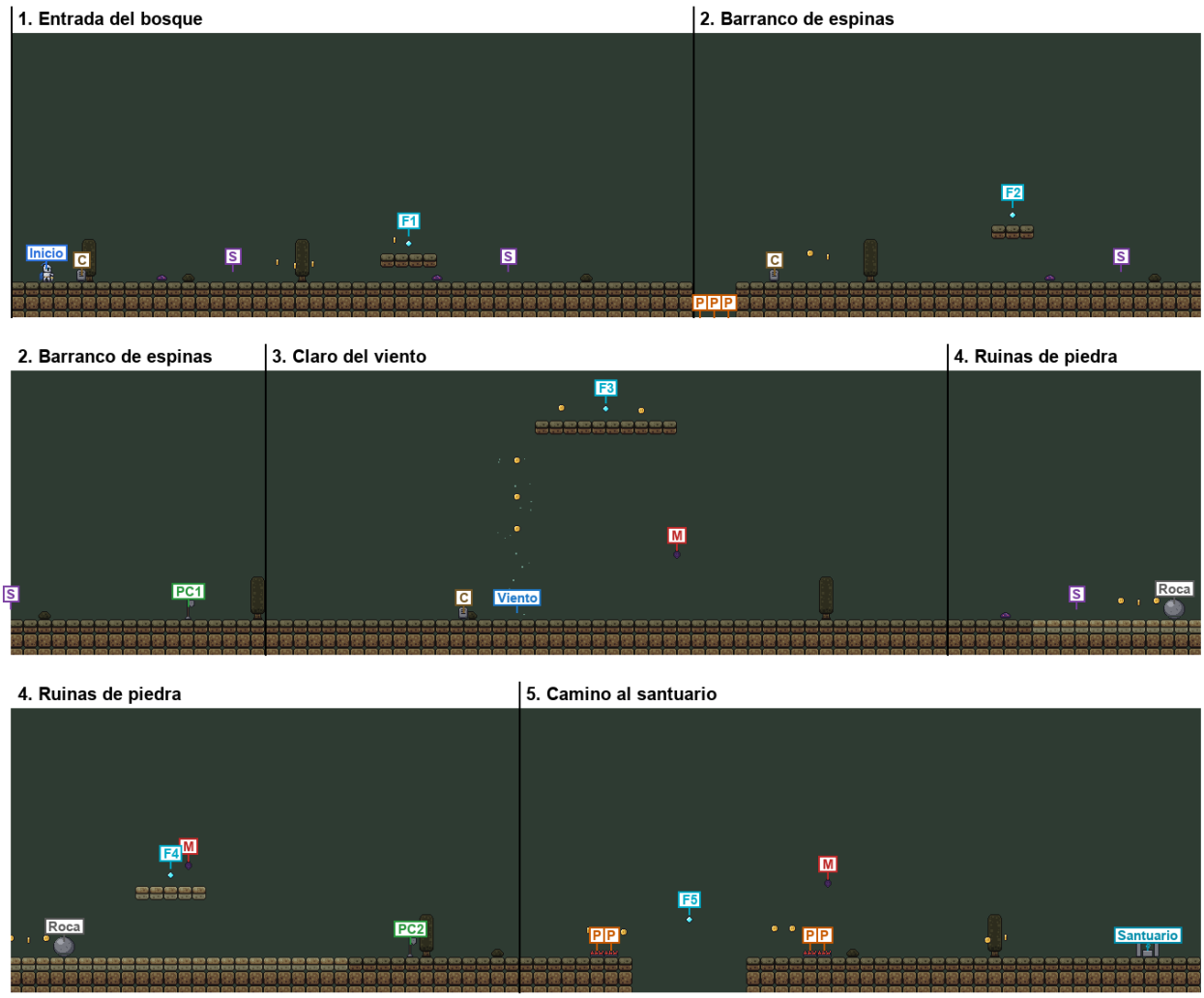
Zonas del nivel Bosque Marchito

Zona	Nombre	Mecánica que introduce	Contenido	Fragmento
1	Entrada del bosque	Movimiento, salto y espada	Cartel 1 (moverse, saltar y atacar), 2 slimes y monedas fáciles	F1 sobre una plataforma baja
2	Barranco de espinas	Doble salto e impulso	Cartel 2 (salto bendecido y embestida), foso con púas, cornisa alta y 1 slime; punto de control 1 al final	F2 en una cornisa que exige doble salto
3	Claro del viento	Corriente de viento (`Area2D`)	Cartel 3 (el aliento del bosque), columna de viento que sube a una plataforma alta, 1 murciélago y monedas en el aire	F3 en la plataforma alta
4	Ruinas de piedra	Roca empujable (`RigidBody2D`)	Roca que hay que empujar bajo una cornisa para usarla de escalón, 1 slime y 1 murciélago; punto de control 2 al final	F4 en la cornisa a la que se llega con la roca
5	Camino al santuario	Combinación de todo	Hueco largo que exige salto, doble salto e impulso; púas; 1 murciélago y monedas de riesgo; santuario al final	F5 sobre el hueco largo

Nota. Elaboración propia.

Figura 8

Mapa del nivel Bosque Marchito con sus zonas y objetos



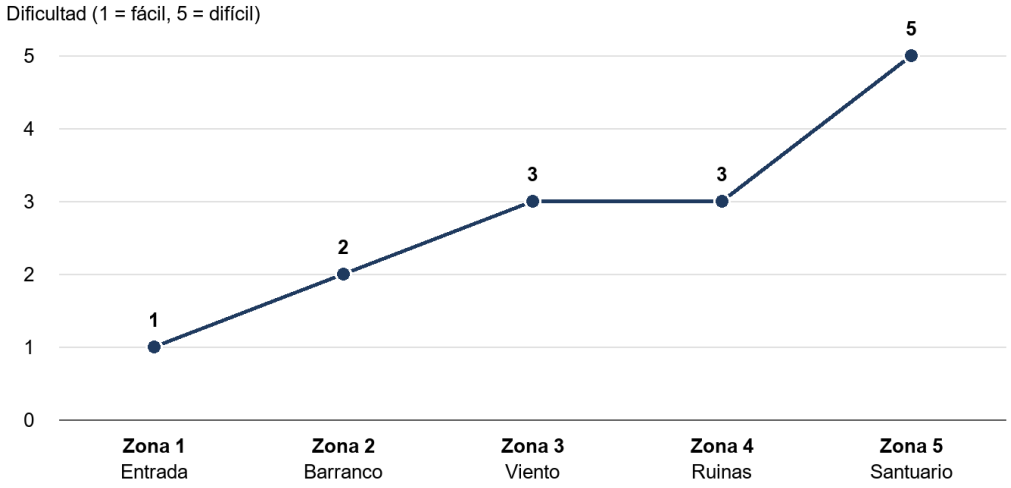
Nota. Imagen del nivel completo generada desde el proyecto, versión 1.0.0; anotaciones propias.

2.5.2 Curva de dificultad

La dificultad sube de forma gradual (Figura 9): cada zona enseña una sola mecánica nueva en un entorno seguro antes de exigirla. Los puntos de control están justo antes de los dos saltos de dificultad (zonas 3 y 5), para que un error no obligue a repetir medio nivel. La zona 5 no enseña nada nuevo, solo combina lo aprendido, como un examen final. La Tabla 21 lista los peligros de cada zona.

Figura 9

Curva de dificultad por zona



Nota. Elaboración propia.

Tabla 21

Dificultad y peligros por zona

Zona	Dificultad (1 a 5)	Peligros activos
1	1	2 slimes
2	2	Foso con púas, 1 slime y huecos
3	3	1 murciélago y la subida con el viento
4	3	1 slime y 1 murciélago
5	5	Hueco largo, púas y 1 murciélago

Nota. Elaboración propia.

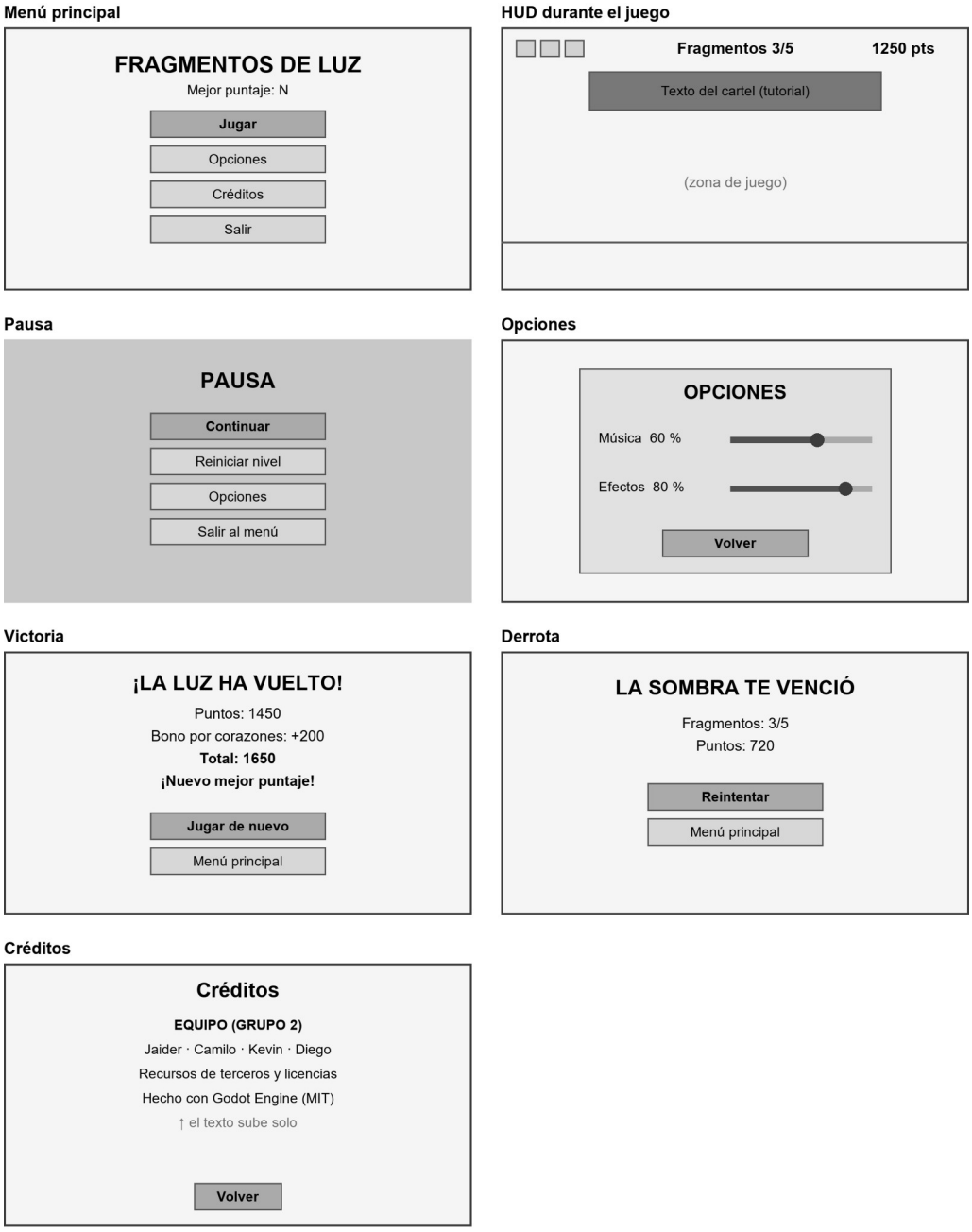
2.6 Interfaz y experiencia de usuario

2.6.1 Bocetos de pantalla

La Figura 10 presenta el boceto de cada pantalla: menú principal, HUD, pausa, opciones, victoria, derrota y créditos. Las pantallas finales del juego se muestran en el manual de usuario (capítulo 7).

Figura 10

Bocetos de las pantallas del juego



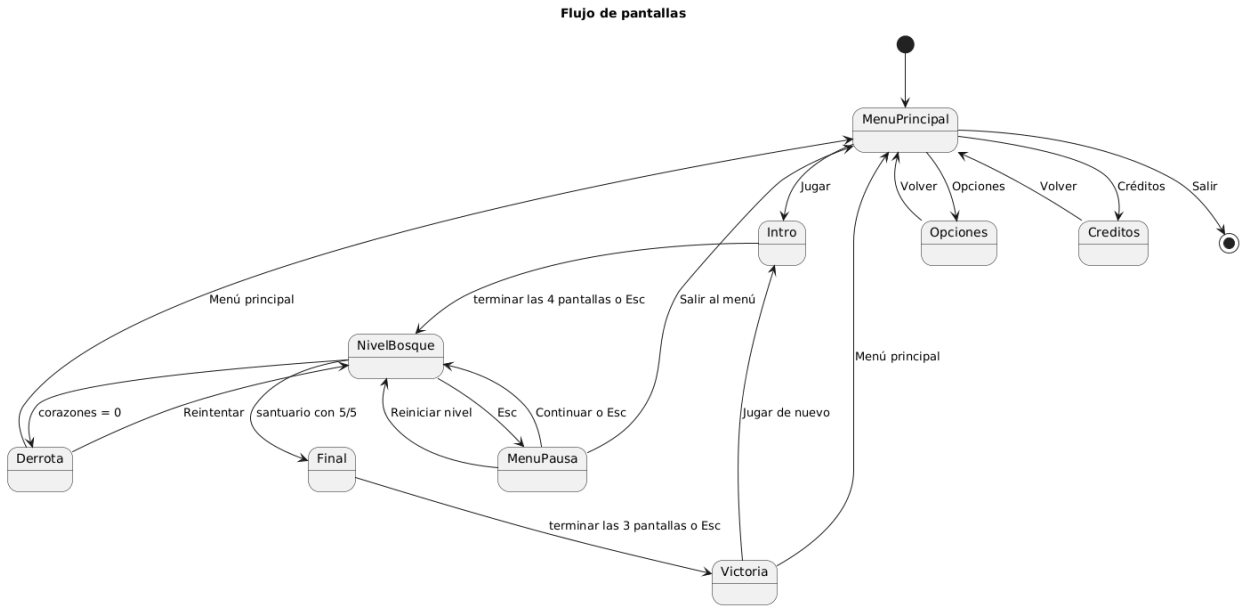
Nota. Elaboración propia.

2.6.2 Flujo de pantallas

La Figura 11 muestra el flujo entre pantallas y la Tabla 22 lista el evento que produce cada transición. Reintentar y reiniciar llevan al inicio del nivel, no al punto de control. La fuente del diagrama está en `docs/diagramas/actividades_y_flujo.puml`.

Figura 11

Diagrama de flujo de pantallas



Nota. Elaboración propia.

Tabla 22

Transiciones entre pantallas

Desde	Evento	Hacia
Menú principal	Botón Jugar	Introducción
Menú principal	Botón Opciones	Opciones (Volver regresa al menú)
Menú principal	Botón Créditos	Créditos (Volver o Esc regresan al menú)
Menú principal	Botón Salir	Cierra el juego
Introducción	Terminar las 4 pantallas o presionar Esc	Nivel
Nivel	Esc	Pausa (con el juego detenido)
Pausa	Continuar o Esc	Nivel
Pausa	Reiniciar nivel	Nivel (desde el inicio)
Pausa	Opciones	Opciones (Volver regresa a la pausa)
Pausa	Salir al menú	Menú principal
Nivel	Corazones = 0	Derrota
Derrota	Reintentar	Nivel (desde el inicio)
Derrota	Menú principal	Menú principal
Nivel	Tocar el santuario con 5/5 fragmentos	Final (cinemática)
Final	Terminar las 3 pantallas o presionar Esc	Victoria

Desde	Evento	Hacia
Victoria	Jugar de nuevo	Introducción
Victoria	Menú principal	Menú principal

Nota. Elaboración propia.

2.6.3 Decisiones de usabilidad

- **Legibilidad:** fuente de arte de píxeles de al menos 8 px en la resolución base, que se ve de 32 px a 1280 × 720. Texto claro sobre paneles oscuros. El HUD va arriba y en las esquinas, sin tapar la zona de juego.
- **Retroalimentación visual:** el caballero parpadea al recibir daño, el corazón perdido pasa de lleno a vacío, el contador de fragmentos crece un instante al sumar uno y el estandarte del punto de control cambia de color al activarse.
- **Retroalimentación sonora:** cada acción principal tiene su sonido (salto, doble salto, impulso, espada, golpe, daño, fragmento, moneda, punto de control, victoria, derrota y botones).
- **Accesibilidad:** volumen de música y de efectos por separado, que se guarda entre sesiones; controles duplicados (WASD y flechas); la introducción y el final se pueden saltar; los menús se manejan con teclado y con ratón.
- **Fuera de alcance:** reasignación de teclas y mando.

2.7 Arte, audio y licencias

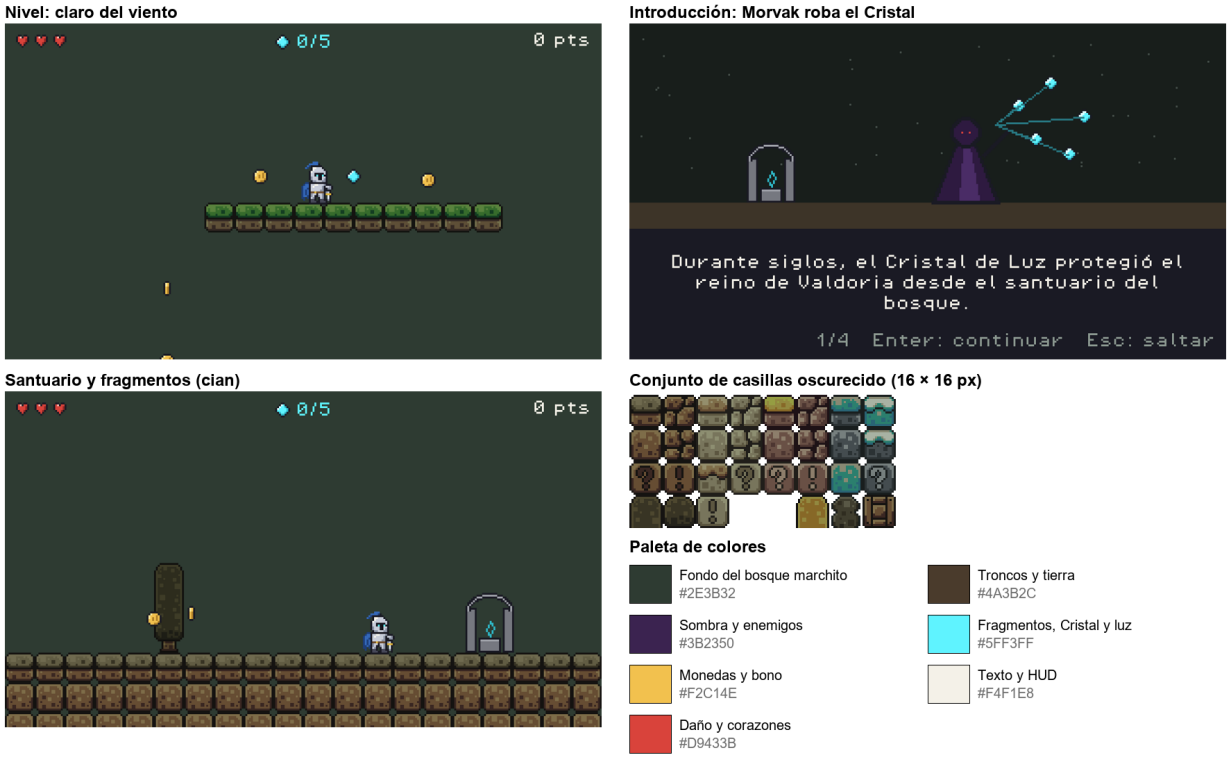
2.7.1 Estilo visual

- **Estilo:** arte de píxeles 2D con vista lateral y casillas de 16 × 16 px.
- **Resolución base:** 320 × 180, escalada a una ventana de 1280 × 720 con el modo de estiramiento `viewport`.
- **Texturas:** filtro `Nearest` en todo el proyecto (ajuste `rendering/textures/canvas_textures/default_texture_filter` de la configuración del proyecto), para que los píxeles no se vean borrosos.
- **Tratamiento del conjunto de casillas:** se desaturó y oscureció en la propia imagen, y no con `CanvasModulate`, para que los personajes y los fragmentos conserven su brillo.

La Figura 12 reúne el tablero de referencias visuales (moodboard) del juego con su paleta, y la Tabla 23 indica el uso de cada color. El contraste entre el bosque desaturado y el cian de los fragmentos hace que el objetivo se distinga de inmediato.

Figura 12

Tablero de referencias visuales y paleta de colores



Nota. Capturas del juego e imágenes del proyecto, versión 1.0.0.

Tabla 23

Paleta de colores

Uso	Muestra	Color	Código
Fondo del bosque marchito		Verde grisáceo oscuro	`#2E3B32`
Troncos y tierra		Marrón apagado	`#4A3B2C`
Sombra y enemigos		Morado oscuro	`#3B2350`
Fragmentos, Cristal y luz		Cian brillante	`#5FF3FF`
Monedas y bono		Dorado	`#F2C14E`
Texto y HUD		Blanco hueso	`#F4F1E8`
Daño y corazones		Rojo	`#D9433B`

Nota. Elaboración propia.

2.7.2 Audio

La Tabla 24 lista cada pista y efecto, cuándo suena y el bus de audio que lo controla. Los buses son `Master`, `Musica` y `Efectos`; los dos deslizadores del panel de opciones controlan `Musica` y `Efectos`, y el autoload `GestorAudio` reproduce todos los sonidos.

Tabla 24

Música y efectos de sonido

Pista o efecto	Momento en que suena	Bus
`musica_menu.mp3`	Menú principal, opciones y créditos; también tras ganar	`Musica`
`musica_nivel.mp3`	Durante el nivel; baja 12 dB en la pausa	`Musica`
`salto.wav`	Salto desde el suelo	`Efectos`
`doble_salto.wav`	Doble salto	`Efectos`
`dash.wav`	Impulso	`Efectos`
`espada.wav`	Ataque con espada	`Efectos`
`golpe_enemigo.wav`	La espada golpea a un enemigo	`Efectos`
`dano.wav`	El caballero recibe daño	`Efectos`
`fragmento.wav`	Recoger un fragmento	`Efectos`
`moneda.wav`	Recoger una moneda	`Efectos`
`checkpoint.wav`	Activar un punto de control	`Efectos`
`victoria.wav`	Al ganar la partida	`Efectos`
`derrota.wav`	Al perder la partida	`Efectos`
`boton.wav`	Presionar cualquier botón de los menús	`Efectos`

Nota. Elaboración propia.

2.7.3 Recursos y licencias

La Tabla 25 registra todos los recursos de terceros y los propios, con su licencia, la atribución y el uso de herramientas de inteligencia artificial. Los recursos propios se crearon con scripts de Python que dibujan las imágenes y sintetizan los sonidos; esos scripts los escribió el equipo con apoyo de un asistente de inteligencia artificial y están en `herramientas/arte/`. También se respeta la licencia MIT del propio motor, cuyo aviso aparece en la pantalla de créditos (Godot Engine, s. f.-a).

Tabla 25

Recursos y licencias

Recurso (ruta en el proyecto)	Tipo	Autor	Fuente (URL)	Licencia	Atribución exigida y dónde se da	Uso de inteligencia artificial
`assets/sprites/caballero/`, `assets/sprites/enemigos/`	Imágenes de personajes	Propias del equipo (`herramientas/arte/generar_personajes.py`)	—	Propia	—	Sí: el script lo escribió el equipo con apoyo de un asistente de IA (Claude)
`assets/sprites/objetos/`, `assets/sprites/ui/`	Imágenes de objetos e interfaz	Propias del equipo (`generar_objetos.py`, `generar_ui.py`)	—	Propia	—	Sí: el script lo escribió el equipo con apoyo de un asistente de IA (Claude)
`assets/sprites/narrativa/`	Ilustraciones de la historia	Propias del equipo (`generar_ilustraciones.py`)	—	Propia	—	Sí: el script lo escribió el equipo con apoyo de un asistente de IA (Claude)
`assets/sprites/tileset/tileset_bosque_marchito.png`	Conjunto de casillas	RottingPixels (Four Seasons Platformer Tileset), reempaquetado por Brackeys; oscurecido por el equipo (`oscurecer_tileset.py`)	https://brackeysgames.itch.io/brackeys-platformer-bundle	CC0	No exigida; se da en los créditos	Solo en el script de oscurecido
`assets/fuentes/PixelOperator8.ttf` y `-Bold.ttf`	Fuente	Jayvee Enaguas (HarvettFox96), en Brackeys Platformer Bundle	https://brackeysgames.itch.io/brackeys-platformer-bundle	CC0	No exigida; se da en los créditos	No
`assets/audio/` (salto, moneda, daño, fragmento, botón y derrota)	Efectos de sonido	Brackeys y Asbjørn Thirslund, en Brackeys Platformer Bundle (renombrados)	https://brackeysgames.itch.io/brackeys-platformer-bundle	CC0	No exigida; se da en los créditos	No
`assets/audio/` (doble salto, impulso, espada, golpe, punto de control y victoria)	Efectos de sonido	Propios del equipo (`generar_sonidos.py`)	—	Propia	—	Sí: el script lo escribió el equipo con apoyo de un asistente de IA (Claude)
`assets/musica/musica_nivel.mp3` («Time for Adventure»)	Música	Brackeys y Sofia Thirslund, en Brackeys Platformer Bundle	https://brackeysgames.itch.io/brackeys-platformer-bundle	CC0	No exigida; se da en los créditos	No
`assets/musica/musica_menu.mp3` («[Chiptune] Medieval: The Bard's Tale»)	Música	RandomMind	https://opengameart.org/node/94347	CC0	No exigida; se da en los créditos	No
`addons/gut/` (GUT 9.5.0)	Complemento de pruebas	Butch Wesley (bitwes)	https://github.com/bitwes/Gut	MIT	Licencia incluida en la carpeta	No
Godot Engine 4.5	Motor	Juan Linietsky, Ariel Manzur y colaboradores	https://godotengine.org/license	MIT	Aviso de licencia en la pantalla de créditos	No

Nota. Elaboración propia.

3. Requisitos

3.1 Marco de referencia

La especificación sigue la estructura de una especificación de requisitos de software según IEEE Std 830-1998 (Institute of Electrical and Electronics Engineers [IEEE], 1998) y su sucesora, ISO/IEC/IEEE 29148:2018 (International Organization for Standardization [ISO], 2018).

- **Propósito:** especificar los requisitos de la versión 1.0.0 de *Fragmentos de Luz* para el equipo de desarrollo y para el docente como cliente.
- **Alcance:** el producto es un juego de plataformas 2D para Windows con un nivel completo; su alcance es el del acta (sección 1.1).
- **Definiciones y siglas:** se encuentran en el glosario de los preliminares.
- **Perspectiva del producto:** sistema independiente (ejecutable y paquete `.pck`) que solo guarda datos locales en `user://` y no depende de servicios en línea.
- **Características de los usuarios:** jugadores casuales de 10 años en adelante, con experiencia básica en el uso del teclado y sin conocimientos técnicos.
- **Restricciones:** Godot 4.5-stable con GDScript; Windows 10 o posterior (x86_64); teclado y ratón; recursos con licencia libre o propios; plazo de cinco días.
- **Supuestos y dependencias:** el equipo de referencia cumple los requisitos mínimos; el proyecto depende de las plantillas de exportación 4.5-stable y del complemento GUT 9.5.0.

Cada requisito es necesario, no ambiguo, verificable, consistente, factible y trazable, y se redacta en la forma «El sistema debe…». La prioridad se asigna con el método MoSCoW, con cuatro niveles: **obligatoria** (debe tener), **importante** (debería tener), **deseable** (podría tener) y **excluida** (no se tendrá en esta versión). Los requisitos obligatorios e importantes están implementados en el ejecutable; los deseables implementados se tratan igual, y los excluidos se listan aparte. El origen indica de dónde sale el requisito: un apartado del GDD, una regla de negocio, el acta o el enunciado.

3.2 Requisitos funcionales

La Tabla 26 especifica los 19 requisitos funcionales. Los cinco elementos obligatorios del nivel (música, física, movimientos especiales, dinámica de juego e historia) aparecen como requisitos obligatorios. El estado indica si el requisito ya está en el código; pasa a «Verificado» cuando sus pruebas se aprueban (capítulo 6).

Tabla 26

Requisitos funcionales

ID	Nombre	Descripción («El sistema debe…»)	Prioridad	Criterio de aceptación verificable	Origen	Estado
RF-01	Movimiento y salto	El sistema debe permitir al caballero caminar a izquierda y derecha y saltar desde el suelo	Obligatoria	Al mantener `mover_derecha`, el caballero avanza a 100 px/s; al presionar `saltar` en el suelo, se eleva unos 42 px	GDD 2.3	Implementado
RF-02	Doble salto	El sistema debe permitir un segundo salto en el aire, una vez por salto	Obligatoria	En el aire, `saltar` aplica −220 px/s; un tercer intento no tiene efecto; al tocar el suelo se recarga	GDD 2.4; Elementos obligatorios del enunciado	Implementado
RF-03	Impulso	El sistema debe permitir un desplazamiento horizontal rápido con enfriamiento	Obligatoria	`dash` desplaza al caballero unos 39 px en 0,15 s sin gravedad; un segundo `dash` antes de 0,5 s no tiene efecto	GDD 2.4; Elementos obligatorios del enunciado	Implementado
RF-04	Ataque con espada	El sistema debe permitir al caballero atacar con la espada hacia donde mira	Obligatoria	`atacar` activa una zona de golpe de 16 × 12 px delante del caballero durante 0,25 s; un enemigo dentro pierde 1 punto de vida	GDD 2.3	Implementado
RF-05	Vida y daño	El sistema debe gestionar 3 corazones, restar uno por cada daño y dar invulnerabilidad temporal	Obligatoria	Al tocar un enemigo o púas, los corazones pasan de 3 a 2 y durante 1,0 s no se recibe más daño	GDD 2.3; RN-01, RN-02	Implementado
RF-06	Puntos de control y caída	El sistema debe activar puntos de control y hacer reaparecer al caballero en el último activado tras caer al vacío	Obligatoria	Tras tocar el punto de control 1 y caer al vacío, el caballero reaparece en él con un corazón menos	GDD 2.3 y 2.5	Implementado
RF-07	Slime patrullero	El sistema debe incluir un enemigo slime que patrulla y gira en paredes y bordes	Obligatoria	El slime invierte su dirección al detectar una pared o un borde y muere con 1 golpe	GDD 2.2; Elementos obligatorios del enunciado	Implementado
RF-08	Murciélago perseguidor	El sistema debe incluir un enemigo murciélago que persigue al caballero cuando está cerca y regresa si se aleja	Importante	Con el caballero a menos de 80 px, el murciélago lo persigue; a más de 140 px, regresa a su punto de origen; muere con 2 golpes	GDD 2.2	Implementado
RF-09	Roca empujable	El sistema debe incluir una roca `RigidBody2D` que el caballero puede empujar y usar como plataforma	Obligatoria	Al empujar la roca, esta se desplaza; el caballero puede pararse sobre ella para alcanzar el fragmento F4	GDD 2.3; Elementos obligatorios del enunciado	Implementado
RF-10	Corriente de viento	El sistema debe incluir un `Area2D` que eleva al caballero mientras está dentro	Obligatoria	Dentro de la corriente, el caballero sube sin saltar hasta 120 px/s; al salir vuelve la gravedad normal	GDD 2.3; Elementos obligatorios del enunciado	Implementado
RF-11	Fragmentos y puntaje	El sistema debe contar los fragmentos recogidos y sumar puntos por fragmentos y enemigos derrotados	Obligatoria	Al recoger un fragmento, el contador sube (por ejemplo, de 2/5 a 3/5) y el puntaje suma 100; al derrotar un enemigo, suma 50	GDD 2.3; RN-04	Implementado
RF-12	Monedas	El sistema debe incluir monedas opcionales que suman puntos	Deseable	Al recoger una moneda, el puntaje suma 10 y la moneda desaparece	GDD 2.3	Implementado
RF-13	Victoria y derrota	El sistema debe declarar la victoria al tocar el santuario con 5/5 fragmentos y la derrota con 0 corazones	Obligatoria	Con 5/5 en el santuario se inicia la cinemática final; con menos se muestra «Falta 1 fragmento» o «Faltan X fragmentos»; con 0 corazones aparece la pantalla de derrota	GDD 2.3; Elementos obligatorios del enunciado	Implementado
RF-14	HUD	El sistema debe mostrar durante el nivel los corazones, los fragmentos y el puntaje	Obligatoria	El HUD se actualiza en el mismo cuadro en que cambia cada valor	GDD 2.6; Elementos obligatorios del enunciado	Implementado

ID	Nombre	Descripción («El sistema debe…»)	Prioridad	Criterio de aceptación verificable	Origen	Estado
RF-15	Pausa y reinicio	El sistema debe pausar el juego con Esc y permitir continuar, reiniciar, abrir opciones o salir al menú	Obligatoria	Con Esc, el juego se detiene (los enemigos quedan quietos) y aparece el menú de pausa; Reiniciar nivel carga el nivel desde el inicio	GDD 2.6; Elementos obligatorios del enunciado	Implementado
RF-16	Narrativa en el juego	El sistema debe presentar la introducción, los 3 carteles y la cinemática final	Obligatoria	Al jugar se muestran las 4 pantallas de la introducción; al acercarse a un cartel aparece su texto; al ganar se muestran las 3 pantallas finales	GDD 2.2; Elementos obligatorios del enunciado	Implementado
RF-17	Música y efectos	El sistema debe reproducir música en el menú y en el nivel, y efectos en las acciones principales	Obligatoria	Cada acción de la tabla de audio (2.7) reproduce su sonido con un nodo `AudioStreamPlayer`; la música cambia entre el menú y el nivel	GDD 2.7; Elementos obligatorios del enunciado	Implementado
RF-18	Menú, opciones y créditos	El sistema debe ofrecer el menú principal, el panel de opciones con volumen y la pantalla de créditos	Obligatoria	Cada botón lleva a la pantalla indicada en el flujo (2.6); los deslizadores cambian el volumen de los buses `Musica` y `Efectos`	GDD 2.6; enunciado, sección 4	Implementado
RF-19	Persistencia	El sistema debe guardar el mejor puntaje y el volumen en `user://` y cargarlos al iniciar	Obligatoria	Después de cerrar y abrir el juego, el menú muestra el mismo mejor puntaje y los deslizadores conservan su valor	GDD 2.3; enunciado, sección 11.5	Implementado

Nota. Elaboración propia.

Excluidos (no se tendrán en la versión 1.0.0): jefe final Morvak, más niveles, guardado a mitad del nivel, reasignación de teclas, soporte de mando, y versiones web y Linux.

3.3 Requisitos no funcionales

La Tabla 27 especifica los 8 requisitos no funcionales, clasificados según el modelo de calidad de producto de ISO/IEC 25010:2023 (ISO, 2023). Cada uno tiene una métrica, un umbral, un método de medición y un entorno de referencia; el equipo de referencia se describe con sus características reales en el plan de pruebas (sección 6.1).

Tabla 27

Requisitos no funcionales

ID	Característica (ISO/IEC 25010)	Descripción	Métrica	Umbral	Método y entorno de medición	Prioridad	Origen
RNF-01	Eficiencia de desempeño (comportamiento temporal)	El juego debe mantener una tasa de cuadros fluida durante el nivel	Cuadros por segundo (FPS)	Promedio ≥ 60 FPS y mínimo ≥ 30 FPS	Monitor FPS del panel Depurador › Monitores; 3 mediciones de 60 s en 2 escenarios (zona 3 con murciélago y viento; zona 5) en el equipo de referencia (6.1)	Obligatoria	Enunciado, sección 10.3
RNF-02	Eficiencia de desempeño (comportamiento temporal)	El paso de la introducción al nivel debe ser rápido	Tiempo desde la última pulsación de Enter de la introducción hasta el primer cuadro del nivel (s)	≤ 2 s	Registro de tiempo con `Time.get_ticks_usec()` desde el cambio de escena pedido en la introducción hasta el primer cuadro del nivel; 3 repeticiones en el equipo de referencia	Importante	GDD 2.6
RNF-03	Capacidad de interacción (capacidad de aprendizaje)	Los jugadores deben aprender los controles con los carteles del nivel, sin ayuda	Porcentaje de usuarios externos que superan las zonas 1 y 2 sin ayuda del observador	≥ 80 % (al menos 4 de 5)	Observación en las sesiones de pruebas de juego (protocolo 6.5), con el ejecutable v0.2.0 o posterior	Obligatoria	GDD 2.2; enunciado, sección 10.3
RNF-04	Capacidad de interacción (satisfacción y operabilidad)	La experiencia de uso debe ser satisfactoria	Puntuación media del cuestionario SUS (Brooke, 1996)	≥ 68	Cuestionario SUS aplicado a los 5 usuarios externos al terminar la sesión	Obligatoria	Enunciado, sección 10.3
RNF-05	Flexibilidad (instalabilidad y adaptabilidad)	El ejecutable debe funcionar sin instalar Godot ni otras dependencias	Ejecuciones exitosas del ejecutable en equipos distintos	100 % en al menos 2 equipos (Windows 10 y Windows 11)	Descomprimir el `.zip` de la versión publicada v1.0.0 y abrir el `.exe` en equipos sin Godot; registrado como caso funcional	Obligatoria	Acta (restricciones); enunciado, sección 10.3
RNF-06	Mantenibilidad (analizabilidad y modificabilidad)	Todos los scripts deben usar tipado estático, seguir la guía de estilo oficial de GDScript y no generar advertencias	Número de advertencias del analizador de GDScript; porcentaje de scripts con tipado estático	0 advertencias; 100 % tipados	Revisión en cada solicitud de integración, con el aviso `untyped_declaration` activado en la configuración del proyecto; apertura del proyecto en la etiqueta v1.0.0	Obligatoria	Enunciado, secciones 10.3 y 12.1
RNF-07	Mantenibilidad (capacidad de ser probado)	La lógica del juego debe estar cubierta por pruebas automatizadas que se ejecuten por línea de comandos	Número de pruebas unitarias; porcentaje aprobado	≥ 15 pruebas; 100 % aprobadas	Ejecución de GUT 9.5.0 con `gut_cmdln.gd` sobre la etiqueta v1.0.0; reporte JUnit XML en `docs/reporte_gut.xml`	Obligatoria	Enunciado, sección 13.2
RNF-08	Fiabilidad (tolerancia a fallos y madurez)	El juego no debe fallar por un archivo de guardado ausente o dañado, ni producir errores durante una partida completa	Errores en el panel Depurador en una partida completa; comportamiento ante un archivo ausente o dañado	0 errores; con archivo ausente o dañado inicia con los valores por defecto	Partida completa en el editor revisando el panel Depurador; prueba unitaria que carga un archivo dañado	Obligatoria	Enunciado, sección 11.5 y condición de invalidez 4

Nota. Elaboración propia.

3.4 Reglas de negocio

La Tabla 28 expresa las reglas del juego de forma verificable e independiente de la implementación. Se implementan como lógica pura en el autoload `EstadoJuego` (y `GestorGuardado` para RN-09), lo que permite probarlas con GUT sin abrir el nivel.

Tabla 28

Reglas de negocio

ID	Regla	Ejemplo verificable	Requisitos relacionados
RN-01	**Vida inicial y daño.** El caballero inicia con 3 corazones; cada daño resta exactamente 1; los corazones nunca bajan de 0	Con 3 corazones, al tocar un slime quedan 2; con 1 corazón, al tocar púas quedan 0 y no −1	RF-05
RN-02	**Invulnerabilidad temporal.** Después de recibir daño, el caballero no recibe más daño durante 1,0 s	Con 3 corazones toca un slime (quedan 2) y 0,5 s después toca púas: sigue con 2. A los 1,2 s vuelve a tocar púas: queda con 1	RF-05
RN-03	**Caída al vacío.** Caer fuera del nivel resta 1 corazón y el caballero reaparece en el último punto de control activado, o en el inicio si no activó ninguno	Con 3 corazones y el punto de control 1 activado, cae al vacío: reaparece en el punto de control 1 con 2 corazones	RF-05, RF-06
RN-04	**Valores de puntaje.** Un fragmento vale 100 puntos, una moneda 10 y un enemigo derrotado 50	Con 0 puntos, recoge 2 fragmentos, 3 monedas y derrota 1 enemigo: queda con 280 puntos	RF-11, RF-12, RF-04
RN-05	**Resistencia de los enemigos.** El slime se derrota con 1 golpe de espada y el murciélago con 2	Un murciélago golpeado una vez sigue activo; al segundo golpe desaparece y suma 50 puntos	RF-04, RF-07, RF-08
RN-06	**Condición de victoria.** El nivel se gana solo al tocar el santuario con los 5 fragmentos	Con 4/5, tocar el santuario muestra «Falta 1 fragmento» y no termina el nivel; con 3/5 muestra «Faltan 2 fragmentos»; con 5/5 inicia la cinemática final	RF-13, RF-11
RN-07	**Condición de derrota.** La partida se pierde cuando los corazones llegan a 0	Con 1 corazón recibe daño: aparece la pantalla de derrota con los fragmentos y puntos que tenía	RF-13, RF-05
RN-08	**Puntaje final.** El puntaje final es la suma de los puntos acumulados más 100 por cada corazón restante al ganar	Gana con 1450 puntos y 2 corazones: el puntaje final es 1650	RF-13, RF-11
RN-09	**Mejor puntaje.** El mejor puntaje guardado solo se reemplaza si el puntaje final de la partida es estrictamente mayor	Mejor guardado 1650: una partida de 1500 no lo cambia; una de 1700 lo reemplaza y muestra «¡Nuevo mejor puntaje!»	RF-19, RF-13
RN-10	**Reinicio de partida.** Reintentar o reiniciar el nivel restaura 3 corazones, 0 puntos, 0/5 fragmentos y todos los enemigos y objetos, y no conserva los puntos de control activados	Con 1 corazón, 2/5 y 640 puntos elige Reiniciar nivel: empieza con 3 corazones, 0/5 y 0 puntos en el inicio del nivel	RF-15, RF-13

Nota. Elaboración propia.

3.5 Pila del producto e historias de usuario

La pila del producto contiene 23 historias de usuario con el formato «Como <rol>, quiero <acción>, para <beneficio>» (Cohn, 2004), que cumplen los criterios INVEST: independiente, negociable, valiosa, estimable, pequeña y verificable. Cada historia tiene uno o más criterios de

aceptación con la estructura Dado / Cuando / Entonces (Cucumber, s. f.), su estimación en puntos, su prioridad, el sprint en que se terminó y los requisitos que satisface. La Tabla 29 resume la pila; el Anexo A presenta cada historia con la plantilla completa.

Tabla 29

Resumen de la pila del producto

ID	Título	Puntos	Prioridad	Sprint	Responsable	RF/RNF
HU-01	Moverse y saltar	3	Obligatoria	1	Jaider	RF-01
HU-02	Doble salto	3	Obligatoria	1	Jaider	RF-02
HU-03	Impulso	5	Obligatoria	1	Jaider	RF-03
HU-04	Slime patrullero	5	Obligatoria	1	Kevin	RF-07
HU-05	Puntos de control y caída al vacío	3	Obligatoria	1	Camilo	RF-06
HU-06	Menú principal	3	Obligatoria	1	Diego	RF-18
HU-07	Pruebas automatizadas	3	Obligatoria	1	Diego	RNF-07
HU-08	Atacar con la espada	3	Obligatoria	2	Jaider	RF-04
HU-09	Vida, daño e invulnerabilidad	3	Obligatoria	2	Jaider	RF-05
HU-10	Murciélago perseguidor	5	Importante	2	Kevin	RF-08
HU-11	Roca empujable	3	Obligatoria	2	Kevin	RF-09
HU-12	Corriente de viento	3	Obligatoria	2	Kevin	RF-10
HU-13	Fragmentos y puntaje	3	Obligatoria	2	Camilo	RF-11
HU-14	Monedas	1	Deseable	2	Camilo	RF-12
HU-15	Historia en el juego	5	Obligatoria	2	Camilo	RF-16, RNF-03
HU-16	Música y efectos de sonido	3	Obligatoria	2	Camilo	RF-17
HU-17	HUD	2	Obligatoria	2	Diego	RF-14
HU-18	Victoria y derrota	3	Obligatoria	2	Diego	RF-13
HU-19	Pausa y reinicio	3	Obligatoria	2	Diego	RF-15
HU-20	Opciones de volumen	3	Obligatoria	3	Diego	RF-18
HU-21	Guardar mejor puntaje y volumen	3	Obligatoria	3	Diego	RF-19, RNF-08
HU-22	Créditos	1	Obligatoria	3	Camilo	RF-18
HU-23	Jugar sin instalar Godot	3	Obligatoria	3	Diego	RNF-05

Nota. Elaboración propia.

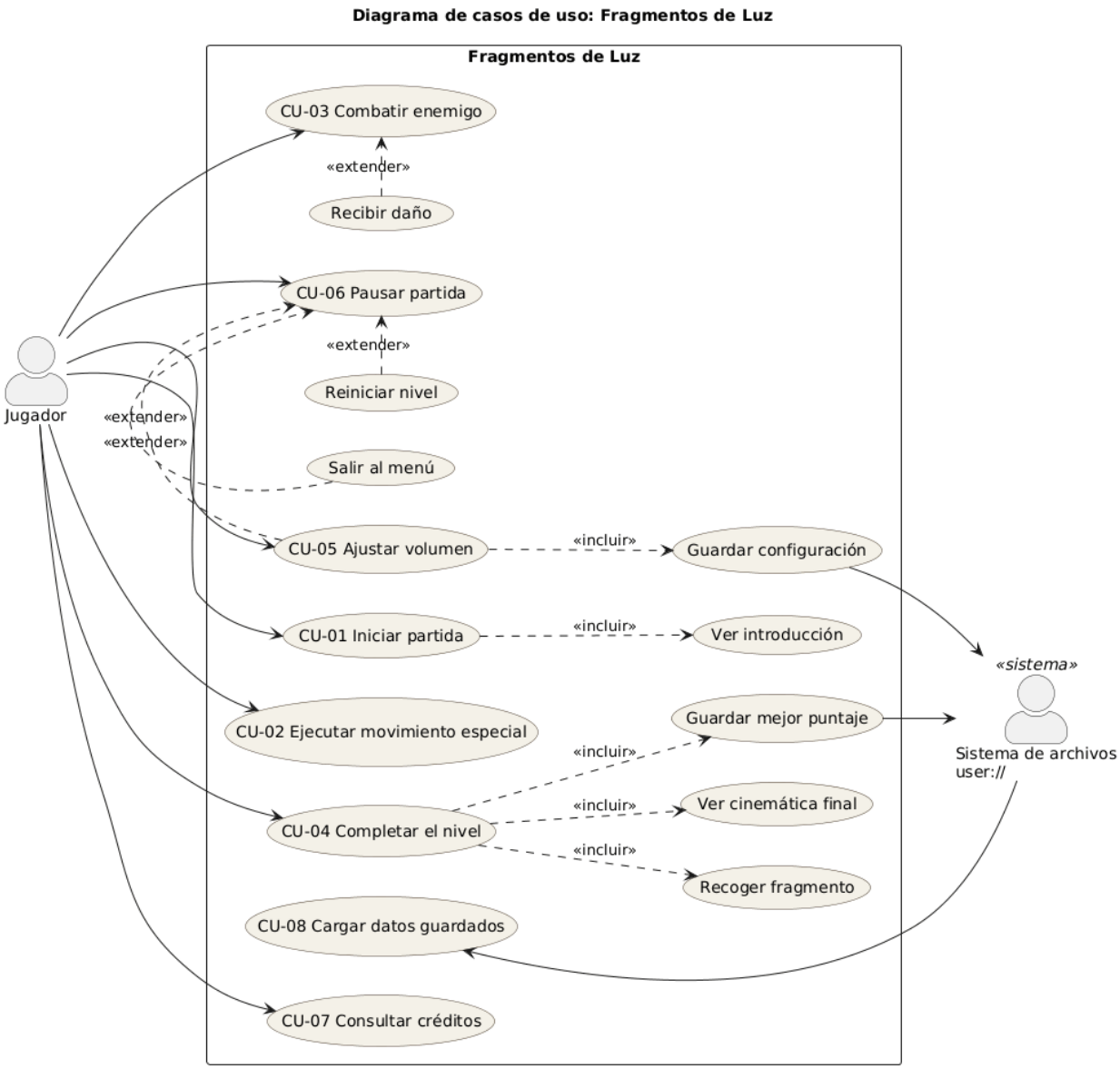
Puntos por sprint: Sprint 1 = 25; Sprint 2 = 37; Sprint 3 = 10; total = 72.

3.6 Casos de uso

La Figura 13 presenta el diagrama de casos de uso según UML 2.5.1 (Object Management Group [OMG], 2017). El actor principal es el jugador y el secundario, el sistema de archivos local `user://`, donde se guarda la configuración. La Tabla 30 resume las relaciones de inclusión y extensión. La fuente del diagrama está en `docs/diagramas/casos_de_uso.puml`.

Figura 13

Diagrama de casos de uso



Nota. Elaboración propia.

Tabla 30

Casos de uso y sus relaciones

Caso de uso	Actor	Relaciones
CU-01 Iniciar partida	Jugador	«incluir» Ver introducción
CU-02 Ejecutar movimiento especial	Jugador	—
CU-03 Combatir enemigo	Jugador	«extender» Recibir daño (si el enemigo toca al caballero)
CU-04 Completar el nivel	Jugador	«incluir» Recoger fragmento; «incluir» Ver cinemática final; «incluir» Guardar mejor puntaje
CU-05 Ajustar volumen	Jugador, sistema de archivos	«incluir» Guardar configuración
CU-06 Pausar partida	Jugador	«extender» Reiniciar nivel; «extender» Ajustar volumen; «extender» Salir al menú
CU-07 Consultar créditos	Jugador	—
CU-08 Cargar datos guardados	Sistema de archivos	Se ejecuta al iniciar el juego

Nota. Elaboración propia.

3.6.1 Especificaciones de casos de uso

Las Tablas 31 a 35 especifican cinco casos de uso completos; CU-02 involucra los movimientos especiales y CU-01 y CU-05, el flujo entre pantallas y la persistencia.

Tabla 31

Especificación de CU-01: Iniciar partida (flujo entre pantallas)

Campo	Contenido
ID y nombre	CU-01 Iniciar partida (flujo entre pantallas)
Actor principal y secundarios	Jugador / —
Descripción	El jugador inicia una partida desde el menú principal, ve la introducción y llega al nivel
Precondiciones	El juego está abierto en el menú principal
Disparador	El jugador presiona Jugar
Flujo básico	1. El jugador presiona Jugar. 2. El sistema reinicia el estado de la partida y muestra la primera pantalla de la introducción. 3. El jugador presiona Enter para avanzar por las 4 pantallas. 4. Al terminar la cuarta, el sistema carga el nivel con 3 corazones, 0 puntos y 0/5 fragmentos, y muestra el HUD. 5. El caballero aparece en el inicio del nivel y suena la música del nivel.
Flujos alternos	A1 (desde el paso 3): el jugador presiona Esc; el sistema salta la introducción y continúa en el paso 4.
Flujos de excepción	E1 (desde el paso 4): si la escena del nivel no existe o no carga, el sistema registra el error y regresa al menú principal.
Poscondiciones	Éxito: el nivel está en curso con el estado inicial. Fallo: el jugador permanece en el menú principal.
Reglas de negocio y requisitos	RN: RN-01, RN-10 RF/RNF: RF-16, RF-18, RF-14, RNF-02
Escenas y scripts que lo implementan	`menu_principal.tscn` (`menu_principal.gd`), `intro.tscn` (`pantalla_narrativa.gd`), `nivel_bosque.tscn`; autoloads `GestorEscenas` → `iniciar_partida()` y `EstadoJuego` → `reiniciar()`

Nota. Elaboración propia.

Tabla 32

Especificación de CU-02: Ejecutar movimiento especial

Campo	Contenido
ID y nombre	CU-02 Ejecutar movimiento especial
Actor principal y secundarios	Jugador / —
Descripción	El jugador usa el doble salto y el impulso para superar un hueco o alcanzar una plataforma
Precondiciones	El nivel está en curso y el caballero no está en estado `HERIDO` ni `MUERTO`
Disparador	El jugador presiona `saltar` en el aire o presiona `dash`
Flujo básico	1. El jugador salta desde el suelo (estado `SALTANDO`). 2. En el aire presiona `saltar`. 3. El sistema verifica que `saltos_aereos_restantes` > 0, aplica −220 px/s, resta 1 a los saltos restantes, cambia a `DOBLE_SALTO` y reproduce `doble_salto.wav`. 4. El jugador presiona `dash`. 5. El sistema verifica `puede_dash`, anula la gravedad, mueve al caballero a 260 px/s durante 0,15 s (estado `DASH`) y reproduce `dash.wav`. 6. Al terminar el impulso, el sistema pasa a `CAYENDO` e inicia el enfriamiento de 0,5 s. 7. El caballero aterriza y se recargan el doble salto y el impulso.
Flujos alternos	A1 (desde el paso 2): el jugador presiona `dash` primero; se ejecutan los pasos 4 a 6 y luego puede hacer el doble salto. A2 (desde el paso 4): el caballero está en el suelo; el impulso se ejecuta igual y termina en `QUIETO` o `CORRIENDO`.
Flujos de excepción	E1 (desde el paso 3): `saltos_aereos_restantes` es 0; no ocurre nada. E2 (desde el paso 5): el impulso está en enfriamiento o ya se usó en el aire; no ocurre nada.
Poscondiciones	Éxito: el caballero se desplazó según los parámetros. Fallo: el caballero sigue su trayectoria sin cambios.
Reglas de negocio y requisitos	RN: — RF/RNF: RF-02, RF-03, RF-17
Escenas y scripts que lo implementan	`caballero.tscn`; `scripts/jugador/caballero.gd` → `doble_salto()`, `dash()`

Nota. Elaboración propia.

Tabla 33

Especificación de CU-03: Combatir enemigo

Campo	Contenido
ID y nombre	CU-03 Combatir enemigo
Actor principal y secundarios	Jugador / —
Descripción	El jugador ataca a un enemigo con la espada hasta derrotarlo
Precondiciones	El nivel está en curso y hay un enemigo activo cerca del caballero
Disparador	El jugador presiona `atacar`
Flujo básico	1. El jugador presiona `atacar` mirando al enemigo. 2. El sistema cambia a `ATACANDO`, activa la zona de golpe (16 × 12 px, 0,25 s) y reproduce `espada.wav`. 3. La zona toca al enemigo; el sistema le resta 1 punto de vida y reproduce `golpe_enemigo.wav`. 4. Si la vida del enemigo llega a 0, el enemigo desaparece y el sistema suma 50 puntos. 5. El HUD actualiza el puntaje.
Flujos alternos	A1 (desde el paso 4): el enemigo es un murciélago con 1 punto de vida restante; sigue activo y hace falta otro ataque (vuelve al paso 1 tras el enfriamiento de 0,35 s).
Flujos de excepción	E1 (desde el paso 1): el enemigo toca al caballero antes del ataque; el caballero recibe daño (RN-01, RN-02). E2 (desde el paso 3): la zona no toca a ningún enemigo; el ataque termina sin efecto.
Poscondiciones	Éxito: el enemigo fue derrotado y el puntaje aumentó en 50. Fallo: el enemigo sigue activo.
Reglas de negocio y requisitos	RN: RN-01, RN-02, RN-04, RN-05 RF/RNF: RF-04, RF-05, RF-07, RF-08, RF-11, RF-14
Escenas y scripts que lo implementan	`caballero.gd` → `atacar()`; `slime.tscn` y `murcielago.tscn`, que heredan de `enemigo.gd` → `recibir_golpe()`; autoload `EstadoJuego` → `registrar_enemigo_derrotado()`

Nota. Elaboración propia.

Tabla 34

Especificación de CU-04: Completar el nivel

Campo	Contenido
ID y nombre	CU-04 Completar el nivel
Actor principal y secundarios	Jugador / Sistema de archivos `user://`
Descripción	El jugador lleva los 5 fragmentos al santuario y gana la partida
Precondiciones	El nivel está en curso
Disparador	El caballero entra en el área del santuario
Flujo básico	1. El caballero toca el santuario. 2. El sistema verifica que tenga 5/5 fragmentos, restaura el Cristal y marca el nivel como completado. 3. El sistema muestra las 3 pantallas de la cinemática final. 4. El sistema muestra la pantalla de victoria con los puntos, el bono por corazones y el puntaje final (puntos + 100 × corazones restantes). 5. El sistema compara el puntaje final con el mejor guardado y, si es estrictamente mayor, lo guarda en `user://config.cfg` y muestra «¡Nuevo mejor puntaje!».
Flujos alternos	A1 (desde el paso 2): tiene menos de 5 fragmentos; el sistema muestra «Falta 1 fragmento» o «Faltan X fragmentos» y el nivel continúa. A2 (desde el paso 3): el jugador presiona Esc y salta a la pantalla de victoria.
Flujos de excepción	E1 (desde el paso 5): no se puede escribir el archivo; el sistema registra una advertencia y continúa sin guardar.
Poscondiciones	Éxito: pantalla de victoria y mejor puntaje actualizado si correspondía. Fallo: el nivel sigue en curso.
Reglas de negocio y requisitos	RN: RN-06, RN-08, RN-09 RF/RNF: RF-11, RF-13, RF-16, RF-19
Escenas y scripts que lo implementan	`santuario.tscn` (`santuario.gd` → `intentar_restaurar()`), `final.tscn` (`pantalla_narrativa.gd`), `victoria.tscn` (`victoria.gd`); autoloads `EstadoJuego` → `completar_nivel()` y `GestorGuardado` → `registrar_puntaje()`

Nota. Elaboración propia.

Tabla 35

Especificación de CU-05: Ajustar volumen (persistencia)

Campo	Contenido
ID y nombre	CU-05 Ajustar volumen (persistencia)
Actor principal y secundarios	Jugador / Sistema de archivos `user://`
Descripción	El jugador ajusta el volumen de la música y de los efectos, y los valores se conservan entre sesiones
Precondiciones	El jugador está en el menú principal o en la pausa
Disparador	El jugador presiona Opciones
Flujo básico	1. El jugador presiona Opciones. 2. El sistema muestra el panel con los deslizadores en los valores actuales. 3. El jugador mueve el deslizador de Música. 4. El sistema aplica el volumen al bus `Musica` de inmediato. 5. El jugador mueve el deslizador de Efectos; el sistema lo aplica al bus `Efectos` y reproduce `boton.wav` como muestra. 6. El jugador presiona Volver. 7. El sistema guarda ambos valores en `user://config.cfg` y regresa a la pantalla anterior.
Flujos alternos	A1 (desde el paso 3): el jugador solo cambia uno de los deslizadores; el otro se guarda sin cambios.
Flujos de excepción	E1 (desde el paso 7): no se puede escribir el archivo; el volumen se mantiene durante la sesión y se registra una advertencia. E2 (al iniciar el juego): el archivo no existe o está dañado; se usan los valores por defecto (música 80 %, efectos 80 %).
Poscondiciones	Éxito: el volumen cambió y quedó guardado. Fallo: el volumen cambió solo durante la sesión.
Reglas de negocio y requisitos	RN: — RF/RNF: RF-18, RF-19, RNF-08
Escenas y scripts que lo implementan	`opciones.tscn` (`opciones.gd`); autoloads `GestorAudio` → `aplicar_volumen()` y `GestorGuardado` → `guardar_volumen()`

Nota. Elaboración propia.

4. Análisis y diseño

4.1 Arquitectura (modelo C4)

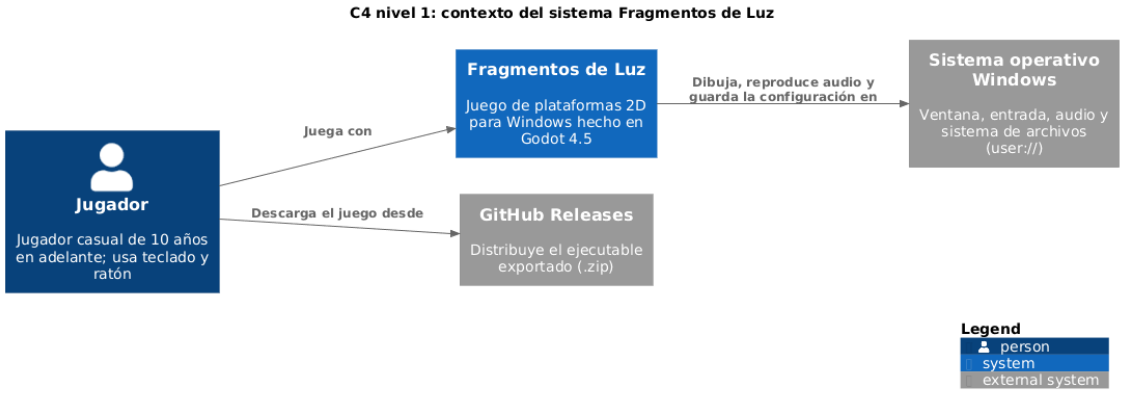
La arquitectura se documenta con el modelo C4 (Brown, s. f.) en tres niveles: contexto, contenedores y componentes. Los diagramas se generaron con PlantUML y la biblioteca C4-PlantUML, incluyen su leyenda y sus fuentes están en `docs/diagramas/c4_*.puml`.

4.1.1 Nivel 1: contexto del sistema

La Figura 14 muestra el juego como un único sistema y la Tabla 36 describe sus elementos.

Figura 14

Diagrama C4 de contexto



Nota. Elaboración propia.

Tabla 36

Elementos del diagrama de contexto

Elemento	Tipo	Descripción	Relación
Jugador	Persona	Jugador casual de 10 años en adelante	Juega a *Fragmentos de Luz* con teclado y ratón
Fragmentos de Luz	Sistema (el juego)	Juego de plataformas 2D para Windows	Lee y escribe datos locales en el sistema operativo
GitHub Releases	Sistema externo	Servicio de GitHub donde se publican las versiones	El jugador descarga desde ahí el `.zip` del juego
Sistema operativo Windows	Sistema externo	Provee la ventana, la entrada, el audio y el sistema de archivos	El juego guarda su configuración en la carpeta de datos del usuario

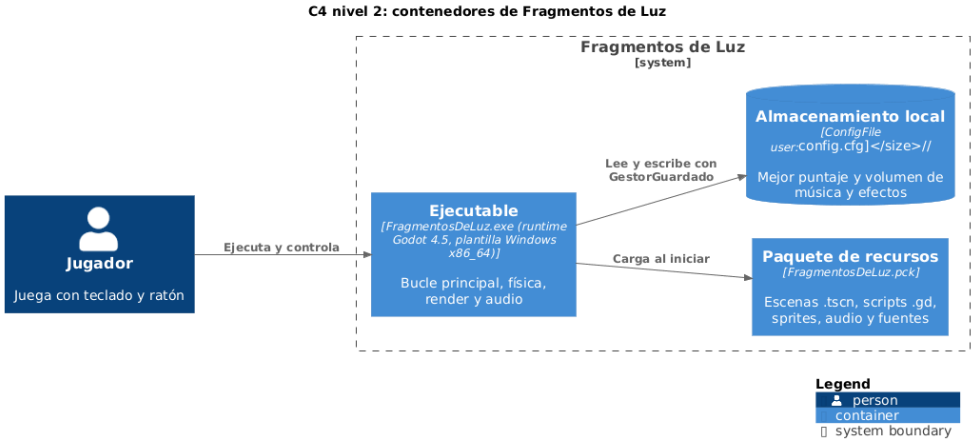
Nota. Elaboración propia.

4.1.2 Nivel 2: contenedores

La Figura 15 muestra los contenedores, es decir, las unidades que se ejecutan o almacenan datos por separado, y la Tabla 37 los describe.

Figura 15

Diagrama C4 de contenedores



Nota. Elaboración propia.

Tabla 37

Contenedores del sistema

Contenedor	Tecnología	Responsabilidad	Se comunica con
Ejecutable `FragmentosDeLuz.exe`	Plantilla de exportación de Godot 4.5 para Windows x86_64	Ejecuta el juego: bucle principal, física, dibujo y audio	Carga el `.pck` al iniciar; lee y escribe en `user://`
Paquete de recursos `FragmentosDeLuz.pck`	Formato PCK de Godot	Contiene las escenas, los scripts, las imágenes, el audio y las fuentes	Solo lo lee el ejecutable
Almacenamiento local `user://config.cfg`	`ConfigFile` de Godot, en `%APPDATA%\Godot\app_userdata\Fragmentos de Luz\`	Guarda el mejor puntaje y el volumen de la música y de los efectos	Lo lee y escribe el ejecutable (autoload `GestorGuardado`)

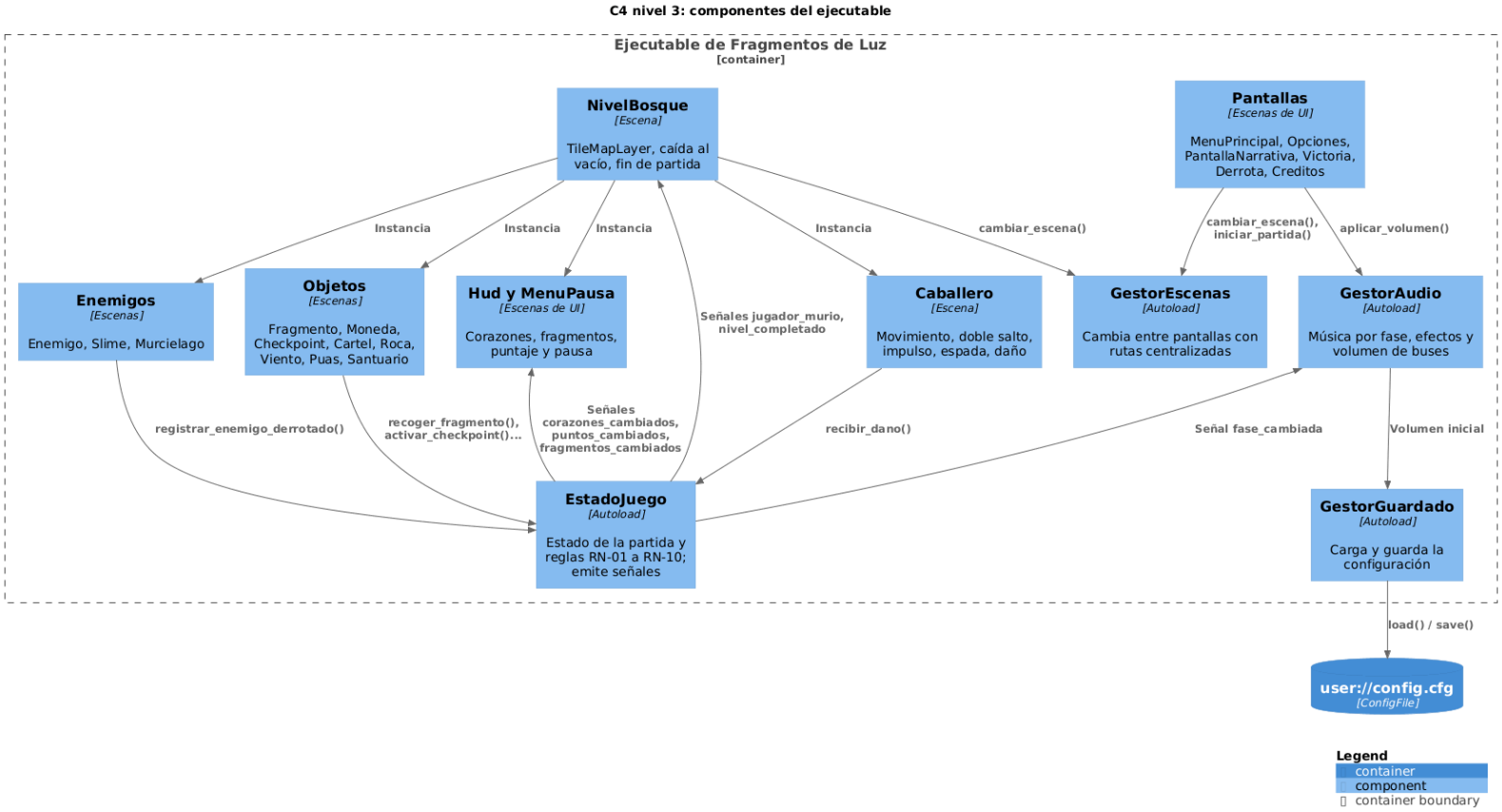
Nota. Elaboración propia.

4.1.3 Nivel 3: componentes del ejecutable

La Figura 16 muestra los componentes internos del ejecutable. La Tabla 38 describe los autoloads y la Tabla 39, las escenas principales.

Figura 16

Diagrama C4 de componentes



Nota. Elaboración propia.

Tabla 38

Componentes: autoloads

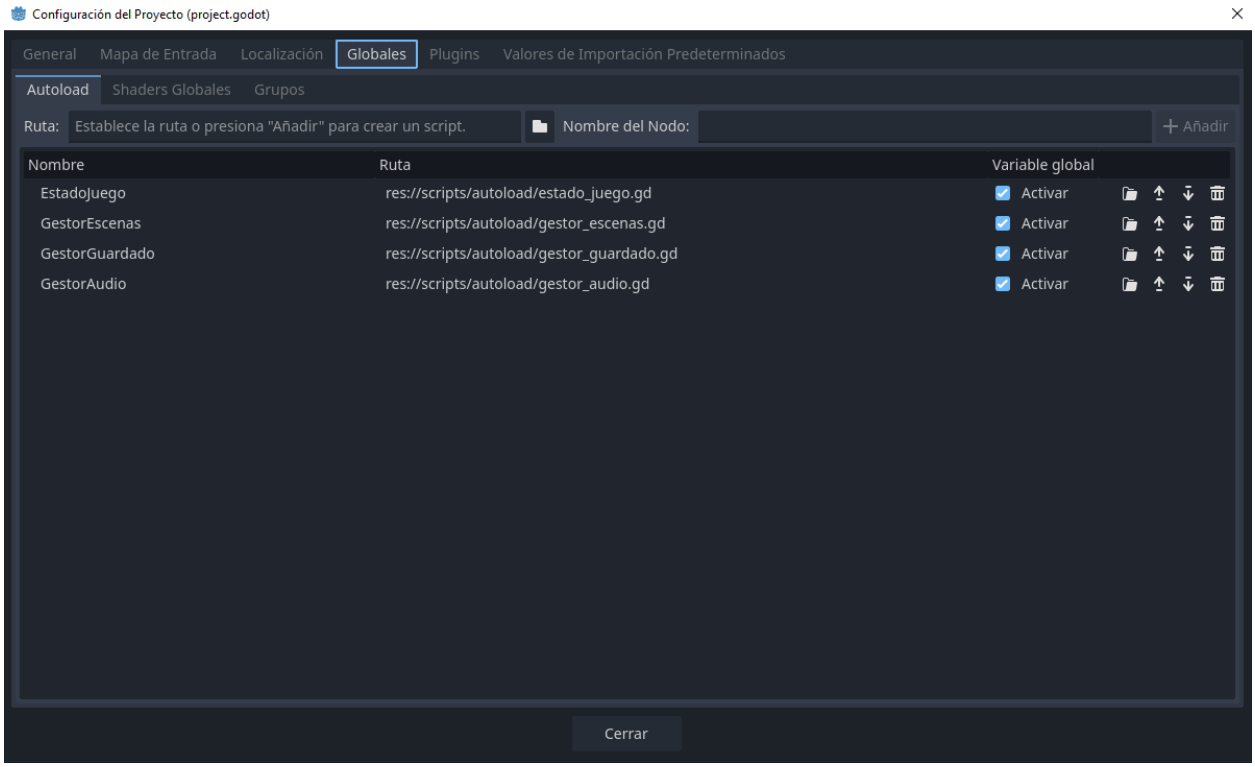
Componente	Script	Responsabilidad	Depende de
`EstadoJuego`	`scripts/autoload/estado_juego.gd`	Estado de la partida (corazones, puntos, fragmentos, punto de control, fase) y reglas RN-01 a RN-08 y RN-10; emite señales cuando cambia	—
`GestorEscenas`	`scripts/autoload/gestor_escenas.gd`	Cambia entre pantallas con rutas centralizadas; vuelve al menú si una escena no carga	`EstadoJuego`
`GestorGuardado`	`scripts/autoload/gestor_guardado.gd`	Carga y guarda `user://config.cfg`; maneja archivos ausentes o dañados; aplica RN-09	—
`GestorAudio`	`scripts/autoload/gestor_audio.gd`	Música según la fase, efectos y volumen de los buses `Musica` y `Efectos`	`EstadoJuego` (señal `fase_cambiada`), `GestorGuardado` (volumen inicial)

Nota. Elaboración propia.

La Figura 17 muestra los cuatro autoloads registrados en la configuración del proyecto, en su orden de carga y como variables globales.

Figura 17

Autoloads en Proyecto › Configuración del Proyecto › Globales



Nota. Captura del editor Godot 4.5, versión 1.0.0.

Tabla 39

Componentes: escenas principales

Componente	Escena y script	Responsabilidad	Depende de
Menú principal	`menu_principal.tscn` y `.gd`	Botones Jugar, Opciones, Créditos y Salir; muestra el mejor puntaje	`GestorEscenas`, `GestorGuardado`
Introducción y final	`intro.tscn` y `final.tscn`, ambas con `pantalla_narrativa.gd`	Pantallas de texto de la historia	`GestorEscenas`
Nivel	`nivel_bosque.tscn` y `.gd`	Contiene el terreno, el caballero, los enemigos, los objetos, el HUD y la pausa; detecta la caída al vacío	`EstadoJuego`, `GestorEscenas`
Caballero	`caballero.tscn` y `.gd`	Movimiento, doble salto, impulso, espada y máquina de estados	`EstadoJuego`, `GestorAudio`
Enemigos	`slime.tscn`, `murcielago.tscn`; scripts `enemigo.gd`, `slime.gd`, `murcielago.gd`	Comportamiento y vida de cada enemigo	`EstadoJuego`, `GestorAudio`
Objetos	`fragmento`, `moneda`, `checkpoint`, `santuario`, `cartel`, `roca`, `viento` y `puas` (`.tscn` y `.gd`)	Interacciones del nivel	`EstadoJuego`, `GestorAudio`
HUD	`hud.tscn` y `.gd`	Muestra corazones, fragmentos, puntaje, mensajes y el texto de los carteles	Señales de `EstadoJuego`
Pausa y opciones	`menu_pausa.tscn`, `opciones.tscn` y sus `.gd`	Menú de pausa y deslizadores de volumen	`EstadoJuego`, `GestorEscenas`, `GestorAudio`, `GestorGuardado`
Victoria, derrota y créditos	`victoria.tscn`, `derrota.tscn`, `creditos.tscn` y sus `.gd`	Pantallas de resultado y créditos	`EstadoJuego`, `GestorEscenas`, `GestorGuardado`

Nota. Elaboración propia.

Regla de dependencias: las escenas dependen de los autoloads, pero los autoloads nunca dependen de las escenas: comunican los cambios mediante señales (por ejemplo, el HUD escucha `EstadoJuego.corazones_cambiados`). El orden de carga en la configuración del proyecto es `EstadoJuego`, `GestorEscenas`, `GestorGuardado` y `GestorAudio`, porque `GestorAudio` toma el volumen inicial de `GestorGuardado`.

4.2 Patrones de diseño

La Tabla 40 justifica los cuatro patrones aplicados y, a continuación, se muestran los fragmentos del repositorio donde se ven.

Tabla 40

Patrones de diseño aplicados

Patrón	Problema que resuelve	Dónde se aplica (escena, script)	Alternativa considerada	Consecuencias (ventajas y costos)
Máquina de estados (Nystrom, 2014)	El caballero tiene acciones que se excluyen entre sí (no puede atacar durante el impulso ni saltar herido); con variables booleanas sueltas aparecen combinaciones inválidas y errores difíciles de rastrear	`caballero.gd` (enum `Estado` con 9 estados); `slime.gd` (`PATRULLANDO`, `HERIDO`, `MUERTO`); `murcielago.gd` (`DORMIDO`, `PERSIGUIENDO`, `REGRESANDO`, `MUERTO`); `estado_juego.gd` (enum `Fase`: `MENU`, `JUGANDO`, `PAUSA`, `VICTORIA`, `DERROTA`)	Varias variables booleanas, o una máquina con un nodo por estado	**Ventajas:** un solo estado activo, transiciones explícitas que coinciden con los diagramas de estados y estados que se pueden probar. **Costo:** un `match` grande en el caballero. Se eligió el enum frente a un nodo por estado por ser más simple para 9 estados en 5 días
Singleton mediante autoload (Godot Engine, s. f.-i)	Varias escenas necesitan el mismo estado (corazones, puntos) y los mismos servicios (audio, guardado, cambio de escena), y ese estado debe sobrevivir a los cambios de escena	`EstadoJuego`, `GestorEscenas`, `GestorGuardado` y `GestorAudio`, registrados como autoloads en la configuración del proyecto	Pasar referencias entre escenas, o guardar el estado en el nodo raíz del nivel	**Ventajas:** una única fuente de verdad, acceso simple y lógica que se prueba con GUT sin cargar escenas. **Costo:** estado global que puede acoplar; se mitiga porque los autoloads nunca referencian escenas y solo comunican por señales
Observador mediante señales (Godot Engine, s. f.-l)	El HUD, el audio y el nivel deben reaccionar cuando cambia el estado sin que el emisor conozca a los receptores	`EstadoJuego` emite `corazones_cambiados`, `puntos_cambiados`, `fragmentos_cambiados`, `jugador_murio`, `nivel_completado` y `fase_cambiada`; `Hud`, `NivelBosque` y `GestorAudio` se conectan; `Opciones` emite `cerrado`	Revisar los valores en cada `_process()`, o llamadas directas de `EstadoJuego` al HUD	**Ventajas:** desacoplamiento, el HUD solo se actualiza cuando hay cambios y se pueden agregar receptores sin tocar el emisor. **Costo:** el flujo es menos evidente al leer el código; se mitiga con el mapa de señales (sección 4.4.3)
Composición de escenas (Godot Engine, s. f.-h)	El nivel tiene muchos objetos repetidos y cada integrante trabaja en piezas distintas	`nivel_bosque.tscn` instancia `caballero`, `slime`, `murcielago`, `fragmento`, `moneda`, `checkpoint`, `cartel`, `roca`, `viento`, `puas`, `santuario`, `hud` y `menu_pausa`; `menu_principal.tscn` y `menu_pausa.tscn` instancian `opciones.tscn`	Construir todo dentro de una sola escena grande	**Ventajas:** piezas reutilizables y que se prueban por separado, menos conflictos de fusión (R-04) y un responsable por escena. **Costo:** más archivos que organizar; se mitiga con la estructura de carpetas del capítulo 5

Nota. Elaboración propia.

4.2.1 Fragmentos de código

Máquina de estados. El caballero declara sus estados en un enum y en cada cuadro de física decide su movimiento según el estado activo; `cambiar_estado()` es el único punto donde cambia el estado y la animación.

scripts/jugador/caballero.gd (líneas 7 a 7)

```
 7  enum Estado { QUIETO, CORRIENDO, SALTANDO, CAYENDO, DOBLE_SALTO, DASH, ATACANDO,
HERIDO, MUERTO }
```

scripts/jugador/caballero.gd (líneas 80 a 92)

```
80  func _physics_process(delta: float) -> void:
81      _enfriamiento_ataque_restante = maxf(_enfriamiento_ataque_restante - delta, 0.0)
82      match estado:
83          Estado.MUERTO:
84              velocity = Vector2.ZERO
85          Estado.DASH:
86              velocity = Vector2(direccion * velocidad_dash, 0.0)
87          Estado.HERIDO:
88              velocity.y += gravedad * delta
89          _:
90              _procesar_movimiento(delta)
91      move_and_slide()
92      _actualizar_estado_por_fisica()
```

scripts/jugador/caballero.gd (líneas 181 a 186)

```
181  func cambiar_estado(nuevo: Estado) -> void:
182      if estado == nuevo:
183          return
184      estado = nuevo
185      if is_node_ready():
186          _sprite.play(ANIMACIONES[estado])
```

Singleton y observador (emisor). El autoload `EstadoJuego` aplica las reglas RN-01, RN-02 y RN-07 y avisa los cambios con señales, sin conocer quién las escucha.

scripts/autoload/estado_juego.gd (líneas 53 a 64)

```
53  func recibir_dano(forzar: bool = false) -> bool:
54      if corazones <= 0:
55          return false
56      if esta_invulnerable() and not forzar:
57          return false
58      corazones -= 1
59      tiempo_invulnerable = DURACION_INVULNERABILIDAD
60      corazones_cambiados.emit(corazones)
61      if corazones == 0:
62          cambiar_fase(Fase.DERROTA)
63          jugador_murio.emit()
64      return true
```

Observador (receptor). El HUD se suscribe a las señales al iniciar.

scripts/ui/hud.gd (líneas 24 a 31)

```
24  func _ready() -> void:
25      add_to_group("hud")
26      _textura_llena = _corazones[0].texture as AtlasTexture
27      _textura_vacia = _textura_llena.duplicate() as AtlasTexture
28      _textura_vacia.region.position.x = 8.0
29      EstadoJuego.corazones_cambiados.connect(_on_corazones_cambiados)
30      EstadoJuego.fragmentos_cambiados.connect(_on_fragmentos_cambiados)
31      EstadoJuego.puntos_cambiados.connect(_on_puntos_cambiados)
```

Composición de escenas. El nivel no define a sus personajes: instancia las escenas de cada uno.

scenes/niveles/nivel_bosque.tscn (líneas 43 a 43)

```
43  [node name="Slime1" parent="Enemigos" instance=ExtResource("3_vwub4")]
```

scenes/niveles/nivel_bosque.tscn (líneas 193 a 198)

```
193  [node name="Caballero" parent="." instance=ExtResource("13_ru04k")]
194  position = Vector2(40, 392)
195
196  [node name="Hud" parent="." instance=ExtResource("14_6ykeb")]
197
198  [node name="MenuPausa" parent="." instance=ExtResource("15_a0orh")]
```

4.3 Modelos UML

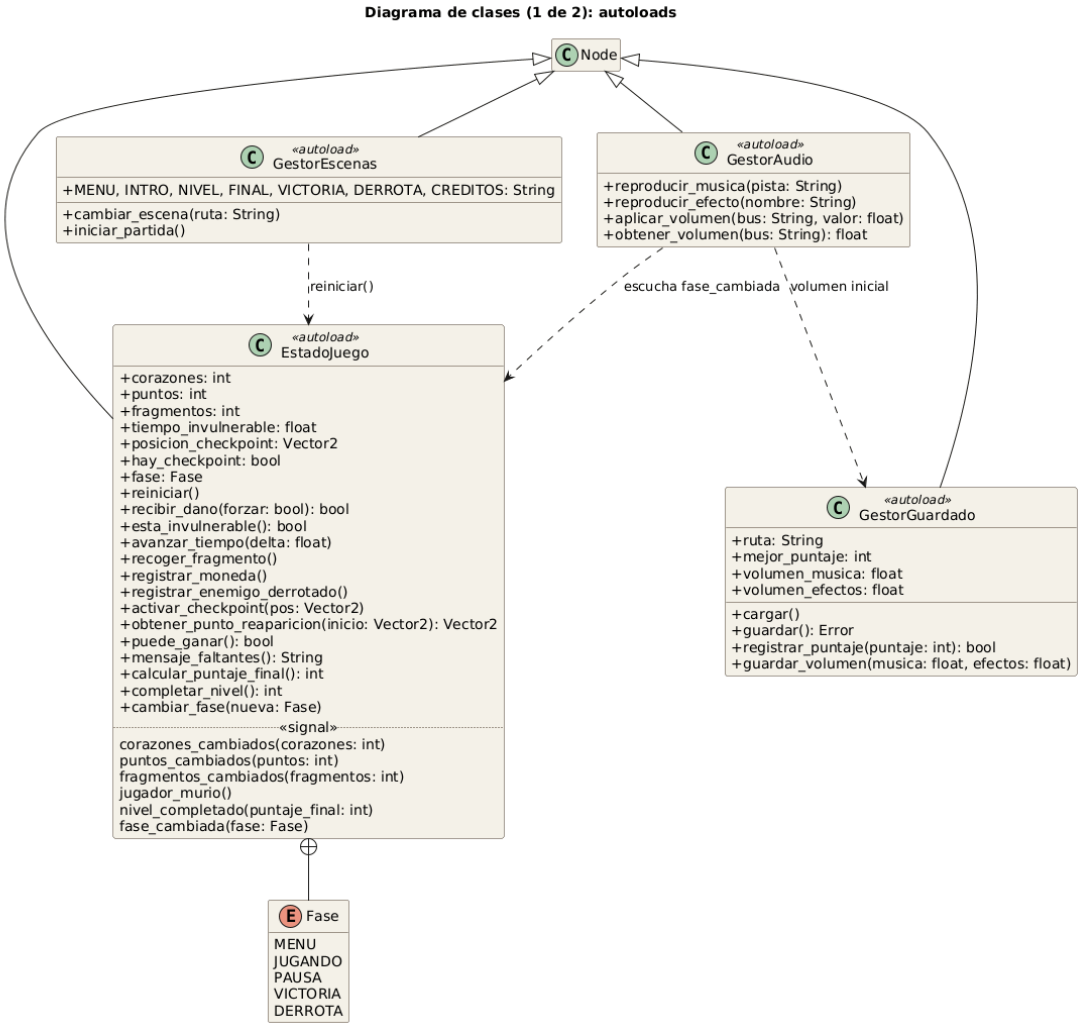
Los diagramas siguen la notación de UML 2.5.1 (OMG, 2017), se generaron con PlantUML y corresponden al código entregado: los nombres de clases, métodos, señales y estados son los mismos del repositorio. Sus fuentes están en `docs/diagramas/`.

4.3.1 Diagrama de clases

El diagrama de clases se divide en dos para que se pueda leer. La Figura 18 presenta los cuatro autoloads con sus atributos, métodos, señales (estereotipo «signal») y dependencias; la Figura 19 presenta los scripts de escena como clases, con su herencia del tipo de nodo que extienden y las composiciones entre escenas. Las dependencias de las escenas hacia los autoloads se muestran en el diagrama de componentes (Figura 16) y en el mapa de señales (sección 4.4.3).

Figura 18

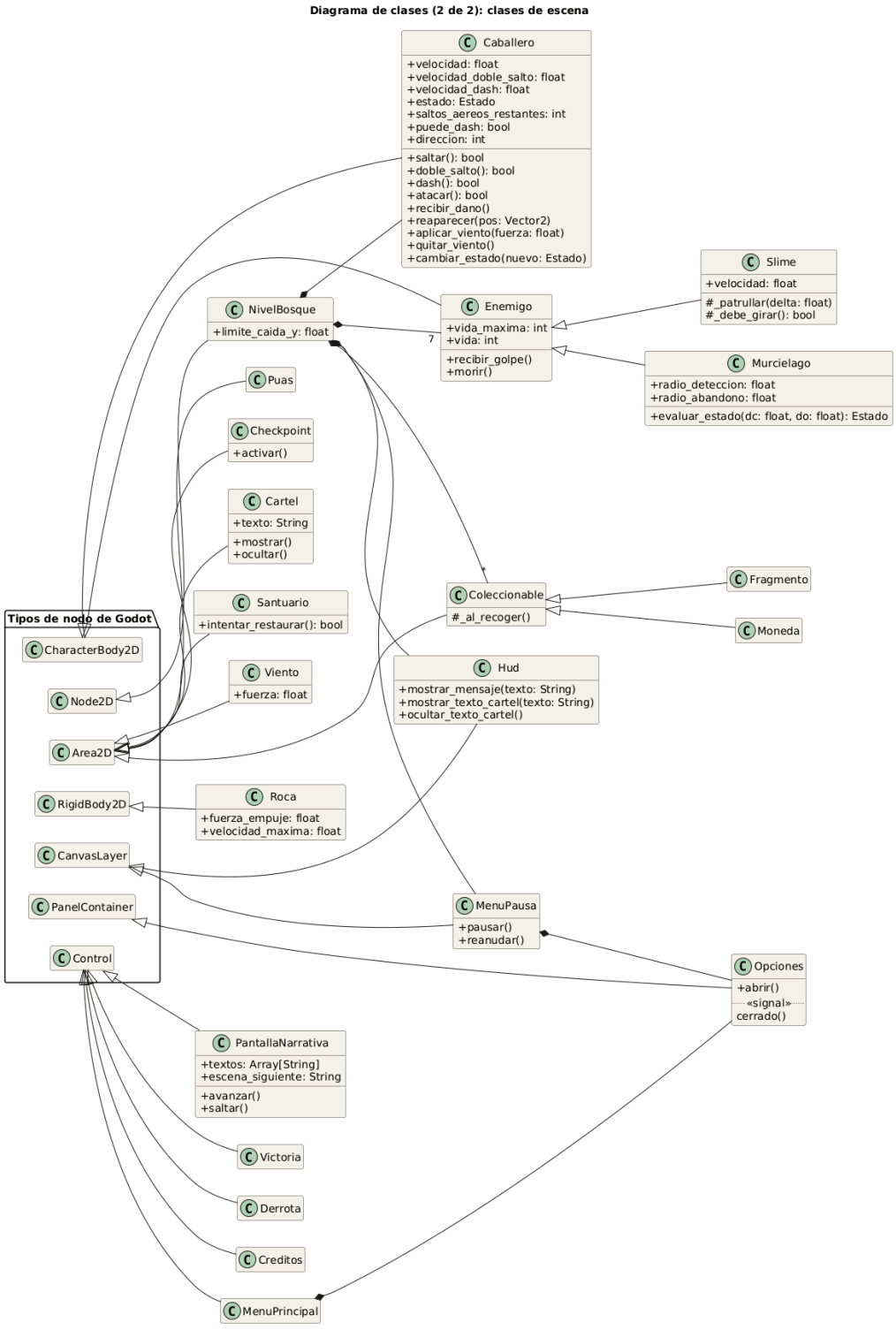
Diagrama de clases: autoloads



Nota. Elaboración propia.

Figura 19

Diagrama de clases: clases de escena



Nota. Elaboración propia.

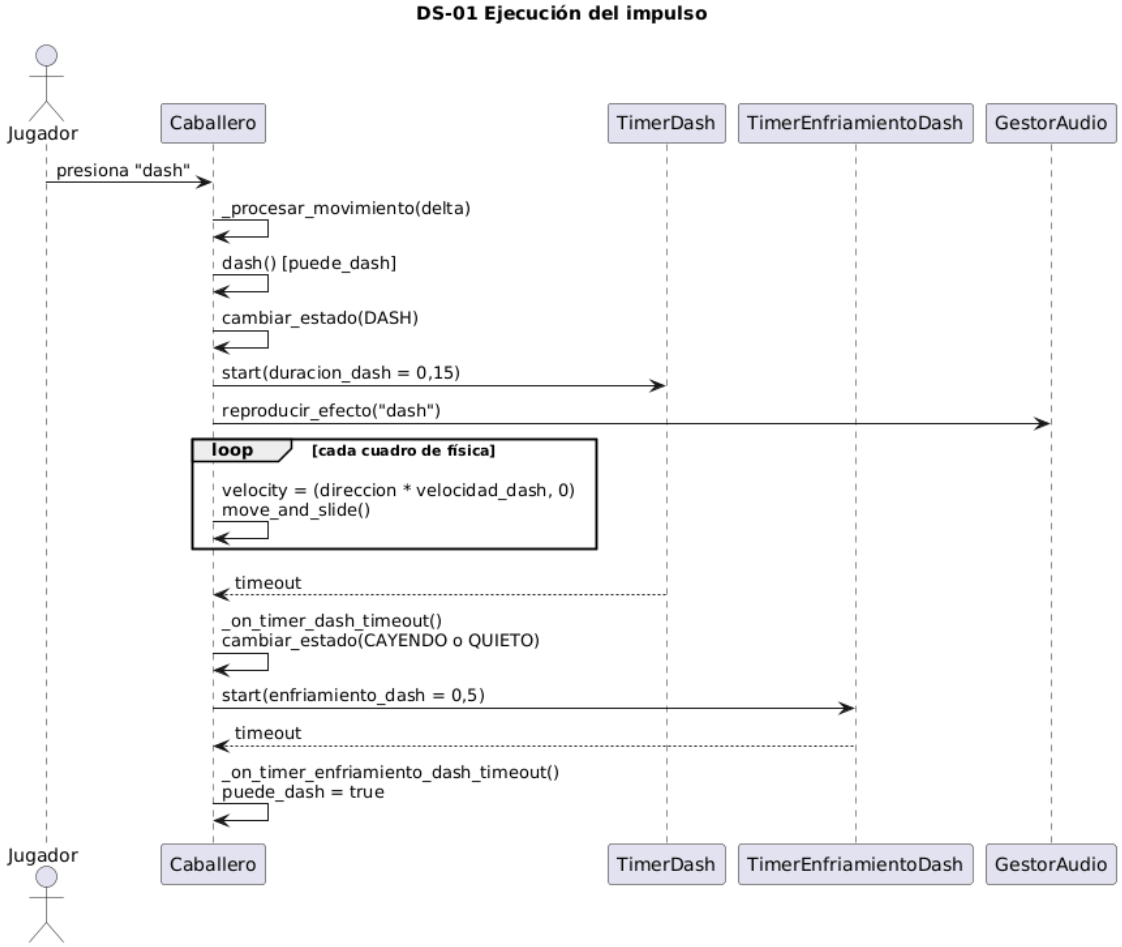
- **Clases base:** `Enemigo` reúne la vida y el daño de `Slime` y `Murcielago`; `Coleccionable` reúne el flote, el brillo y la recolección de `Fragmento` y `Moneda`, que solo sobrescriben `_al_recoger()`.
- **Una sola clase narrativa:** `PantallaNarrativa` sirve a `intro.tscn` y a `final.tscn`, configurada con `textos` y `escena_siguiente`.
- **Viento:** lo aplica el caballero con `aplicar_viento()` y `quitar_viento()`, porque un `CharacterBody2D` no recibe automáticamente la gravedad de un `Area2D`.
- **Roca:** su propio script la empuja cuando el caballero está dentro de su `AreaEmpuje` y camina hacia ella (`fuerza_empuje` = 900, velocidad máxima de 45 px/s), porque un `CharacterBody2D` no empuja un `RigidBody2D` por sí solo.
- **Autoloads:** extienden `Node` y no tienen `class_name`, porque el autoload ya es global.

4.3.2 Diagramas de secuencia

Las Figuras 20 a 23 presentan cuatro flujos: el impulso, el daño y la derrota, la recolección de un fragmento con la actualización del HUD, y el fin del nivel con el guardado del mejor puntaje.

Figura 20

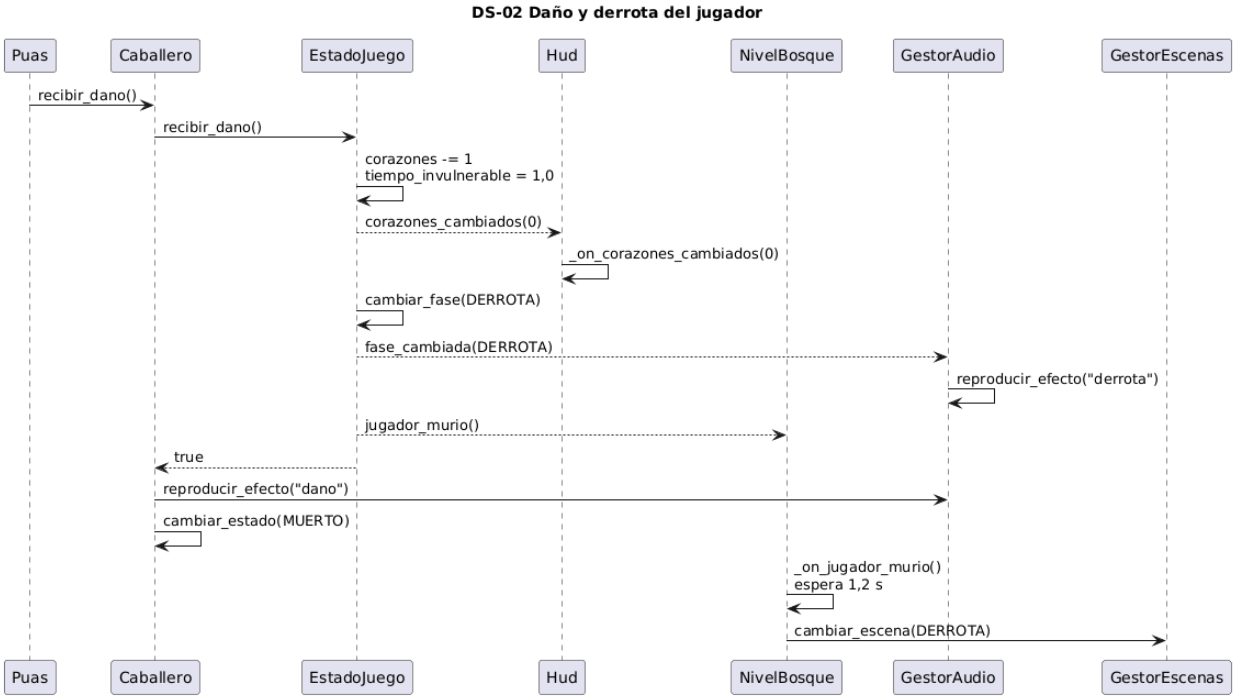
Diagrama de secuencia DS-01: ejecución del impulso



Nota. Elaboración propia.

Figura 21

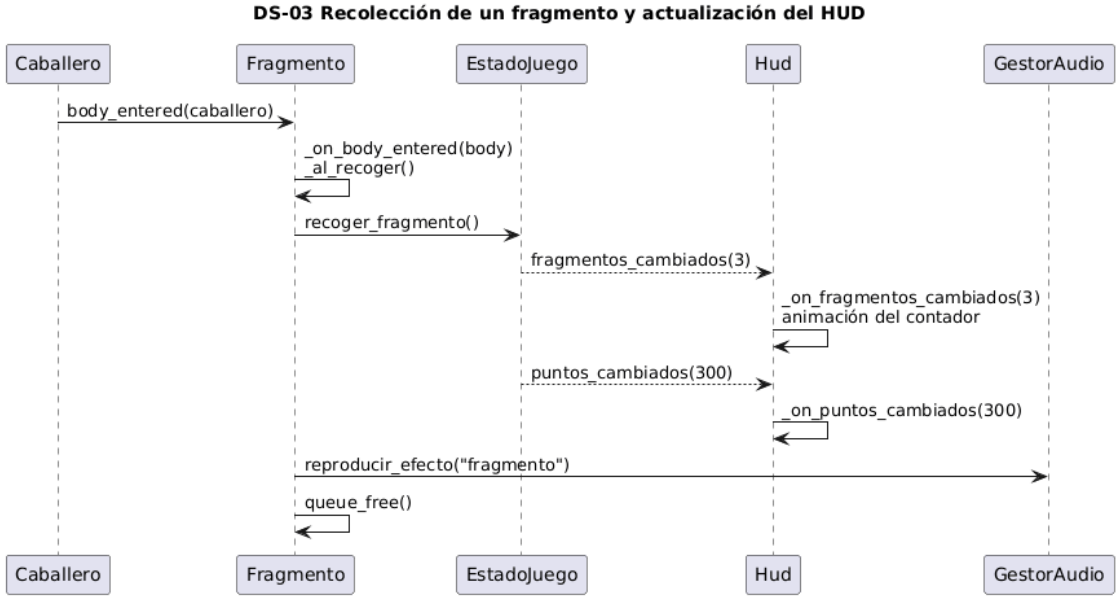
Diagrama de secuencia DS-02: daño y derrota del jugador



Nota. Elaboración propia.

Figura 22

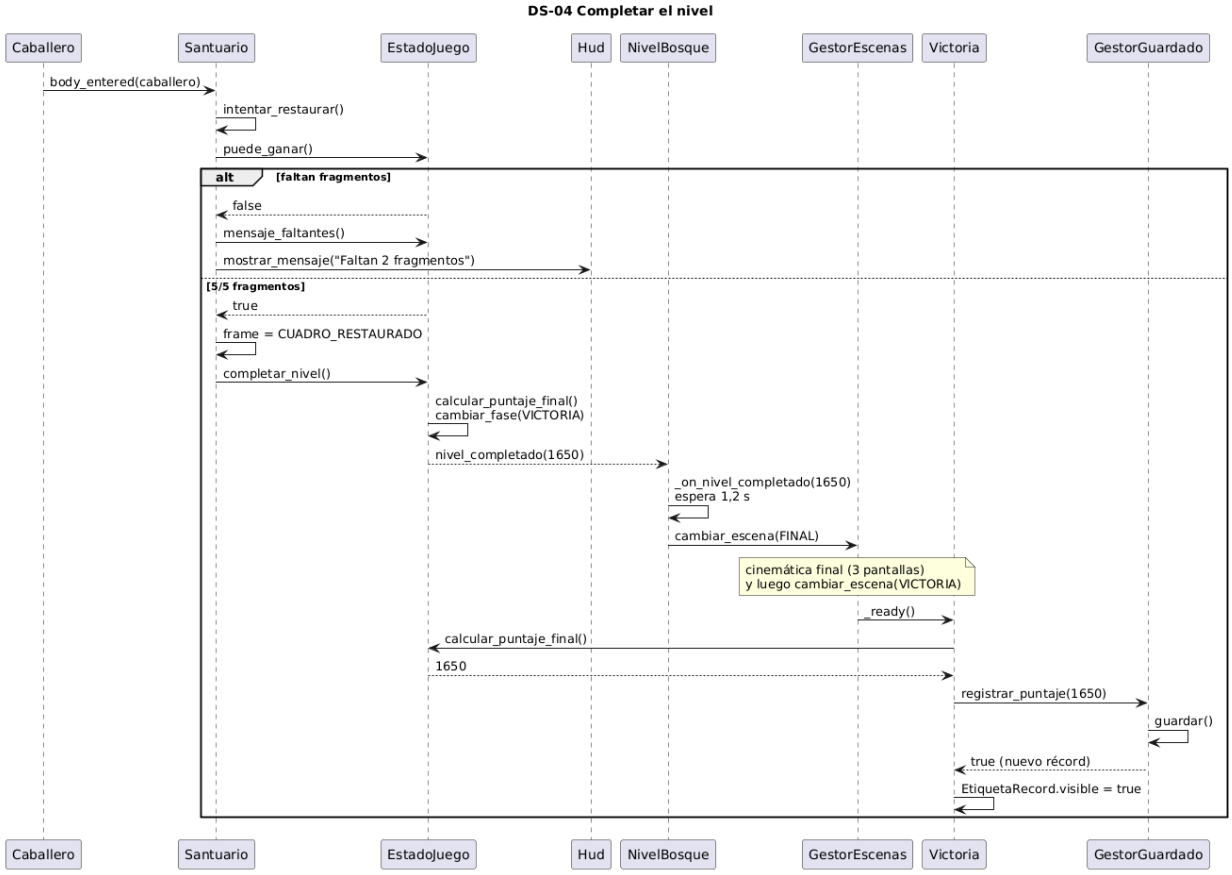
Diagrama de secuencia DS-03: recolección de un fragmento y actualización del HUD



Nota. Elaboración propia.

Figura 23

Diagrama de secuencia DS-04: completar el nivel y guardar el mejor puntaje



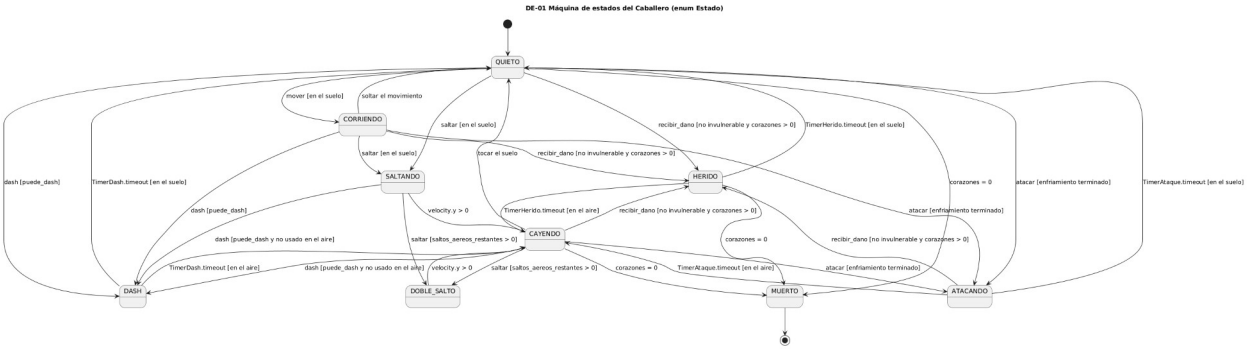
Nota. Elaboración propia.

4.3.3 Diagramas de estados

Las Figuras 24 a 27 presentan las máquinas de estados del caballero (incluidos el doble salto y el impulso), del murciélago, del slime y del ciclo del juego, con los eventos y las condiciones de guarda de cada transición.

Figura 24

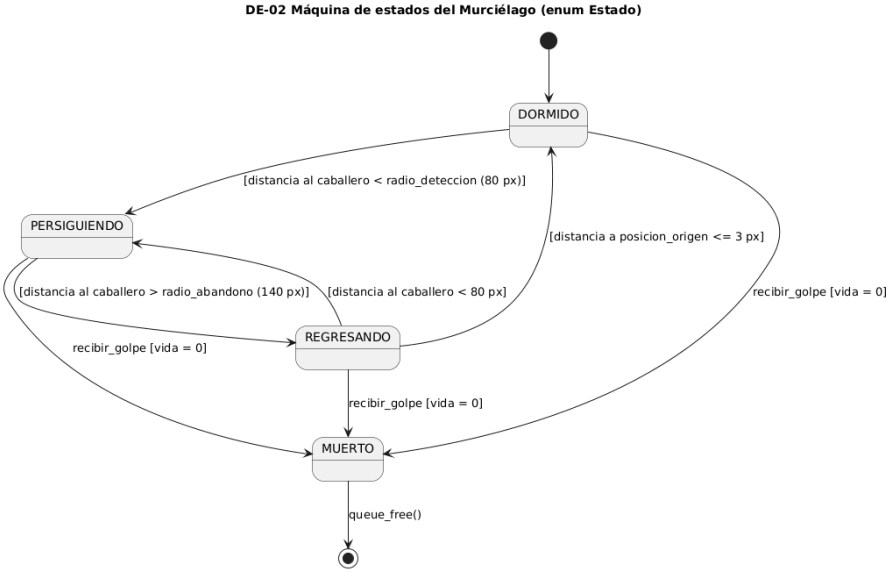
Diagrama de estados DE-01: caballero



Nota. Elaboración propia.

Figura 25

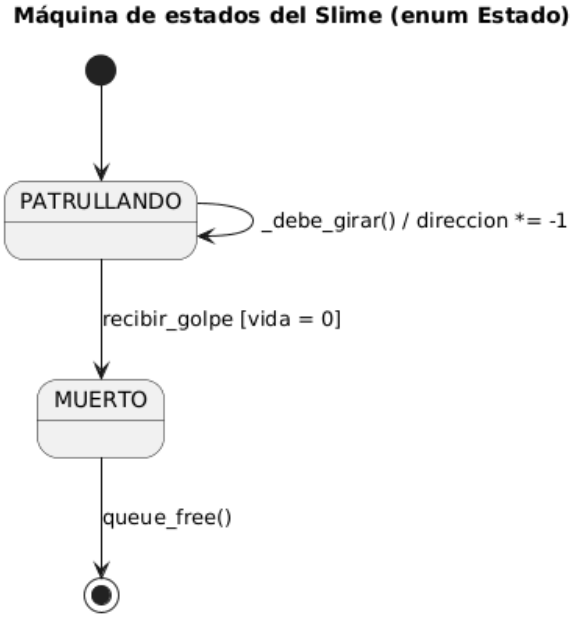
Diagrama de estados DE-02: murciélago



Nota. Elaboración propia.

Figura 26

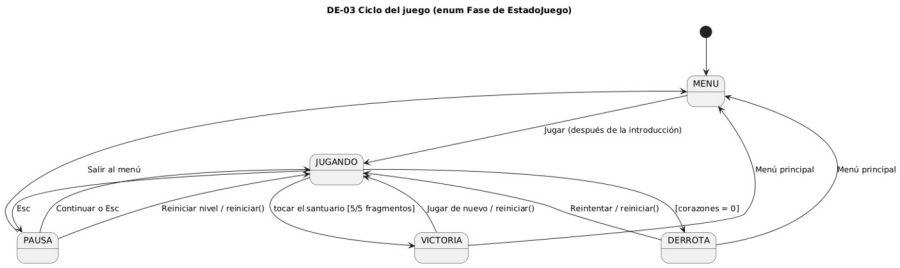
Diagrama de estados slime



Nota. Elaboración propia.

Figura 27

Diagrama de estados DE-03: ciclo del juego



Nota. Elaboración propia.

4.3.4 Diagrama de actividades

El diagrama de actividades del ciclo central de juego, con sus decisiones, ciclos y carriles por responsable (jugador y `EstadoJuego`), se presentó en la Figura 7 (sección 2.3.2).

4.4 Escenas, señales, entradas y colisiones

4.4.1 Árbol de nodos de las escenas principales

La Figura 28 muestra el árbol de nodos de las seis escenas principales, tal como aparecen en el panel Escena de Godot; en el nivel se agrupan las instancias repetidas.

Figura 28

Árbol de nodos de las escenas principales



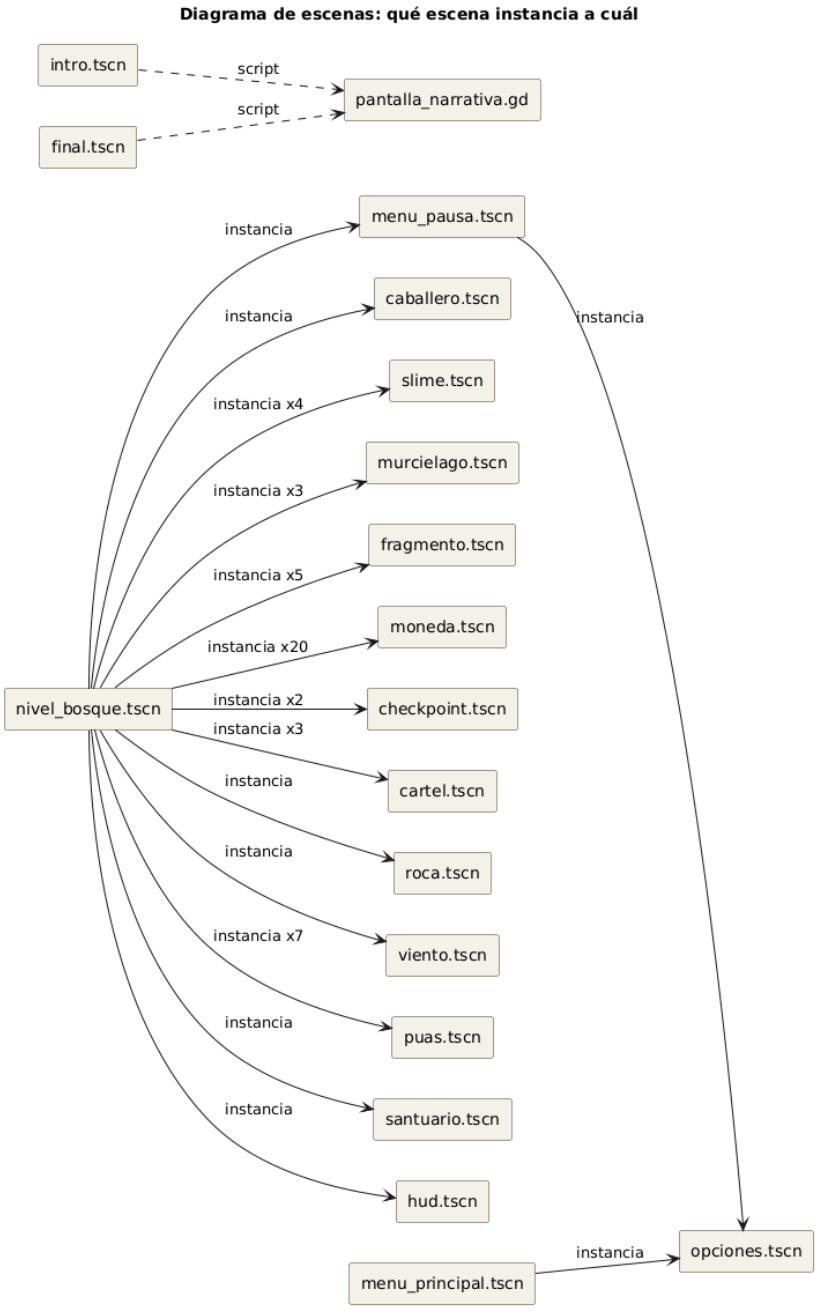
Nota. Elaboración propia a partir de los archivos .tscn del proyecto, versión 1.0.0.

4.4.2 Diagrama de escenas

La Figura 29 muestra qué escena instancia a cuál. Los cambios entre pantallas se hacen con `GestorEscenas.cambiar_escena()` y siguen el flujo de pantallas de la sección 2.6.2.

Figura 29

Diagrama de escenas (composición)



Nota. Elaboración propia.

4.4.3 Mapa de señales

La Tabla 41 lista todas las señales del proyecto. Ninguna conexión usa banderas especiales. Los objetos avisan a `EstadoJuego` llamando a sus métodos, y `EstadoJuego` emite las señales de estado.

Tabla 41

Mapa de señales

Emisor (escena/nodo)	Señal	Parámetros	Receptor (escena/nodo)	Método	Conexión (editor o código)
EstadoJuego	corazones_cambiados	corazones: int	Hud	_on_corazones_cambiados	Código (_ready)
EstadoJuego	puntos_cambiados	puntos: int	Hud	_on_puntos_cambiados	Código (_ready)
EstadoJuego	fragmentos_cambiados	fragmentos: int	Hud	_on_fragmentos_cambiados	Código (_ready)
EstadoJuego	jugador_murio	—	NivelBosque	_on_jugador_murio	Código (_ready)
EstadoJuego	nivel_completado	puntaje_final: int	NivelBosque	_on_nivel_completado	Código (_ready)
EstadoJuego	fase_cambiada	fase: Fase	GestorAudio	_on_fase_cambiada	Código (_ready)
Árbol de escenas (SceneTree)	node_added	node: Node	GestorAudio	_on_nodo_agregado (conecta cada botón)	Código (_ready)
Cualquier BaseButton	pressed	—	GestorAudio	reproducir_efecto("boton")	Código (bind)
Caballero/AreaEspada	body_entered	body: Node2D	Caballero	_on_area_espada_body_entered	Editor
Caballero/TimerDash, TimerEnfriamientoDash, TimerAtaque, TimerHerido	timeout	—	Caballero	_on_timer_dash_timeout, _on_timer_enfriamiento_dash_timeout, _on_timer_ataque_timeout, _on_timer_herido_timeout	Editor
Slime/AreaDano, Murcielago/AreaDano	body_entered, body_exited	body: Node2D	Enemigo (clase base)	_on_area_dano_body_entered, _on_area_dano_body_exited	Editor
Fragmento, Moneda	body_entered	body: Node2D	Coleccionable (clase base)	_on_body_entered	Editor
Checkpoint, Santuario	body_entered	body: Node2D	El mismo nodo	_on_body_entered	Editor
Cartel, Viento, Puas	body_entered, body_exited	body: Node2D	El mismo nodo	_on_body_entered, _on_body_exited	Editor
Roca/AreaEmpuje	body_entered, body_exited	body: Node2D	Roca	_on_area_empuje_body_entered, _on_area_empuje_body_exited	Editor
Hud/TimerMensaje	timeout	—	Hud	_on_timer_mensaje_timeout	Editor
Botones de MenuPrincipal, MenuPausa, Victoria, Derrota, Creditos y Opciones	pressed	—	La escena del botón	_on_boton_<nombre>_pressed	Editor
Opciones/Contenedor/SliderMusica, SliderEfectos	value_changed	value: float	Opciones	_on_slider_musica_value_changed, _on_slider_efectos_value_changed	Editor
Opciones	cerrado	—	MenuPrincipal, MenuPausa	_on_opciones_cerrado	Editor

Nota. Elaboración propia.

4.4.4 Mapa de entradas

La Tabla 42 lista las acciones definidas en la configuración del proyecto (mapa de entradas).

Tabla 42

Mapa de entradas

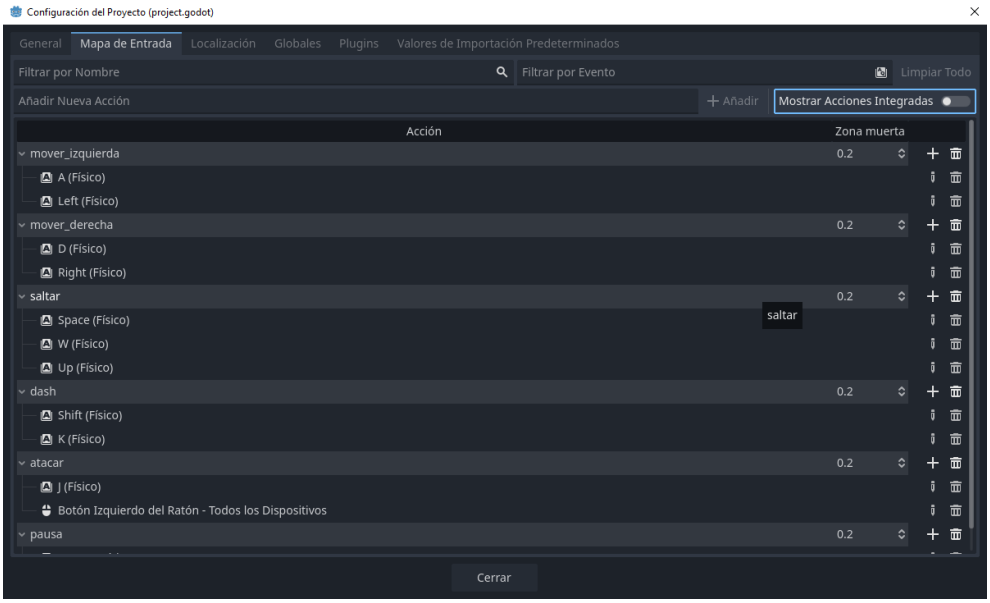
Acción	Teclado y ratón	Mando	Efecto en el juego	Requisito
`mover_izquierda`	A, flecha izquierda	No aplica (fuera de alcance)	Camina a la izquierda	RF-01
`mover_derecha`	D, flecha derecha	No aplica	Camina a la derecha	RF-01
`saltar`	Espacio, W, flecha arriba	No aplica	Salta; en el aire hace el doble salto	RF-01, RF-02
`dash`	Mayúsculas, K	No aplica	Impulso	RF-03
`atacar`	J, clic izquierdo	No aplica	Ataque con espada	RF-04
`pausa`	Esc	No aplica	Pausa y reanuda; salta la introducción y el final	RF-15, RF-16
`ui_accept` (de Godot)	Enter, Espacio	No aplica	Avanza los textos y confirma en los menús	RF-16, RF-18

Nota. Elaboración propia.

Las Figuras 30 y 31 muestran esas acciones en Proyecto › Configuración del Proyecto › Mapa de entrada.

Figura 30

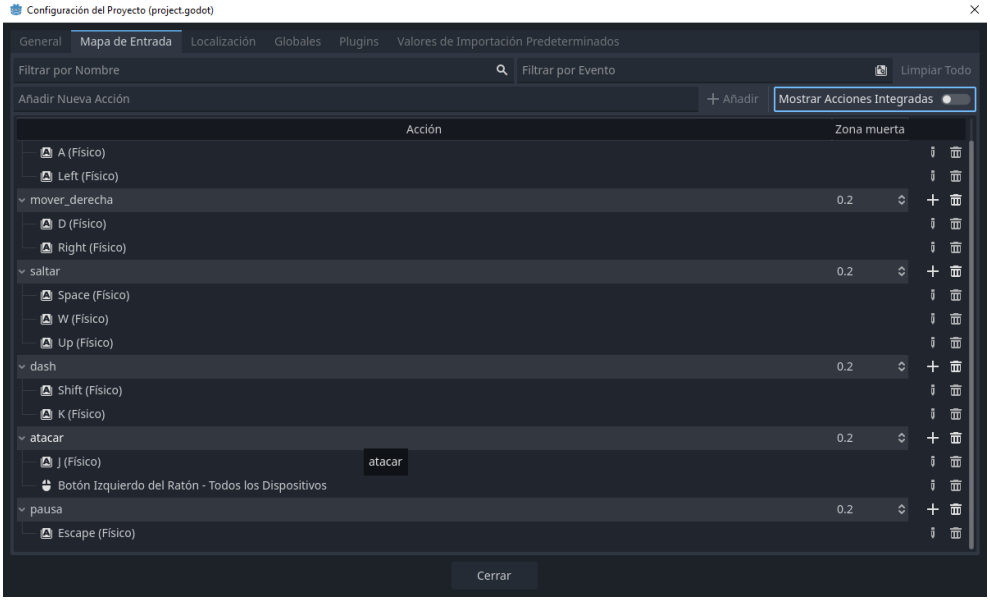
Mapa de entrada (primera parte)



Nota. Captura del editor Godot 4.5, versión 1.0.0.

Figura 31

Mapa de entrada (segunda parte)



Nota. Captura del editor Godot 4.5, versión 1.0.0.

4.4.5 Capas y máscaras de colisión

Las ocho capas tienen nombre en la configuración del proyecto (sección de nombres de capas de física 2D). La Tabla 43 indica qué nodos están en cada capa (`collision_layer`) y cuáles la detectan (`collision_mask`), según los valores de las escenas.

Tabla 43

Capas y máscaras de colisión

Capa	Nombre	Nodos en la capa (collision_layer)	Nodos que la detectan (collision_mask)	Propósito
1	`mundo`	`Terreno` (`TileMapLayer`): suelo, paredes y plataformas	`Caballero`, `Slime` y sus `RayCast2D`, `Murcielago`, `Roca`	Superficies sólidas
2	`jugador`	`Caballero`	`AreaDano` de los enemigos, `Fragmento`, `Moneda`, `Checkpoint`, `Santuario`, `Cartel`, `Viento`, `Puas`, `Roca/AreaEmpuje`	Que los objetos y peligros detecten solo al jugador
3	`enemigos`	`Slime`, `Murcielago`	`Caballero/AreaEspada`	Que la espada solo golpee enemigos
4	`objetos`	`Fragmento`, `Moneda`, `Checkpoint`, `Santuario`, `Cartel`	— (las áreas detectan, no son detectadas)	Agrupar coleccionables y activables
5	`peligros`	`Puas`, `AreaDano` de los enemigos	—	Fuentes de daño al jugador
6	`ataque_jugador`	`Caballero/AreaEspada`	—	Zona de golpe de la espada
7	`fisica`	`Roca`	`Caballero`, `Slime` y sus `RayCast2D`, `Roca`	Objeto físico empujable que sirve de escalón
8	`zonas`	`Viento`	—	Zonas que alteran el movimiento

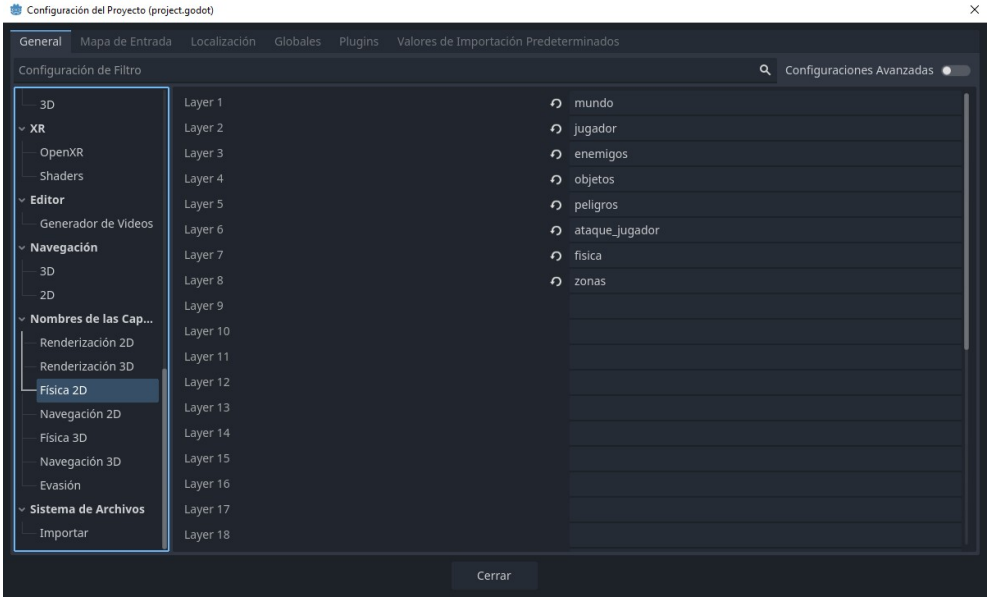
Nota. Elaboración propia.

El caballero no choca físicamente con los enemigos: el daño llega por el `AreaDano` (capa 5) de cada enemigo, que detecta la capa 2, para que el jugador no quede atrapado ni empujado por ellos.

La Figura 32 muestra los nombres de las ocho capas en la configuración del proyecto.

Figura 32

Nombres de las capas de física 2D



Nota. Captura del editor Godot 4.5, versión 1.0.0.

4.5 Modelo de datos y persistencia

4.5.1 Datos del juego

La Tabla 44 clasifica los datos del juego según dónde viven.

Tabla 44

Datos del juego

Tipo de dato	Dónde está	Contenido
Constantes de reglas	`estado_juego.gd`	`MAX_CORAZONES` = 3, `TOTAL_FRAGMENTOS` = 5, `PUNTOS_FRAGMENTO` = 100, `PUNTOS_MONEDA` = 10, `PUNTOS_ENEMIGO` = 50, `BONO_CORAZON` = 100, `DURACION_INVULNERABILIDAD` = 1.0
Parámetros ajustables	Variables `@export` de `caballero.gd`, `slime.gd`, `murcielago.gd`, `roca.gd`, `viento.gd`, `nivel_bosque.gd`	Velocidades, fuerzas, tiempos y radios (secciones 2.3 y 2.4), visibles en el Inspector
Configuración de escena	Propiedades en el Inspector	Masa de la roca, textos de los carteles (`@export_multiline texto`), textos de la introducción y del final (`textos`)
Recursos de Godot	Archivos `.tres`	Animaciones (`SpriteFrames`) de los personajes y tema de la interfaz (`assets/ui/tema.tres`)
Estado de la partida (no se guarda)	Autoload `EstadoJuego`	`corazones`, `puntos`, `fragmentos`, `tiempo_invulnerable`, `posicion_checkpoint`, `hay_checkpoint`, `fase`; se reinicia con `reiniciar()`
Estado persistente	`user://config.cfg` mediante `GestorGuardado`	Mejor puntaje y volúmenes

Nota. Elaboración propia.

Recursos personalizados: no se crean clases que extiendan `Resource`. Con un solo nivel y dos tipos de enemigo, los parámetros caben en variables `@export` y constantes; crear clases `Resource` agregaría archivos sin beneficio. Queda como trabajo futuro (por ejemplo, un `DatosEnemigo.tres` por tipo) si se agregan niveles o enemigos.

4.5.2 Persistencia

La Tabla 45 resume las decisiones de persistencia (Godot Engine, s. f.-g) y la Tabla 46, el esquema del archivo.

Tabla 45

Decisiones de persistencia

Aspecto	Decisión
Qué se guarda	Mejor puntaje, volumen de la música y volumen de los efectos
Formato	`ConfigFile` de Godot (texto tipo INI, con tipos)
Ruta	`user://config.cfg` (en Windows: `%APPDATA%\Godot\app_userdata\Fragmentos de Luz\config.cfg`)
Cuándo se carga	Una vez, en el `_ready()` de `GestorGuardado`, al iniciar el juego (CU-08)
Cuándo se guarda	Al ganar con un nuevo récord (RN-09, pantalla de victoria) y al presionar Volver en Opciones (CU-05)

Nota. Elaboración propia.

Tabla 46

Esquema del archivo de configuración

Sección	Clave	Tipo	Rango válido	Valor por defecto
`meta`	`version`	`int`	1	1
`progreso`	`mejor_puntaje`	`int`	≥ 0	0
`audio`	`volumen_musica`	`float`	0.0 a 1.0	0.8
`audio`	`volumen_efectos`	`float`	0.0 a 1.0	0.8

Nota. Elaboración propia.

Ejemplo del contenido de `user://config.cfg`:

user://config.cfg

```
[meta]
version=1

[progreso]
mejor_puntaje=1650

[audio]
volumen_musica=0.6
volumen_efectos=0.8
```

Control de versión del formato: `VERSION_FORMATO` = 1 se escribe en `meta/version`. Al cargar, si la versión coincide se leen los valores; si falta o es distinta, se usan los valores por defecto y el archivo se reescribe con la versión actual en el siguiente guardado.

La Tabla 47 describe el manejo de archivos ausentes o dañados (RNF-08).

Tabla 47

Manejo de archivos ausentes o dañados

Situación	Cómo se detecta	Comportamiento
El archivo no existe (primera ejecución)	`ConfigFile.load()` devuelve `ERR_FILE_NOT_FOUND`	Valores por defecto, sin mensaje de error; se crea en el primer guardado
El archivo está dañado	`load()` devuelve otro error (por ejemplo, `ERR_PARSE_ERROR`)	Valores por defecto y `push_warning()`; se sobrescribe en el siguiente guardado
Un valor con tipo o rango inválido	Validación al leer cada clave	Valor por defecto de esa clave o ajuste al rango con `clampf()` y `maxi()`
No se puede escribir	`save()` devuelve un error	`push_warning()`; el juego sigue con los valores en memoria (CU-04 E1, CU-05 E1)

Nota. Elaboración propia.

Aplicación del volumen: el valor lineal (0,0 a 1,0) se convierte con `linear_to_db()` y se aplica con `AudioServer.set_bus_volume_db()` a los buses `Musica` y `Efectos`; con 0 el bus se silencia.

Pruebas derivadas: cinco pruebas unitarias de `GestorGuardado` en `test/unit/test_gestor_guardado.gd`: carga sin archivo, archivo dañado, valores fuera de rango, guardar y recargar, y la regla RN-09. Usan un archivo de prueba y nunca tocan el guardado real del jugador.

4.6 Registros de decisiones de arquitectura (ADR)

Las Tablas 48 a 51 registran cuatro decisiones de arquitectura con el formato de Nygard (2011): título, estado, contexto, decisión y consecuencias. Las copias en Markdown están en

`docs/adr/` (`ADR-001-autoloads.md`, `ADR-002-configfile.md`, `ADR-003-renderizador.md` y `ADR-004-gut.md`).

Tabla 48

ADR-001: Gestión del estado global y de los servicios mediante autoloads

Campo	Contenido
ID, título y fecha	ADR-001 · Gestión del estado global y de los servicios mediante autoloads · 19 de octubre de 2026
Estado	Aceptada
Contexto	Los corazones, puntos y fragmentos deben conservarse al pasar del nivel a las pantallas de victoria o derrota; el audio, el guardado y el cambio de escena se usan desde muchas escenas; las reglas de negocio deben probarse con GUT sin cargar el nivel
Decisión	Se crean 4 autoloads: `EstadoJuego` (estado y reglas de negocio), `GestorEscenas`, `GestorGuardado` y `GestorAudio`. Los autoloads nunca referencian escenas: comunican los cambios con señales y las escenas los llaman a ellos
Consecuencias	**Positivas:** única fuente de verdad; el estado sobrevive a los cambios de escena; lógica que se prueba de forma aislada; HUD desacoplado por señales. **Negativas:** el estado global puede crecer sin control y el orden de carga importa (`GestorGuardado` antes que `GestorAudio`); se mitiga con la regla de dependencias (sección 4.1.3) y el orden definido en la configuración del proyecto
Alternativas descartadas	(1) Estado en el nodo raíz del nivel pasado a las pantallas siguientes: acopla escenas y complica el reinicio. (2) Un único autoload con todo: viola la responsabilidad única y dificulta las pruebas

Nota. Elaboración propia.

Tabla 49

ADR-002: Formato de persistencia ConfigFile en user://config.cfg

Campo	Contenido
ID, título y fecha	ADR-002 · Formato de persistencia ConfigFile en user://config.cfg · 19 de octubre de 2026
Estado	Aceptada
Contexto	Hay que guardar entre sesiones el mejor puntaje y dos volúmenes (RF-19), con control de versión del formato y manejo de archivos ausentes o dañados (RNF-08); los datos son pocos y planos
Decisión	Se usa `ConfigFile` con las secciones `meta`, `progreso` y `audio` en `user://config.cfg`, con la clave `meta/version`; `GestorGuardado` valida cada valor al leerlo
Consecuencias	**Positivas:** interfaz nativa de pocas líneas; conserva los tipos (`int`, `float`); archivo legible para depurar; errores de carga detectables por el código de retorno. **Negativas:** no sirve para estructuras anidadas complejas; guardar la partida a mitad del nivel obligaría a revisar esta decisión
Alternativas descartadas	(1) JSON con `FileAccess` y `JSON.parse_string()`: pierde el tipo (todos los números quedan como `float`) y exige más validación. (2) `Resource` con `ResourceSaver`: un `.tres` modificado puede ejecutar código al cargarse, un riesgo innecesario para datos del usuario

Nota. Elaboración propia.

Tabla 50

ADR-003: Uso del renderizador Compatibility

Campo	Contenido
ID, título y fecha	ADR-003 · Uso del renderizador Compatibility · 19 de octubre de 2026
Estado	Aceptada
Contexto	Juego 2D de arte de píxeles a 320 × 180 sin iluminación 3D ni efectos avanzados; debe funcionar en los equipos de los integrantes, de los usuarios de prueba y del docente, que pueden tener gráficos integrados antiguos (RNF-05), con al menos 60 FPS (RNF-01)
Decisión	Se configura el renderizador Compatibility (OpenGL 3.3 / OpenGL ES 3.0), valor `gl_compatibility` del ajuste `rendering/renderer/rendering_method`, desde la creación del proyecto
Consecuencias	**Positivas:** máxima compatibilidad de hardware, incluidos gráficos sin Vulkan; arranque más rápido; suficiente para 2D; permitiría exportar a la web en el futuro. **Negativas:** no tiene las funciones avanzadas de Forward+, que este juego no necesita
Alternativas descartadas	(1) Forward+ (Vulkan, valor por defecto): exige hardware compatible con Vulkan. (2) Mobile: pensado para móviles, sin ventaja para un juego 2D de PC

Nota. Elaboración propia.

Tabla 51

ADR-004: Uso de GUT 9.5.0 para las pruebas unitarias

Campo	Contenido
ID, título y fecha	ADR-004 · Uso de GUT 9.5.0 para las pruebas unitarias · 19 de octubre de 2026
Estado	Aceptada
Contexto	El parcial exige al menos 15 pruebas unitarias automatizadas, ejecutables por línea de comandos y con reporte (RNF-07), con GUT o gdUnit4; el equipo tiene poca experiencia con pruebas y cinco días (R-09)
Decisión	Se usa GUT 9.5.0 en `addons/gut/`, con las pruebas en `test/unit/` (archivos `test_*.gd` que extienden `GutTest`), ejecutadas con `gut_cmdln.gd` y con reporte JUnit XML en `docs/reporte_gut.xml`
Consecuencias	**Positivas:** pruebas en GDScript puro; ejecución por consola con un solo comando y sin variables de entorno; versión indicada para Godot 4.5. **Negativas:** el reporte es solo XML (gdUnit4 genera además HTML); se complementa con una captura del panel GUT
Alternativas descartadas	gdUnit4 6.0: más funciones (reporte HTML y dobles de prueba avanzados), pero exige configurar `GODOT_BIN` y `runtest`, y su interfaz es más extensa de aprender en el tiempo disponible

Nota. Elaboración propia.

5. Construcción

5.1 Estándar de codificación

El código sigue la guía de estilo oficial de GDScript (Godot Engine, s. f.-e). La Tabla 52 resume las convenciones con ejemplos reales del proyecto.

Tabla 52

Convenciones de nombres

Elemento	Convención	Ejemplo del proyecto
Archivos y carpetas	`snake_case`	`caballero.gd`, `nivel_bosque.tscn`, `scripts/autoload/`
Clases (`class_name`) y nodos	`PascalCase`	`class_name Caballero`, nodo `AreaEspada`
Funciones y variables	`snake_case`	`doble_salto()`, `saltos_aereos_restantes`
Funciones y variables privadas	Prefijo `_`	`_procesar_movimiento()`, `_reproductor_musica`
Constantes y valores de enum	`CONSTANT_CASE`	`MAX_CORAZONES`, `Estado.DOBLE_SALTO`
Señales	`snake_case` en tiempo pasado	`corazones_cambiados`, `jugador_murio`
Manejadores de señales	`_on_<nodo>_<señal>`	`_on_timer_dash_timeout()`
Sangría	Tabuladores	—

Nota. Elaboración propia.

Orden de declaraciones: `class_name`, `extends`, comentario `##` de la clase, señales, enums, constantes, variables `@export`, variables públicas, variables privadas, variables `@onready`, `_ready()`, otros métodos virtuales, métodos públicos y métodos privados.

Reglas adicionales del grupo:

1. Tipado estático obligatorio en variables, parámetros y retornos (Godot Engine, s. f.-j), con el aviso `untyped_declaration` activado en la configuración del proyecto (RNF-06).
2. Comentarios de documentación `##` (Godot Engine, s. f.-d) en cada clase, función pública y variable `@export`.
3. Identificadores en español, sin tildes ni eñes (`dano`, no `daño`).
4. Sin números mágicos: los valores de juego son constantes con nombre o variables `@export` (las excepciones conocidas están en la deuda técnica, DT-07).
5. Ningún script supera las 300 líneas (el más largo, `caballero.gd`, tiene 273; ver DT-03).
6. Cero advertencias del analizador de GDScript antes de abrir una solicitud de integración.

Verificación: cada solicitud de integración la revisa otro integrante con la lista de chequeo de la plantilla `.github/pull_request_template.md` (estilo, tipado, comentarios `##` y

advertencias); antes de cada versión publicada se abre el proyecto en el editor y se confirma que no hay advertencias. Ejemplo real del estándar aplicado:

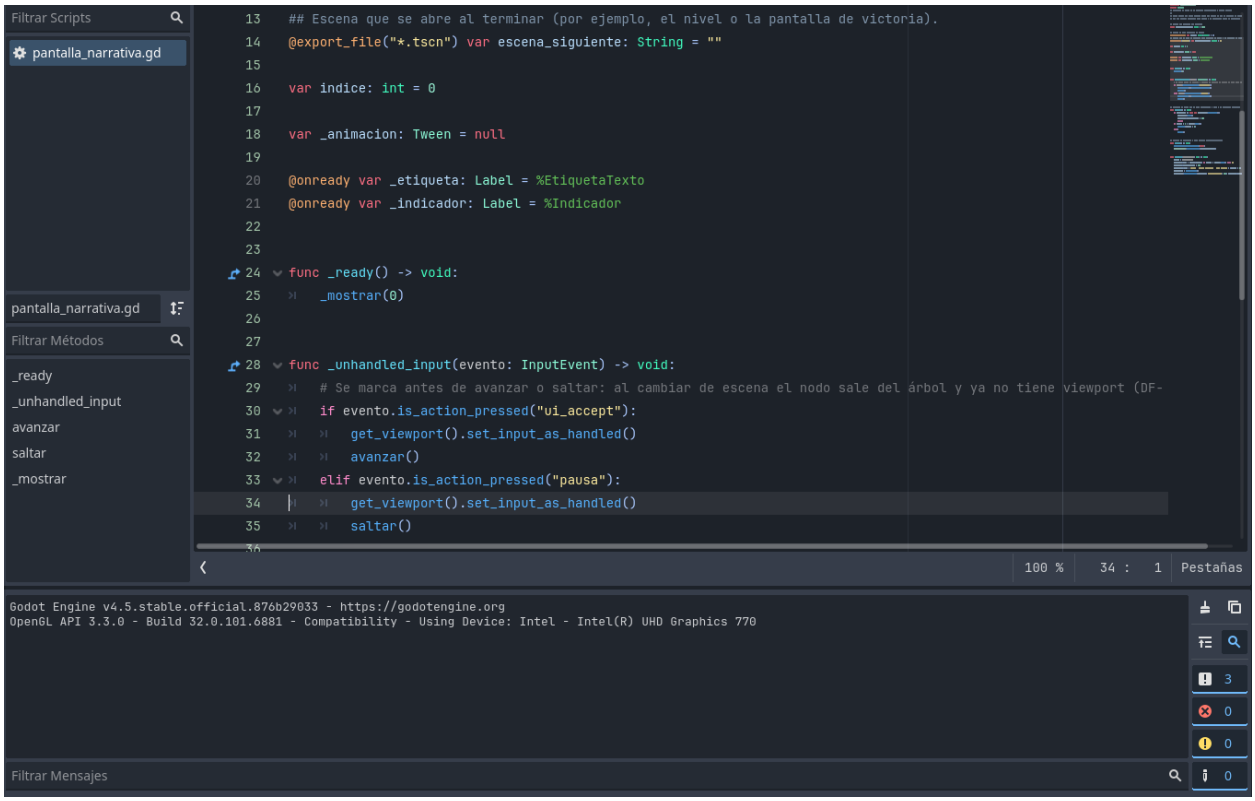
scripts/jugador/caballero.gd (líneas 1 a 24)

```
  1  class_name Caballero
  2  extends CharacterBody2D
  3  ## Personaje jugable (Sir Aldric): movimiento, salto, doble salto, dash, ataque con
espada y daño.
  4  ##
  5  ## El comportamiento se organiza con una máquina de estados (enum [enum Estado]).
  6
  7  enum Estado { QUIETO, CORRIENDO, SALTANDO, CAYENDO, DOBLE_SALTO, DASH, ATACANDO,
HERIDO, MUERTO }
  8
  9  const ANIMACIONES: Dictionary = {
 10      Estado.QUIETO: "quieto", Estado.CORRIENDO: "caminar", Estado.SALTANDO: "saltar",
 11      Estado.CAYENDO: "caer", Estado.DOBLE_SALTO: "doble_salto", Estado.DASH: "dash",
 12      Estado.ATACANDO: "atacar", Estado.HERIDO: "herido", Estado.MUERTO: "muerto",
 13  }
 14  ## Distancia horizontal del área de la espada al centro del caballero, en px.
 15  const ALCANCE_ESPADA_X: float = 10.0
 16  ## Intervalo del parpadeo durante la invulnerabilidad, en segundos.
 17  const INTERVALO_PARPADEO: float = 0.1
 18  ## Velocidad máxima de subida dentro de una corriente de viento, en px/s (RF-10).
 19  const VELOCIDAD_MAXIMA_VIENTO: float = 120.0
 20
 21  ## Velocidad horizontal al caminar, en px/s (RF-01).
 22  @export var velocidad: float = 100.0
 23  ## Velocidad vertical inicial del salto desde el suelo, en px/s (RF-01).
 24  @export var velocidad_salto: float = 260.0
```

La comprobación sobre la integración de la versión 1.0.0 (21 de octubre de 2026) encontró un error de ejecución en la pantalla de introducción, registrado y corregido como DF-05 (sección 6.6). Después de la corrección, el panel Salida no muestra errores ni advertencias (Figura 33).

Figura 33

Editor sin errores ni advertencias después de ejecutar el juego



Nota. Captura del editor Godot 4.5, versión 1.0.0.

5.2 Estructura de carpetas y nombres

La estructura del repositorio sigue la organización mínima que pide el enunciado y la recomendada por Godot (Godot Engine, s. f.-f):

Estructura del repositorio en la versión 1.0.0

```
FragmentosDeLuz/
├── project.godot   export_presets.cfg   icon.svg
├── README.md   CHANGELOG.md   .gitignore   .gitattributes
├── .github/                pull_request_template.md
├── addons/gut/             GUT 9.5.0
├── assets/
│   ├── audio/              12 efectos .wav
│   ├── musica/             musica_menu.mp3, musica_nivel.mp3
│   ├── sprites/            caballero/, enemigos/, objetos/, tileset/, ui/, narrativa/
│   ├── fuentes/            PixelOperator8.ttf y PixelOperator8-Bold.ttf
│   ├── ui/                 tema.tres
│   └── LICENCIAS.md
├── scenes/
│   ├── jugador/            caballero.tscn
│   ├── enemigos/           slime.tscn, murcielago.tscn
│   ├── objetos/            fragmento, moneda, checkpoint, santuario, cartel, roca, viento, puas (.tscn)
│   ├── niveles/            nivel_bosque.tscn
│   ├── narrativa/          intro.tscn, final.tscn
│   └── ui/                 menu_principal, hud, menu_pausa, opciones, victoria, derrota, creditos (.tscn)
├── scripts/
│   ├── autoload/           estado_juego.gd, gestor_escenas.gd, gestor_guardado.gd, gestor_audio.gd
│   ├── jugador/            caballero.gd
│   ├── enemigos/           enemigo.gd, slime.gd, murcielago.gd
│   ├── objetos/            coleccionable.gd y un .gd por objeto
│   ├── niveles/            nivel_bosque.gd
│   ├── narrativa/          pantalla_narrativa.gd
│   └── ui/                 menu_principal, hud, menu_pausa, opciones, victoria, derrota, creditos (.gd)
├── herramientas/           arte/ (Python), nivel/ (construir_nivel.tscn), capturas/
├── test/unit/              9 archivos test_*.gd (42 pruebas)
└── docs/
    ├── Parcial_Grupo2_FragmentosDeLuz.pdf
    ├── diagramas/          fuentes .puml y sus .png
    ├── adr/                ADR-001 a ADR-004 (.md)
    ├── evidencias/         actas, capturas y pruebas de juego
    └── reporte_gut.xml
```

Convenciones: cada escena tiene un script con el mismo nombre en la carpeta espejo de `scripts/` (excepciones: `pantalla_narrativa.gd`, compartido por `intro.tscn` y `final.tscn`, y las clases base `enemigo.gd` y `coleccionable.gd`); las pruebas se llaman `test_<lo_que_prueba>.gd`; los recursos de terceros llevan un nombre descriptivo en español y se registran en `assets/LICENCIAS.md`. La carpeta `herramientas/` y la carpeta `test/` no se incluyen en el ejecutable exportado.

Alternativa considerada: organizar por funcionalidad (escena y script juntos), que Godot también sugiere. Se separaron `scenes/` y `scripts/` porque es la estructura mínima que pide el enunciado (sección 3.1).

5.3 Control de versiones

El repositorio es público: https://github.com/jaider23100/FragmentosDeLuz. Camilo, Kevin y Diego son colaboradores con permiso de escritura y cada integrante sube su trabajo desde su propia cuenta.

5.3.1 Estrategia de ramas

La Tabla 53 resume los tipos de rama, con ejemplos reales del repositorio.

Tabla 53

Estrategia de ramas

Rama	Uso	Ejemplo del repositorio
`main`	Protegida; solo recibe cambios por solicitud de integración aprobada; de aquí salen las etiquetas	—
`feature/HU-xx-descripcion`	Una o varias historias de usuario relacionadas	`feature/HU-08-09-espada-vida-y-dano`
`fix/DF-xx-descripcion`	Corrección de un defecto registrado (capítulo 6)	Según los defectos encontrados
`docs/descripcion`	Documentación y diagramas	`docs/diagramas-planeacion`
`test/descripcion`	Pruebas y reportes	`test/reporte-gut-v1.0.0`
`chore/descripcion`	Configuración y versiones publicadas	`chore/release-v0.1.0`

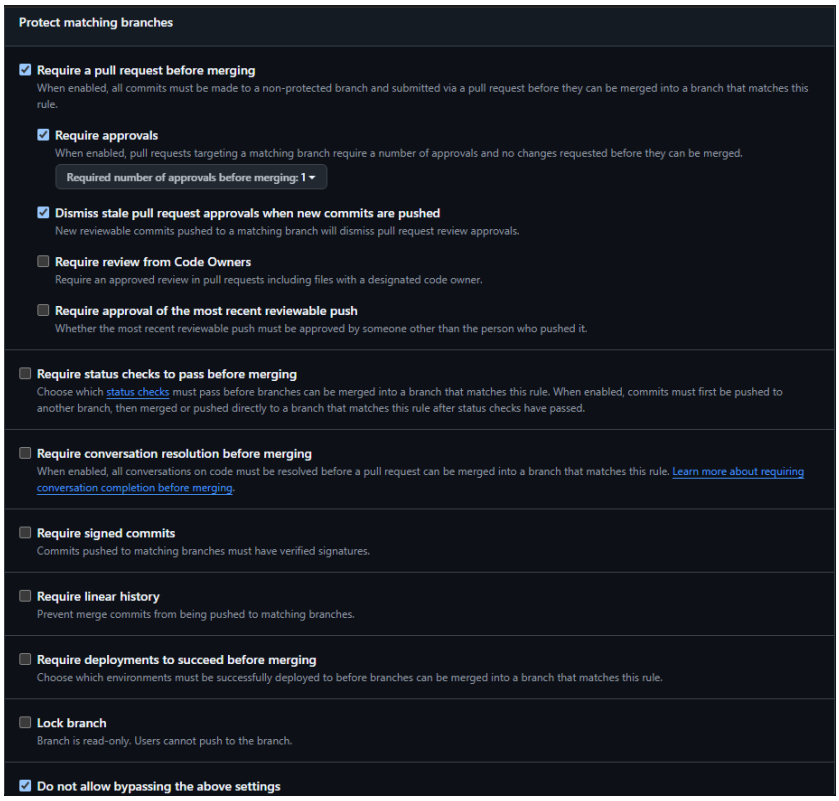
Nota. Elaboración propia.

Reglas de integración: (1) nadie integra directamente en `main`: la protección está activa también para los administradores y no permite forzar el historial ni borrar la rama; (2) cada solicitud de integración exige una aprobación de un integrante distinto del autor, y las aprobaciones se descartan si llegan confirmaciones nuevas; (3) las ramas son de corta duración, se crean desde `main` actualizado y se borran después de la fusión, que se hace con la opción «Squash and merge» de GitHub para reunir las confirmaciones de la rama en una sola; (4) antes de abrir la solicitud se ejecutan las pruebas GUT; (5) cada escena la edita solo su responsable (R-04).

La Figura 34 muestra la regla de protección de `main` en la configuración del repositorio.

Figura 34

Regla de protección de la rama main en GitHub



Nota. Captura de GitHub (configuración del repositorio).

5.3.2 Convención de confirmaciones

Los mensajes siguen Conventional Commits 1.0.0 (Conventional Commits, s. f.): `tipo(ámbito): descripción`, en minúscula y en presente. La Tabla 54 lista los tipos con ejemplos reales.

Tabla 54

Tipos de confirmación

Tipo	Uso	Ejemplo del repositorio
`feat`	Funcionalidad nueva	`feat(enemigos): agrega murciélago perseguidor con máquina de estados`
`fix`	Corrección de un defecto	`fix(<ámbito>): <corrección>` (según los defectos del capítulo 6)
`test`	Pruebas	`test(persistencia): agrega pruebas de GestorGuardado`
`docs`	Documentación	`docs(diagramas): agrega modelo C4 y diagramas UML…`
`refactor`	Cambio interno sin cambiar el comportamiento	`refactor(enemigos): extrae clase base Enemigo`
`chore`	Configuración y versiones	`chore(release): prepara la versión 1.0.0`

Nota. Elaboración propia.

Ámbitos usados: `jugador`, `enemigos`, `fisica`, `nivel`, `objetos`, `narrativa`, `audio`, `ui`, `estado`, `persistencia`, `release`, `adr`, `diagramas` y `qa`.

5.3.3 Solicitudes de integración revisadas

Todo cambio entra a `main` mediante una solicitud de integración revisada y aprobada por un integrante distinto del autor (GitHub, s. f.-a), con la plantilla `.github/pull_request_template.md` (historia o defecto, qué cambia, cómo probarlo y lista de chequeo del estándar y de la trazabilidad). La revisión es cruzada y rotativa (Tabla 55) para que todos conozcan el código de los demás (R-07).

Tabla 55

Rotación de revisores por sprint

Sprint	Jaider revisa a	Camilo revisa a	Kevin revisa a	Diego revisa a
1	Kevin	Jaider	Diego	Camilo
2	Diego	Kevin	Camilo	Jaider
3	Camilo	Diego	Jaider	Kevin

Nota. Elaboración propia.

La Tabla 56 lista las 24 solicitudes de integración del proyecto, muy por encima del mínimo de ocho, con todos los integrantes como autores y como revisores.

Tabla 56

Solicitudes de integración revisadas

N.º	Título	Rama	Autor	Revisor	Sprint	Enlace
1	feat(jugador): movimiento, doble salto y dash del caballero (HU-01 a HU-03)	`feature/HU-01-03-movimiento-caballero`	Jaider	Camilo	1	https://github.com/jaider23100/FragmentosDeLuz/pull/24
2	docs(diagramas): EDT, cronograma de Gantt y casos de uso	`docs/diagramas-planeacion`	Jaider	Camilo	1	https://github.com/jaider23100/FragmentosDeLuz/pull/25
3	test(estado): pruebas automatizadas (HU-07)	`feature/HU-07-pruebas-automatizadas`	Diego	Kevin	1	https://github.com/jaider23100/FragmentosDeLuz/pull/26
4	docs(adr): agrega ADR-001 a ADR-004	`docs/adr-iniciales`	Diego	Jaider	1	https://github.com/jaider23100/FragmentosDeLuz/pull/27
5	Feature/hu 04 slime patrullero	`feature/HU-04-slime-patrullero`	Kevin	Jaider	1	https://github.com/jaider23100/FragmentosDeLuz/pull/28
6	feat(ui): menú principal (HU-06)	`feature/HU-06-menu-principal`	Diego	Kevin	1	https://github.com/jaider23100/FragmentosDeLuz/pull/29
7	Feature/hu 05 checkpoints y nivel	`feature/HU-05-checkpoints-y-nivel`	Camilo	Diego	1	https://github.com/jaider23100/FragmentosDeLuz/pull/30
8	chore(release): versión 0.1.0	`chore/release-v0.1.0`	Diego	Kevin	1	https://github.com/jaider23100/FragmentosDeLuz/pull/31
9	feat(jugador): espada, vida y daño del caballero (HU-08, HU-09)	`feature/HU-08-09-espada-vida-y-dano`	Jaider	Diego	2	https://github.com/jaider23100/FragmentosDeLuz/pull/33
10	feat(física): roca, viento y púas (HU-11, HU-12)	`feature/HU-11-12-roca-viento-y-puas`	Kevin	Camilo	2	https://github.com/jaider23100/FragmentosDeLuz/pull/34
11	feat(ui): HUD y menú de pausa (HU-17, HU-19)	`feature/HU-17-19-hud-y-pausa`	Diego	Jaider	2	https://github.com/jaider23100/FragmentosDeLuz/pull/32
12	feat(nivel): fragmentos, monedas y nivel con tileset (HU-13, HU-14)	`feature/HU-13-14-fragmentos-y-nivel`	Camilo	Kevin	2	https://github.com/jaider23100/FragmentosDeLuz/pull/35
13	feat(audio): música y efectos de sonido (HU-16)	`feature/HU-16-musica-y-efectos`	Camilo	Kevin	2	https://github.com/jaider23100/FragmentosDeLuz/pull/36
14	feat(jugador): sonidos del caballero y diagramas UML	`feature/sonidos-caballero-y-uml`	Jaider	Diego	2	https://github.com/jaider23100/FragmentosDeLuz/pull/37
15	feat(enemigos): murciélago perseguidor (HU-10)	`feature/HU-10-murcielago`	Kevin	Camilo	2	https://github.com/jaider23100/FragmentosDeLuz/pull/38
16	fix(física): la roca no se movía al empujarla (DF-06)	`fix/DF-06-empuje-roca`	Kevin	Camilo	2	https://github.com/jaider23100/FragmentosDeLuz/pull/39
17	feat(ui): victoria, derrota y opciones (HU-18, HU-20)	`feature/HU-18-20-victoria-derrota-opciones`	Diego	Jaider	2	https://github.com/jaider23100/FragmentosDeLuz/pull/40
18	feat(narrativa): historia en el juego y nivel completo (HU-15)	`feature/HU-15-historia-y-nivel`	Camilo	Kevin	2	https://github.com/jaider23100/FragmentosDeLuz/pull/41
19	chore(release): prepara la versión 0.2.0	`chore/release-v0.2.0`	Diego	Jaider	2	https://github.com/jaider23100/FragmentosDeLuz/pull/43
20	Feature/hu 21 guardado	`feature/HU-21-guardado`	Diego	Kevin	3	https://github.com/jaider23100/FragmentosDeLuz/pull/44
21	Feature/hu 22 creditos	`feature/HU-22-creditos`	Camilo	Diego	3	https://github.com/jaider23100/FragmentosDeLuz/pull/45

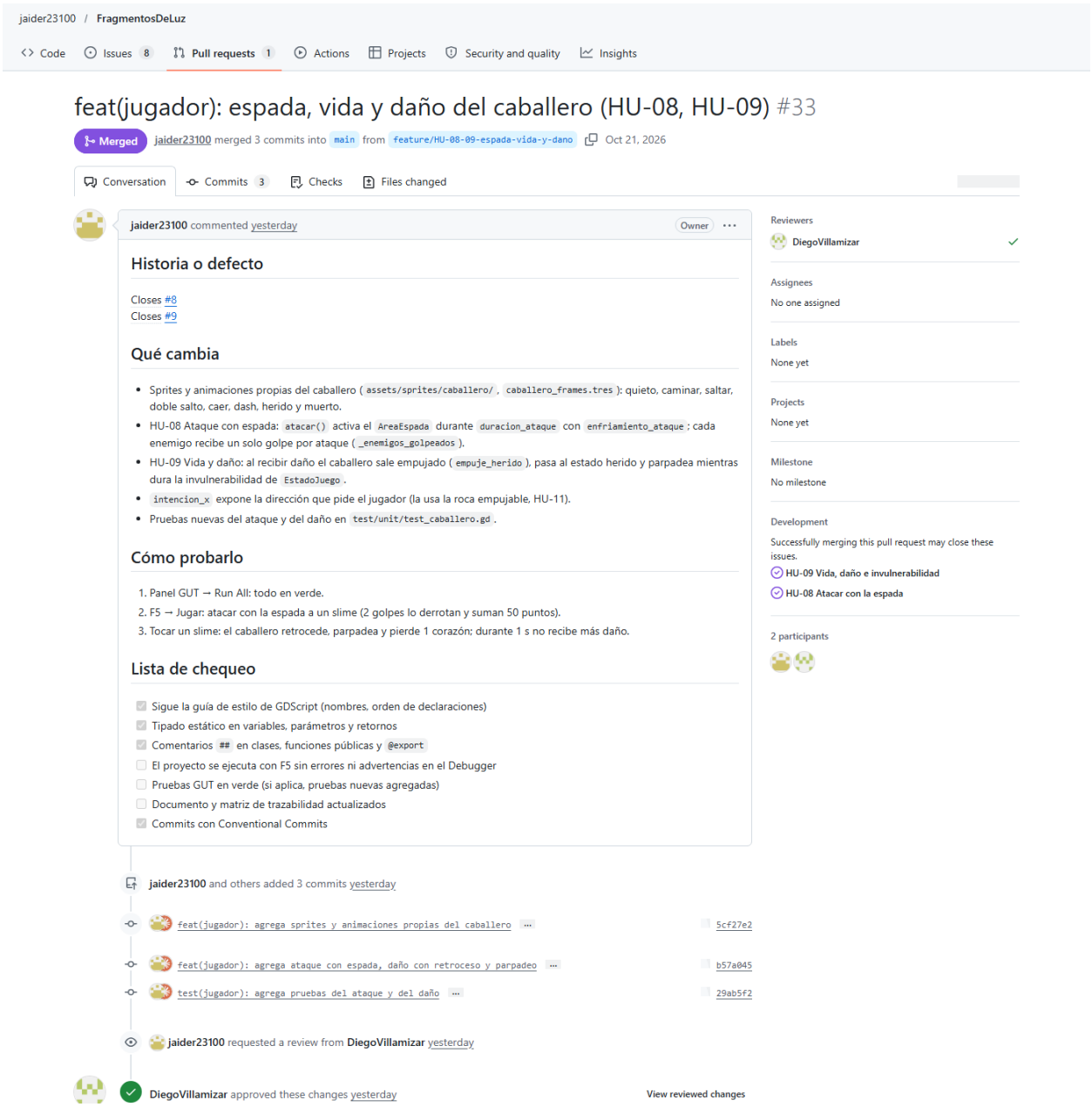
N.º	Título	Rama	Autor	Revisor	Sprint	Enlace
22	test(qa): agrega reporte GUT de la versión 1.0.0	`test/reporte-gut-v1.0.0`	Kevin	Jaider	3	https://github.com/jaider23100/FragmentosDeLuz/pull/46
23	chore(release): prepara la versión 1.0.0	`chore/release-v1.0.0`	Diego	Kevin	3	https://github.com/jaider23100/FragmentosDeLuz/pull/47
24	docs: documento final y matriz de trazabilidad	`docs/documento-final`	Jaider	Camilo	3	https://github.com/jaider23100/FragmentosDeLuz/pull/48

Nota. Elaboración propia con datos de GitHub. La solicitud #42 (versión 0.2.0) la abrió por error otro integrante y se cerró sin fusionar; la versión 0.2.0 entró por la #43. Los títulos de #28, #30, #44 y #45 no siguen la convención de 5.3.2 (se dejaron como se abrieron).

La Figura 35 muestra una solicitud revisada: la #33 (HU-08 y HU-09), con la plantilla llena, Diego como revisor y su aprobación antes de la fusión.

Figura 35

Solicitud de integración #33 aprobada por el revisor



Nota. Captura de GitHub, 22 oct. 2026.

5.3.4 Versiones publicadas y etiquetas

Las versiones siguen el versionado semántico (Preston-Werner, s. f.) con etiquetas anotadas (`git tag -a`). Cada versión publicada (GitHub, s. f.-b) tiene adjunto el ejecutable de Windows y sus notas, copiadas de `CHANGELOG.md` (Tabla 57).

Tabla 57

Versiones publicadas

Versión	Hito	Contenido	Ejecutable adjunto
`v0.1.0`	H-02	Base jugable (Sprint 1)	`FragmentosDeLuz-v0.1.0-windows.zip`
`v0.2.0`	H-03	Juego completo con los cinco elementos (Sprint 2)	`FragmentosDeLuz-v0.2.0-windows.zip`
`v1.0.0`	H-04	Versión entregada (Sprint 3)	`FragmentosDeLuz-v1.0.0-windows.zip`

Nota. Elaboración propia.

5.3.5 Archivos de Godot en el repositorio

El `.gitignore` excluye `.godot/` (que contiene `export_credentials.cfg`, con opciones de exportación confidenciales) y `builds/`, porque los ejecutables se publican en las versiones y no en el repositorio. El `.gitattributes` normaliza los finales de línea con `* text=auto eol=lf` y marca como binarios los archivos `png`, `jpg`, `wav`, `ogg`, `mp3`, `ttf` y `otf` (Godot Engine, s. f.-m).

5.3.6 Tablero

El tablero de GitHub Projects (https://github.com/users/jaider23100/projects/4) contiene las 23 historias como incidencias, con etiquetas de sprint, prioridad, responsable y puntos, y las columnas Pila del producto, Listo, En progreso, En revisión y Terminado. Cada solicitud de integración se enlaza a su historia con `Closes #n`, de modo que al fusionarla la historia pasa a Terminado.

5.4 Gestión de configuración

La Tabla 58 registra los elementos de configuración con su versión, para que cualquier persona pueda reproducir el ejecutable. Las plantillas de exportación tienen exactamente la misma versión que el editor (Godot Engine, s. f.-c).

Tabla 58

Elementos de configuración

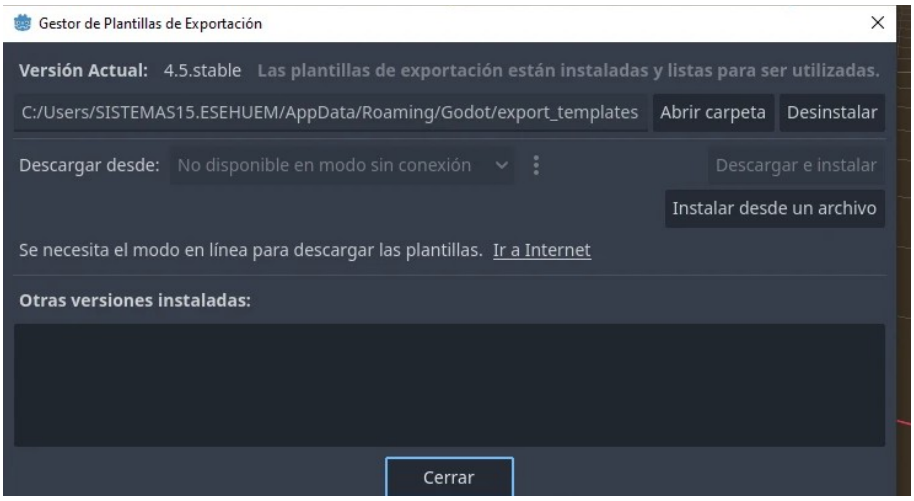
Elemento	Versión o valor	Ubicación o fuente	Licencia u observación
Editor de Godot	4.5.stable.official (876b29033)	https://godotengine.org/download/archive/4.5-stable/	MIT
Plantillas de exportación	4.5.stable (Windows x86_64)	Editor › Administrar plantillas de exportación; en `%APPDATA%\Godot\export_templates\4.5.stable\`	MIT; descargadas de la versión publicada oficial y verificadas con su suma SHA-512
Renderizador	`gl_compatibility`	`project.godot` (`rendering/renderer/rendering_method`)	ADR-003
Resolución y filtro	320 × 180, ventana de 1280 × 720, estiramiento `viewport`, filtro `Nearest`	`project.godot`	—
Versión del proyecto	1.0.0 (ejecutable 1.0.0.0)	`project.godot` (`config/version`) y `export_presets.cfg`	Versionado semántico
Configuración de exportación	«Windows Desktop», x86_64, sin `test/*`, `herramientas/*`, `docs/*` ni `addons/gut/*`; salida `builds/windows/FragmentosDeLuz.exe`	`export_presets.cfg`	Sin credenciales (están en `.godot/`, excluido del repositorio)
Complemento de pruebas	GUT 9.5.0	`addons/gut/`	MIT
Otros complementos de Godot	Ninguno	—	—
Control de versiones	Git 2.53 y GitHub	https://github.com/jaider23100/FragmentosDeLuz	Repositorio público
Herramientas de arte (no van en el ejecutable)	Python 3.14 y Pillow 12.3	`herramientas/arte/`	Licencias PSF y MIT-CMU; ver DT-06
Diagramas (no van en el ejecutable)	PlantUML con la biblioteca C4-PlantUML	`docs/diagramas/*.puml`	GPL (PlantUML) y MIT (C4-PlantUML)

Nota. Elaboración propia.

La Figura 36 confirma que las plantillas instaladas son las 4.5.stable, iguales al editor; la configuración «Windows Desktop» se ve en la Figura del manual técnico (sección 7.2.6).

Figura 36

Gestor de plantillas de exportación con 4.5.stable instaladas



Nota. Captura del editor Godot 4.5, 20 oct. 2026.

5.5 Deuda técnica

La Tabla 59 registra la deuda técnica conocida al momento de la entrega: atajos tomados, valores fijos, código que creció más de lo previsto y pruebas faltantes, cada uno con su ubicación, causa, impacto y plan de pago.

Tabla 59

Registro de deuda técnica

ID	Descripción	Ubicación (script o escena)	Causa	Impacto	Plan y prioridad
DT-01	Varios scripts buscan nodos por el nombre de su grupo escrito como texto (`"hud"`, `"jugador"`)	`cartel.gd`, `santuario.gd`, `murcielago.gd`	Solución rápida para no pasar referencias entre escenas	Si se cambia el nombre de un grupo, el error aparece en ejecución y no al compilar	Definir el nombre como constante en `Hud` y `Caballero` y usarla desde los demás scripts. Prioridad baja
DT-02	El enum `Estado` del slime declara `HERIDO`, que nunca se usa porque el slime muere de un golpe	`scripts/enemigos/slime.gd`	Se planeó un parpadeo al recibir un golpe que no hizo falta	Confunde al leer el código; el diagrama de estados del slime no lo muestra	Eliminar el valor o usarlo si el slime pasa a tener más vida. Prioridad baja
DT-03	`caballero.gd` tiene 273 líneas, cerca del límite de 300 del estándar	`scripts/jugador/caballero.gd`	Reúne movimiento, impulso, ataque, daño y animación en un solo script	Cada cambio en el jugador toca el mismo archivo y aumenta el riesgo de conflictos	Separar el ataque y el daño en nodos componentes hijos. Prioridad media
DT-04	No hay pruebas automatizadas que recorran el nivel completo ni la mayoría de las pantallas	`test/unit/`	Plazo de cinco días; se priorizaron las reglas de negocio y la persistencia	Una regresión en el flujo de pantallas solo se detecta con los casos funcionales manuales	Agregar pruebas de escena con GUT que simulen entradas. Prioridad media
DT-05	El nivel se genera con una herramienta; si se edita `nivel_bosque.tscn` a mano y luego se vuelve a ejecutar la herramienta, se pierden los cambios	`herramientas/nivel/construir_nivel.tscn`, `scenes/niveles/nivel_bosque.tscn`	Construir el nivel por código fue más rápido y repetible	Riesgo de perder ediciones manuales del nivel	Elegir una sola fuente (la herramienta o el editor) y documentarla en el manual técnico. Prioridad media
DT-06	Las imágenes y sonidos propios solo se pueden regenerar con Python y Pillow, fuera de Godot	`herramientas/arte/*.py`	Se generaron por código para tener arte propio en poco tiempo	Quien continúe el proyecto necesita instalar Python 3 y Pillow para cambiar el arte	Documentado en el manual técnico; a futuro, editar las imágenes con un editor de arte de píxeles. Prioridad baja
DT-07	Algunos valores de juego son literales o constantes en vez de `@export` (desplazamiento de 16 px en los créditos, espera de 1,2 s antes de cambiar de pantalla, atenuación de 12 dB en la pausa)	`creditos.gd`, `nivel_bosque.gd`, `gestor_audio.gd`	Se ajustaron una sola vez y no se esperaba cambiarlos	Cambiarlos exige editar el código en vez del Inspector	Convertirlos en variables `@export`. Prioridad baja
DT-08	El ejecutable no está firmado digitalmente	Exportación (`export_presets.cfg`)	Firmar exige un certificado de pago	Windows SmartScreen puede advertir al abrirlo la primera vez	Explicar en el manual de usuario cómo abrirlo; firmar si el juego se distribuye públicamente. Prioridad baja

Nota. Elaboración propia.

6. Calidad y pruebas

6.1 Plan de pruebas

El plan toma como referencia la estructura de documentación de pruebas de ISO/IEC/IEEE 29119-3:2021 (ISO, 2021).

Objetivo y alcance: verificar que la versión 1.0.0 cumple los 19 requisitos funcionales, los 8 no funcionales y las 10 reglas de negocio, y que la documentación coincide con el ejecutable. Se prueban la lógica del juego, los movimientos, los enemigos, la física, la interfaz, el audio, la persistencia y el ejecutable exportado. No se prueban el soporte de mando, otras plataformas ni la reasignación de teclas, que están fuera de alcance.

Elementos de prueba: las escenas de `scenes/` y los scripts de `scripts/` en la etiqueta v1.0.0, y el ejecutable `FragmentosDeLuz-v1.0.0-windows.zip` de la versión publicada.

La Tabla 60 resume los tipos de prueba y la Tabla 61, el entorno.

Tabla 60

Tipos de prueba

Tipo	Qué verifica	Técnica	Herramienta
Unitarias	Reglas de negocio, máquinas de estados, puntaje, vidas y persistencia	Caja blanca; valores límite	GUT 9.5.0
Integración	Escenas instanciadas que se comunican por señales (HUD, santuario, pausa)	Caja gris	GUT 9.5.0 (escenas en la prueba)
Funcionales	Cada requisito funcional sobre el ejecutable	Caja negra; partición de equivalencia y valores límite	Casos CP-01 a CP-20
Regresión	Que lo integrado no rompa lo anterior	Repetir todas las pruebas unitarias en cada solicitud de integración y en cada versión	GUT por línea de comandos
Rendimiento	RNF-01 y RNF-02	Medición repetida en dos escenarios	Depurador › Monitores y Perfilador
Usabilidad	RNF-03 y RNF-04	Pruebas de juego con observación y pensamiento en voz alta; cuestionario SUS	Protocolo de la sección 6.5
Aceptación	RNF-05 y los criterios de éxito del acta	Ejecución en un equipo sin Godot; revisión del Product Owner	Caso CP-20

Nota. Elaboración propia.

El equipo de desarrollo es también el equipo de referencia: en él se ejecutan las pruebas unitarias, los casos funcionales y las mediciones de rendimiento.

Tabla 61

Entorno de prueba

Elemento	Equipo de desarrollo y de referencia
Equipo	Dell OptiPlex AIO 7410 (todo en uno)
Procesador	Intel Core i5-13500T (14 núcleos, 20 hilos)
Memoria	8 GB (7,7 GB utilizables)
Gráficos	Intel UHD Graphics 770 (controlador 32.0.101.6881)
Almacenamiento	SSD NVMe de 256 GB y disco duro de 1 TB
Sistema operativo	Windows 10 Pro 22H2, 64 bits (10.0.19045)
Pantalla	1920 × 1080 a 60 Hz
Godot y versión del juego	Godot 4.5-stable con la integración y la etiqueta v1.0.0; para CP-20, el ejecutable v1.0.0 sin abrir Godot

Nota. Elaboración propia.

- **Criterios de entrada:** el ejecutable se exporta desde una etiqueta; el 100 % de las pruebas unitarias está en verde; el proyecto abre sin errores en el panel Depurador.
- **Criterios de salida:** el 100 % de las pruebas unitarias aprobadas; al menos el 95 % de los casos funcionales aprobados (19 de 20); ningún defecto crítico o alto abierto; todos los requisitos con al menos una prueba con resultado en la matriz.
- **Roles:** Kevin coordina los casos funcionales y el rendimiento; Diego, las pruebas unitarias; Camilo, las pruebas de juego; Jaider (Product Owner) acepta los resultados y mantiene la matriz de trazabilidad; cada integrante ejecuta los casos de su sistema.
- **Calendario:** Sprint 1, pruebas unitarias de las reglas de negocio (GUT en la HU-07); Sprint 2, pruebas de los movimientos, enemigos, física e interfaz y primeros casos funcionales sobre v0.2.0; Sprint 3, casos funcionales completos, rendimiento, pruebas de juego y reporte final sobre v1.0.0.

6.2 Pruebas unitarias automatizadas

El proyecto tiene 42 pruebas unitarias con GUT 9.5.0 (Bitwes, s. f.) en nueve archivos de `test/unit/`. Todas extienden `GutTest` y prueban lógica del juego, no funciones triviales. Se ejecutan desde el panel GUT del editor o por línea de comandos:

Ejecución por línea de comandos (Windows, desde la carpeta del proyecto)

```
"Godot_v4.5-stable_win64_console.exe" --headless --path . -s addons/gut/gut_cmdln.gd -d
    -gdir=res://test/unit -gexit -gjunit_xml_file=res://docs/reporte_gut.xml
```

La Tabla 62 lista cada prueba con el requisito o regla que verifica.

Tabla 62

Pruebas unitarias

ID	Archivo	Prueba	Qué verifica	Requisito o regla
PU-01	test_estado_juego.gd	test_inicia_con_3_corazones_0_puntos_y_0_fragmentos	Estado inicial de la partida	RN-01
PU-02	test_estado_juego.gd	test_recibir_dano_resta_un_corazon	Cada daño resta exactamente un corazón	RN-01, RF-05
PU-03	test_estado_juego.gd	test_corazones_nunca_bajan_de_cero	Los corazones no bajan de 0	RN-01
PU-04	test_estado_juego.gd	test_invulnerabilidad_bloquea_dano_durante_un_segundo	No hay daño durante 1,0 s tras un golpe	RN-02, RF-05
PU-05	test_estado_juego.gd	test_invulnerabilidad_termina_despues_de_un_segundo	El daño vuelve a contar después de 1,0 s	RN-02
PU-06	test_estado_juego.gd	test_caida_ignora_invulnerabilidad_y_usa_checkpoint	La caída resta un corazón y reaparece en el punto de control	RN-03, RF-06
PU-07	test_estado_juego.gd	test_sin_checkpoint_reaparece_en_el_inicio	Sin punto de control se reaparece en el inicio	RN-03, RF-06
PU-08	test_estado_juego.gd	test_puntaje_de_fragmentos_monedas_y_enemigos	Valores de puntaje (100, 10 y 50)	RN-04, RF-11, RF-12
PU-09	test_estado_juego.gd	test_mensaje_de_fragmentos_faltantes_en_singular_y_plural	«Falta 1 fragmento» y «Faltan X fragmentos»	RN-06, RF-13
PU-10	test_estado_juego.gd	test_solo_gana_con_los_5_fragmentos	Solo se gana con 5/5	RN-06, RF-13
PU-11	test_estado_juego.gd	test_derrota_al_llegar_a_cero_corazones	Derrota con 0 corazones	RN-07, RF-13
PU-12	test_estado_juego.gd	test_puntaje_final_suma_bono_por_corazones	Puntaje final con bono de 100 por corazón	RN-08
PU-13	test_estado_juego.gd	test_reiniciar_restaura_el_estado_inicial	Reiniciar restaura el estado y olvida los puntos de control	RN-10, RF-15
PU-14	test_caballero.gd	test_doble_salto_solo_una_vez_en_el_aire	Un solo doble salto por salto	RF-02
PU-15	test_caballero.gd	test_dash_en_el_aire_solo_una_vez_por_salto	Un solo impulso en el aire	RF-03
PU-16	test_caballero.gd	test_dash_bloqueado_durante_el_enfriamiento	El impulso respeta el enfriamiento de 0,5 s	RF-03
PU-17	test_caballero.gd	test_dash_avanza_sin_gravedad_hacia_donde_mira	El impulso es horizontal, sin gravedad, hacia donde mira	RF-03
PU-18	test_caballero.gd	test_ataque_bloqueado_durante_el_enfriamiento	El ataque respeta el enfriamiento de 0,35 s	RF-04
PU-19	test_caballero.gd	test_dano_pasa_a_herido_con_retroceso	El daño pasa a HERIDO con retroceso	RF-05
PU-20	test_caballero.gd	test_dano_durante_invulnerabilidad_no_resta	El caballero no pierde corazones mientras es invulnerable	RF-05, RN-02
PU-21	test_caballero.gd	test_dano_mortal_pasa_a_muerto	Con el último corazón pasa a MUERTO	RF-05, RN-07
PU-22	test_enemigos_y_fisica.gd	test_slime_muere_con_un_golpe_y_suma_50_puntos	El slime muere de un golpe y suma 50	RF-07, RN-04, RN-05
PU-23	test_enemigos_y_fisica.gd	test_enemigo_con_dos_de_vida_resiste_el_primer_golpe	Un enemigo con 2 de vida resiste un golpe	RN-05
PU-24	test_enemigos_y_fisica.gd	test_viento_aplica_y_quita_su_fuerza_al_caballero	El viento aplica y quita su fuerza	RF-10
PU-25	test_murcielago.gd	test_despierta_y_persigue_cuando_el_caballero_se_acerca	Despierta a menos de 80 px	RF-08
PU-26	test_murcielago.gd	test_regresa_cuando_el_caballero_se_aleja_y_vuelve_a_dormir	Regresa a más de 140 px y vuelve a dormir	RF-08
PU-27	test_murcielago.gd	test_muere_con_dos_golpes	El murciélago muere con 2 golpes	RF-08, RN-05
PU-28	test_historia_y_santuario.gd	test_santuario_no_se_restaura_con_fragmentos_faltantes	El santuario no se restaura con menos de 5	RN-06, RF-13
PU-29	test_historia_y_santuario.gd	test_santuario_completa_el_nivel_con_los_cinco_fragmentos	Con 5/5 el santuario completa el nivel	RN-06, RF-13
PU-30	test_historia_y_santuario.gd	test_la_introduccion_tiene_cuatro_pantallas_y_avanza	La introducción tiene 4 pantallas y avanza	RF-16
PU-31	test_interfaz.gd	test_hud_se_actualiza_al_recoger_un_fragmento	El HUD cambia al recoger un fragmento	RF-14
PU-32	test_interfaz.gd	test_hud_muestra_corazon_vacio_al_recibir_dano	El HUD muestra el corazón vacío	RF-14
PU-33	test_interfaz.gd	test_pausa_detiene_y_reanuda_el_juego	La pausa detiene y reanuda el juego	RF-15
PU-34	test_pantallas.gd	test_victoria_muestra_el_puntaje_final_con_bono	La victoria muestra el bono y el total	RN-08, RF-13
PU-35	test_pantallas.gd	test_opciones_silencia_la_musica_sin_afectar_los_efectos	Música al 0 % silencia solo la música	RF-18
PU-36	test_creditos.gd	test_creditos_muestran_equipo_licencias_y_aviso_de_godot	Créditos con equipo, licencias y aviso de Godot	RF-18

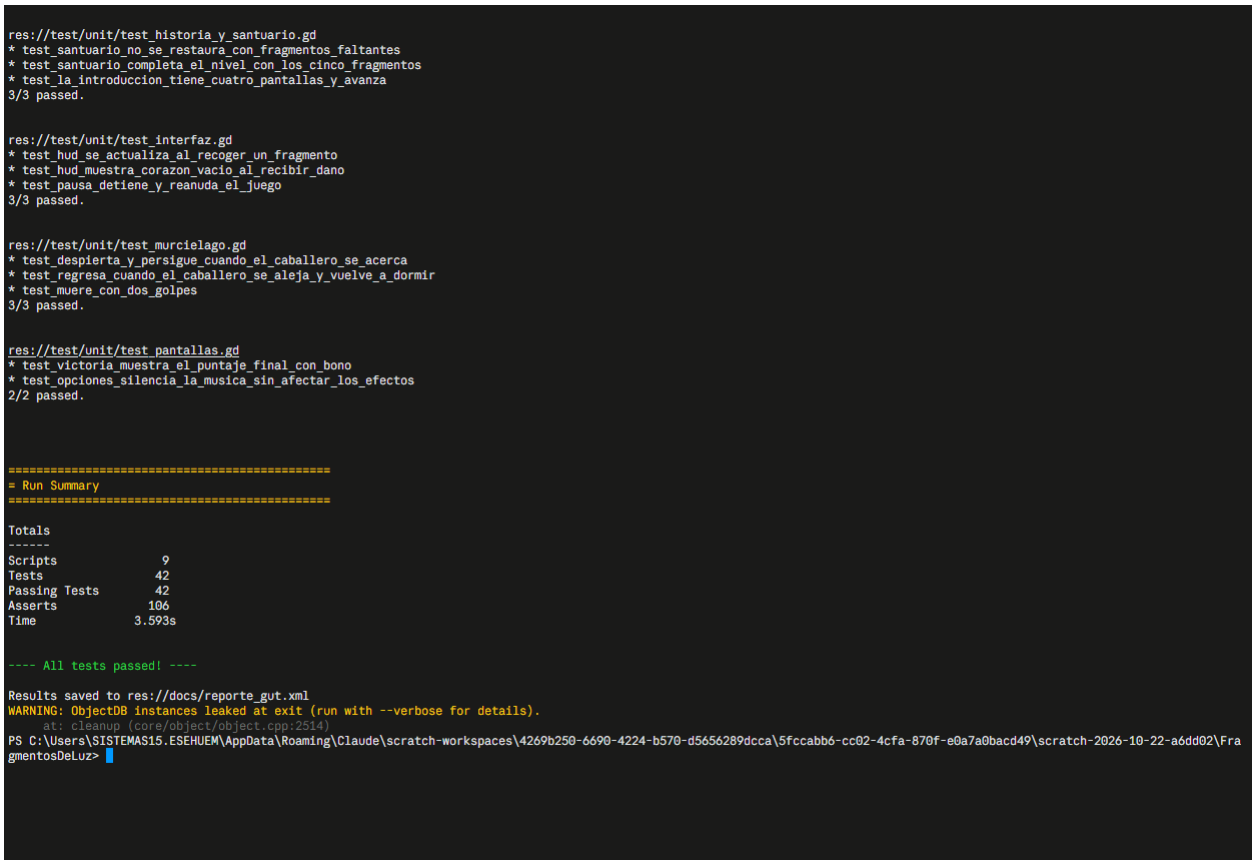
ID	Archivo	Prueba	Qué verifica	Requisito o regla
PU-37	`test_creditos.gd`	`test_el_texto_sube_solo`	Los créditos se desplazan solos	RF-18
PU-38	`test_gestor_guardado.gd`	`test_sin_archivo_usa_valores_por_defecto`	Sin archivo: valores por defecto	RF-19, RNF-08
PU-39	`test_gestor_guardado.gd`	`test_archivo_danado_usa_valores_por_defecto_sin_fallar`	Archivo dañado: valores por defecto, sin fallar	RNF-08
PU-40	`test_gestor_guardado.gd`	`test_volumen_fuera_de_rango_se_ajusta`	Valores fuera de rango se ajustan	RNF-08
PU-41	`test_gestor_guardado.gd`	`test_guardar_y_recargar_conserva_los_valores`	Guardar y recargar conserva los valores	RF-19
PU-42	`test_gestor_guardado.gd`	`test_mejor_puntaje_solo_cambia_si_es_estrictamente_mayor`	El récord solo cambia si es estrictamente mayor	RN-09, RF-19

Nota. Elaboración propia.

Resultado: la última ejecución sobre la integración de la versión 1.0.0 (20 de octubre de 2026) dio 42 pruebas ejecutadas, 42 aprobadas, 0 fallidas y 106 aserciones, en nueve archivos; el reporte JUnit XML está en `docs/reporte_gut.xml`. El reporte de la versión final lo subió Kevin Yaruro con la solicitud #46 (22 de octubre, 5:18 p. m.). La Figura 37 muestra una ejecución por línea de comandos en el equipo de referencia sobre `main` con esa solicitud ya fusionada (commit `5c187aa`, 22 de octubre, 5:24 p. m.): 9 archivos, 42 pruebas, 42 aprobadas, 106 aserciones, en 3,6 s.

Figura 37

Ejecución de las 42 pruebas GUT por línea de comandos



Nota. Captura de la consola en el equipo de referencia, 22 oct. 2026.

6.3 Casos de prueba funcionales

Se especifican 20 casos de prueba funcionales sobre el ejecutable, trazados a requisitos e historias; todo requisito funcional tiene al menos un caso. La Tabla 63 los resume y el Anexo B presenta cada uno con la plantilla completa. Un caso fallido genera un defecto en la sección 6.6.

Tabla 63

Resumen de los casos de prueba funcionales

ID	Nombre	Requisito	HU	Estado	Versión	Evidencia
CP-01	Moverse y saltar	RF-01	HU-01	Aprobado	v1.0.0	E-01
CP-02	Doble salto	RF-02	HU-02	Aprobado	v1.0.0	E-02
CP-03	Impulso con enfriamiento	RF-03	HU-03	Aprobado	v1.0.0	E-03
CP-04	Atacar con la espada	RF-04	HU-08	Aprobado	v1.0.0	E-04
CP-05	Daño, invulnerabilidad y derrota	RF-05, RF-13	HU-09, HU-18	Aprobado	v1.0.0	E-05
CP-06	Caída al vacío y punto de control	RF-06	HU-05	Aprobado	v1.0.0	E-06
CP-07	Slime patrullero	RF-07	HU-04	Aprobado	v1.0.0	E-07
CP-08	Murciélago perseguidor	RF-08	HU-10	Aprobado	v1.0.0	E-08
CP-09	Roca empujable	RF-09	HU-11	Aprobado	v1.0.0	E-09
CP-10	Corriente de viento	RF-10	HU-12	Aprobado	v1.0.0	E-10
CP-11	Recoger fragmentos	RF-11	HU-13	Aprobado	v1.0.0	E-11
CP-12	Recoger monedas	RF-12	HU-14	Aprobado	v1.0.0	E-12
CP-13	Santuario y victoria	RF-13	HU-18	Aprobado	v1.0.0	E-13
CP-14	HUD	RF-14	HU-17	Aprobado	v1.0.0	E-14
CP-15	Pausa y reinicio	RF-15	HU-19	Aprobado	v1.0.0	E-15
CP-16	Historia en el juego	RF-16	HU-15	Aprobado	v1.0.0	E-16
CP-17	Música y efectos	RF-17	HU-16	Aprobado	v1.0.0	Al oído
CP-18	Menú, opciones y créditos	RF-18	HU-06, HU-20, HU-22	Aprobado	v1.0.0	E-18
CP-19	Persistencia del récord y del volumen	RF-19	HU-21	Aprobado	v1.0.0	E-19
CP-20	Ejecutable sin Godot	RNF-05	HU-23	Aprobado	v1.0.0	E-20

Nota. Elaboración propia.

6.4 Pruebas de rendimiento

Los requisitos de eficiencia se verifican en el equipo de referencia con el panel Depurador de Godot: la pestaña Monitores para los FPS, la memoria, los objetos y nodos y los tiempos de proceso y de física, y las pestañas Perfilador y Perfilador visual para las funciones y etapas de dibujo más costosas (Godot Engine, s. f.-b; Godot Engine, s. f.-k). Cada medición dura 60 s y se repite tres veces.

Cómo se midió (21 de octubre de 2026, versión 1.0.0, equipo de referencia): para tener cifras exactas en lugar de leerlas en la gráfica, un script temporal de la herramienta del equipo (no incluido en el ejecutable) lee cada segundo los mismos monitores del panel (`Performance.TIME_FPS` y `TIME_PROCESS`) con el juego dibujándose en pantalla. Coloca al caballero en el escenario, espera 3 s y registra 60 muestras; durante la medición el caballero es invulnerable para que el murciélago no termine la partida. El tiempo de carga se mide con `Time.get_ticks_usec()` desde el cambio de escena pedido en la introducción hasta el primer

cuadro del nivel. La pantalla es de 60 Hz y el juego usa sincronización vertical, así que 60 FPS es el tope que se puede medir.

Escenario 1: zona 3, con el murciélago persiguiendo al caballero y la corriente de viento con sus partículas. **Escenario 2:** zona 5, el tramo con más objetos a la vista (púas, monedas, murciélago y santuario). **Escenario 3 (RNF-02):** paso de la introducción al nivel.

La Tabla 64 registra los resultados.

Tabla 64

Resultados de rendimiento

RNF	Escenario	Métrica (monitor)	Umbral	Medición 1 / 2 / 3	Promedio	Cumple	Evidencia
RNF-01	1. Zona 3	FPS promedio	≥ 60	60,0 / 59,8 / 60,0	59,9	Con observación	C-21
RNF-01	1. Zona 3	FPS mínimo	≥ 30	59 / 51 / 60	51 (el menor)	Sí	C-21
RNF-01	2. Zona 5	FPS promedio	≥ 60	60,0 / 60,0 / 60,0	60,0	Sí	C-22
RNF-01	2. Zona 5	FPS mínimo	≥ 30	58 / 60 / 60	58 (el menor)	Sí	C-22
RNF-02	3. Introducción a nivel	Tiempo de carga (s)	≤ 2	0,134 / 0,050 / 0,066	0,083	Sí	Registro

Nota. Elaboración propia. Registro completo en `anexos/rendimiento_21oct.log`.

Observación sobre la zona 3: el promedio fue 59,9 FPS, 0,1 por debajo del umbral, por una caída aislada a 51 FPS en la segunda medición; las otras dos dieron 60,0. Como la pantalla limita el juego a 60 FPS, cualquier caída momentánea deja el promedio por debajo de 60, de modo que el umbral «≥ 60» solo se cumple si no hay ninguna. Se registra como cumplido con observación, no como defecto, porque el mínimo (51) queda muy por encima de 30 y la caída no se notó al jugar; la lección se recoge en la sección 7.5 (LA-06).

Figura 38

Panel Depurador › Monitores con FPS, tiempo de proceso y de física



Nota. Captura del editor Godot 4.5, versión 1.0.0, 21 oct. 2026.

Figura 39

Panel Monitores durante el escenario 2 (zona 5)



Nota. Captura del editor Godot 4.5, versión 1.0.0, 21 oct. 2026.

Análisis del Perfilador (20 s de juego en el nivel, Figura 40): el tiempo de cuadro se mantiene estable en 16,54 ms, dentro del presupuesto de 16,67 ms que corresponde a 60 FPS, y la gráfica no muestra picos sostenidos; el tiempo de física es de solo 0,66 ms por cuadro. Con ese margen no se requirieron optimizaciones para cumplir RNF-01.

Figura 40

Perfilador con el tiempo de cuadro, de proceso y de física



Nota. Captura del editor Godot 4.5, versión 1.0.0, 21 oct. 2026.

6.5 Usabilidad y pruebas de juego

Las sesiones se planearon con cinco personas reales externas al grupo, con el siguiente protocolo:

- **Objetivo:** verificar que el juego se aprende con los carteles, sin ayuda (RNF-03), y que la experiencia es satisfactoria (RNF-04).
- **Perfil de los participantes:** personas del público objetivo (10 años en adelante) que no participaron en el desarrollo.
- **Consentimiento:** cada persona aceptó por escrito o de palabra y eligió si su nombre aparece o se usa «Usuario N».

- **Tareas:** jugar hasta 15 minutos con un único objetivo dicho por el observador («recupera los 5 fragmentos»), sin explicar los controles.
- **Guion del observador:** no ayudar salvo que la persona lleve más de 2 minutos atascada; anotar la zona a los 2 y 5 minutos, las caídas, las ayudas y lo que la persona dice en voz alta.
- **Métricas:** éxito en las zonas 1 y 2 sin ayuda, tiempo hasta terminar, corazones perdidos o caídas, ayudas recibidas y puntuación SUS (Brooke, 1996), calculada como (suma de [respuesta − 1] en las frases impares + suma de [5 − respuesta] en las pares) × 2,5.

Ejecución: se planearon cinco sesiones y se realizaron 4 el 21 de octubre de 2026, con el ejecutable v1.0.0 en el equipo de referencia (zip extraído en una carpeta aparte); la quinta persona no pudo asistir (riesgo R-05). Ninguna persona autorizó su nombre y todas aceptaron de palabra. Jaider Villamizar observó y dictó los datos durante cada partida; se registraron en vivo con la hora del sistema, y el inicio de cada partida y el puntaje de la victoria se comprobaron en el equipo (proceso del juego y archivo de guardado). Las sesiones 1 y 2 usaron el ejecutable antes de corregir la roca (DF-06) y las 3 y 4, después.

La Tabla 65 resume las sesiones; los datos completos están en el Anexo C.

Tabla 65

Resultados de las sesiones de pruebas de juego

Participante	Fecha	Zonas 1 y 2 sin ayuda	¿Terminó? (tiempo)	Caídas o daños	Ayudas	SUS
Usuario 1	21 oct.	Sí / Sí (zona 2 en el tercer intento)	No; paró a los ~8 min en la zona 3	3 derrotas	Ninguna	77,5
Usuario 2	21 oct.	Sí / Sí (impulso con Shift)	No; paró a los ~3 min en la zona 4	1 derrota (murciélago, zona 4)	Ninguna	72,5
Usuario 3	21 oct.	Sí / Sí	No; paró a los ~2 min en la zona 4	1 derrota (murciélago, zona 4)	Ninguna	65
Usuario 4	21 oct.	Sí / Sí	Sí, en ~5 min; 890 puntos + 100 de bono = 990 («¡Nuevo mejor puntaje!», guardado a la 1:04 p. m.)	No se anotaron	Ninguna	60
Usuario 5	—	Sesión no realizada	—	—	—	—
Resultado	—	4 de 4 (umbral ≥ 4 de 5): cumple	1 de 4 terminó	—	Ninguna	Promedio 68,75 (umbral ≥ 68): cumple

Nota. Elaboración propia.

Lectura de los resultados: las cuatro personas superaron las zonas 1 y 2 sin ayuda, lo que cumple RNF-03. El SUS promedio (68,75) supera por poco el umbral de 68 (RNF-04), con valores de 60 a 77,5; con cuatro participantes el resultado es indicativo, no concluyente. Solo una persona terminó el nivel: las otras tres dejaron de jugar tras una derrota, dos de ellas contra el murciélago de la zona 4 (PT-03).

La Figura 41 es la pantalla de victoria del Usuario 4; el mismo puntaje quedó escrito en el archivo de guardado a la 1:04 p. m.

Figura 41

Pantalla de victoria del Usuario 4 (sesión 4)



Nota. Captura del ejecutable v1.0.0, 21 oct. 2026.

La Tabla 66 registra los hallazgos, su severidad y el cambio realizado.

Tabla 66

Hallazgos de las pruebas de juego

ID	Hallazgo (observación o comentario)	Participantes que lo reportan	Severidad	Cambio realizado	Confirmación o solicitud
PT-01	Al principio no se entendió el impulso: el observador creyó que no funcionaba; la persona cruzó el barranco en el tercer intento. El cartel solo dice «embestida de escudo (Shift)»	Usuario 1	Media	Sin cambio en v1.0.0; propuesta: nombrar la acción «impulso» en el cartel y mostrar la tecla al acercarse	—
PT-02	La roca de la zona 4 no se movía al empujarla	Usuario 2 (confirmado por el observador)	Alta	`fuerza_empuje` de 900 a 3000 (DF-06)	El Usuario 3 empujó la roca con la corrección; `fix/DF-06-empuje-roca`
PT-03	Dos personas perdieron contra el murciélago de la zona 4 y no quisieron reintentar; una cambiaría «los enemigos»	Usuarios 2 y 3	Media	Sin cambio en v1.0.0; se registra como trabajo futuro (ajustar daño o velocidad del murciélago)	—
PT-04	El barranco de espinas fue lo más difícil	Usuarios 1 y 4 (el Usuario 2 también lo mencionó)	Baja	Sin cambio: es el desafío previsto de la zona 2 y las cuatro personas lo superaron sin ayuda	—
PT-05	Pedidos de contenido: un jefe final, otros diseños y otros enemigos	Usuarios 1, 3 y 4	Baja	Fuera del alcance de la versión 1.0.0; se registra como trabajo futuro (7.5.1)	—

Nota. Elaboración propia.

6.6 Registro de defectos

Todo defecto encontrado en pruebas, pruebas de juego o revisión se registra, también los ya corregidos. La severidad mide el efecto en el juego (crítica: cierre o bloqueo; alta: impide una funcionalidad; media: funciona con un error visible; baja: cosmética). Los estados son abierto, en progreso, resuelto, verificado y cerrado; un defecto se cierra solo después de verificar la corrección en un ejecutable o en las pruebas. La Tabla 67 consolida los defectos registrados hasta ahora; se agregan los que aparezcan en los casos funcionales y en las pruebas de juego.

Tabla 67

Registro de defectos

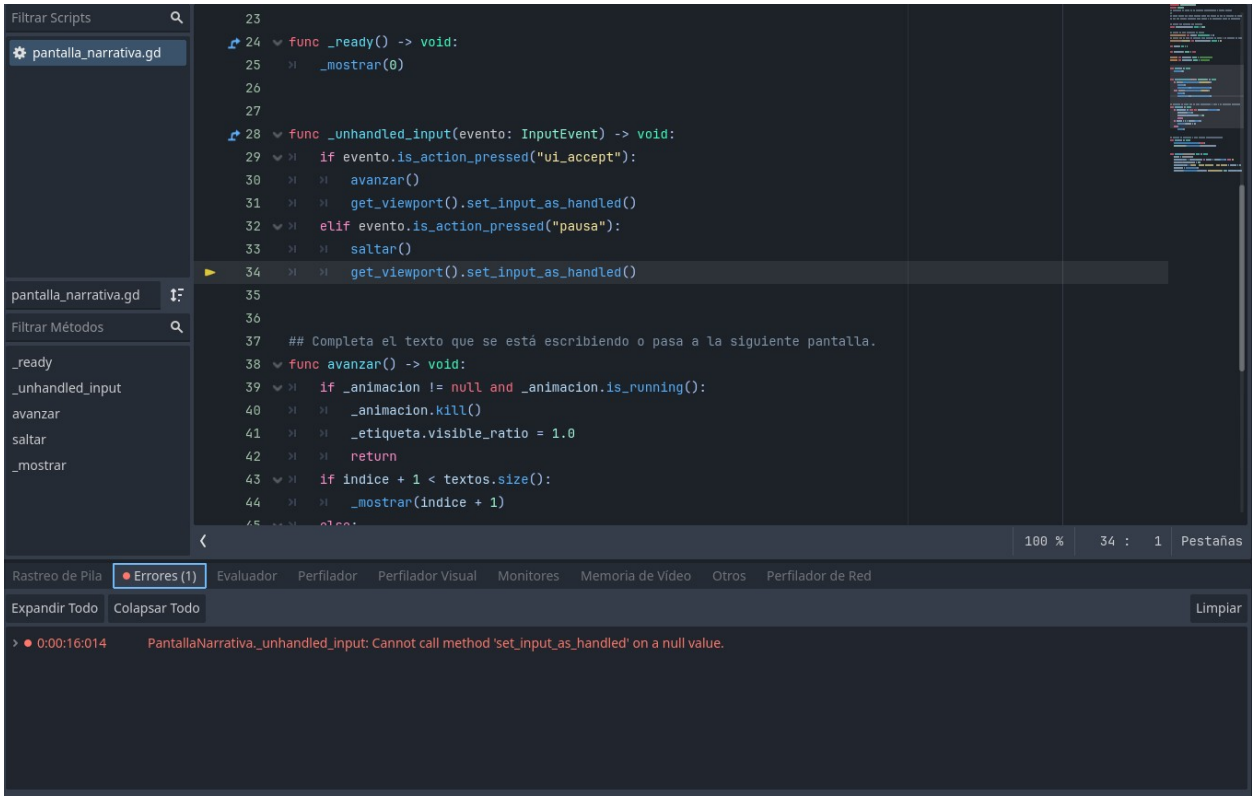
ID	Título	Pasos para reproducir	Esperado / obtenido	Severidad	Estado	Versión	Evidencia y corrección
DF-01	Aviso «Versión de archivo inválida» al exportar	Abrir Proyecto › Exportar… con `application/file_version` = «0.2.0»	Esperado: configuración válida / obtenido: aviso en el diálogo de exportación; el ejecutable sin metadatos de versión	Baja	Cerrado	v0.1.0 y v0.2.0 (antes de publicar)	Evidencia `DF-01_version_invalida.png`; corregido con versiones de cuatro números (0.2.0.0) en `export_presets.cfg` (solicitudes de las versiones publicadas)
DF-02	Una prueba de la pantalla de victoria escribiría el récord en el guardado real del jugador	Ejecutar `test_pantallas.gd` después de agregar el guardado del récord (HU-21)	Esperado: las pruebas no tocan `user://config.cfg` / obtenido: la victoria llamaba a `registrar_puntaje()` sobre el archivo real	Media	Cerrado	Rama de la HU-21 (antes de integrar)	Detectado en revisión; `test_pantallas.gd` usa un archivo de prueba (solicitud de la HU-21); 42/42 pruebas aprobadas
DF-03	La prueba del archivo dañado fallaba por el error esperado del motor	Ejecutar `test_gestor_guardado.gd`	Esperado: la prueba pasa / obtenido: GUT la marcaba como fallida por el error «ConfigFile parse error»	Baja	Cerrado	Rama de la HU-21 (antes de integrar)	Se declara el error como esperado con `assert_engine_error()` (solicitud de la HU-21)
DF-04	El diagrama de secuencia DS-04 indicaba que `NivelBosque` guarda el récord, pero lo hace `Victoria`	Comparar `secuencias.puml` con `victoria.gd` y `nivel_bosque.gd`	Esperado: el diagrama coincide con el código / obtenido: llamada inexistente en `NivelBosque`	Media	Cerrado	Documentación v1.0.0	Detectado en revisión de trazabilidad; diagrama corregido (solicitud `feature/sonidos-caballero-y-uml`)
DF-05	Error «Cannot call method 'set_input_as_handled' on a null value» al saltar o terminar la introducción	Ejecutar con F5 › Jugar › en la introducción presionar Esc (o Enter en el último texto)	Esperado: pasa al nivel sin errores / obtenido: pasa al nivel, pero el panel Depurador › Errores registra el error en `PantallaNarrativa._unhandled_input()`, porque la escena ya cambió y el nodo no tiene viewport	Baja	Cerrado	Integración v1.0.0 (21 oct., antes de integrar la HU-15)	Evidencia `DF-05_error_pantalla_narrativa.png`; se marca la entrada como atendida antes de cambiar de escena (solicitud de la HU-15); verificado con la intro en v0.2.0 y v1.0.0 y 42/42 pruebas aprobadas
DF-06	La roca de la zona 4 no se mueve al empujarla	Ejecutable v1.0.0 › zona 4 › caminar contra la roca hacia cualquier lado	Esperado: la roca avanza (CP-09) / obtenido: la roca no se mueve; `fuerza_empuje` = 900 es menor que la fricción con el suelo (0,8 × 3 × 980 ≈ 2352)	Alta	Cerrado	v1.0.0 (encontrado el 21 oct. en la sesión 2 de pruebas de juego)	Reproducido en el motor con un script de prueba (0 px de desplazamiento en ambos sentidos); `fuerza_empuje` = 3000 (la roca avanza unos 44 px/s); 42/42 pruebas aprobadas; solicitud `fix/DF-06-empuje-roca`; verificado en la sesión 3 y al repetir CP-09 (la primera ejecución de CP-09 se había marcado aprobada por error)

Nota. Elaboración propia.

La Figura 42 es la evidencia del defecto DF-05: el error queda registrado en el panel Depurador › Errores al saltar la introducción.

Figura 42

Evidencia del defecto DF-05



Nota. Captura del editor Godot 4.5, integración de la versión 1.0.0, antes de la corrección.

6.7 Matriz de trazabilidad

La Tabla 68 relaciona cada requisito con su origen, su diseño, su implementación (escena, script y función, para abrirlos en la sustentación) y su verificación. Cumple que todo requisito tiene al menos una prueba, que toda prueba remite a un requisito y que todo script y escena principal aparece en al menos una fila.

Tabla 68

Matriz de trazabilidad

Requisito	HU / CU	Diseño (diagrama, patrón, ADR)	Escena (.tscn)	Script y función (.gd)	Prueba unitaria	Caso funcional	Resultado y versión
RF-01	HU-01 / CU-02	DE-01; máquina de estados	`caballero.tscn`	`caballero.gd` → `_procesar_movimiento()`, `saltar()`	—	CP-01	CP aprobados (v1.0.0)
RF-02	HU-02 / CU-02	DE-01, DS-01; máquina de estados	`caballero.tscn`	`caballero.gd` → `doble_salto()`	PU-14	CP-02	PU aprobadas (42/42); CP aprobados (v1.0.0)
RF-03	HU-03 / CU-02	DE-01, DS-01; máquina de estados	`caballero.tscn`	`caballero.gd` → `dash()`, `_on_timer_dash_timeout()`	PU-15, PU-16, PU-17	CP-03	PU aprobadas (42/42); CP aprobados (v1.0.0)
RF-04	HU-08 / CU-03	DE-01; capas 3 y 6	`caballero.tscn`, `slime.tscn`, `murcielago.tscn`	`caballero.gd` → `atacar()`; `enemigo.gd` → `recibir_golpe()`	PU-18	CP-04	PU aprobadas (42/42); CP aprobados (v1.0.0)
RF-05	HU-09 / CU-03	DS-02; singleton y observador	`caballero.tscn`, `hud.tscn`	`estado_juego.gd` → `recibir_dano()`; `caballero.gd` → `recibir_dano()`	PU-02, PU-04, PU-19, PU-20, PU-21	CP-05	PU aprobadas (42/42); CP aprobados (v1.0.0)
RF-06	HU-05	RN-03	`checkpoint.tscn`, `nivel_bosque.tscn`	`checkpoint.gd` → `activar()`; `nivel_bosque.gd` → `_comprobar_caida()`	PU-06, PU-07	CP-06	PU aprobadas (42/42); CP aprobados (v1.0.0)
RF-07	HU-04 / CU-03	DE del slime; clase base `Enemigo`	`slime.tscn`	`slime.gd` → `_patrullar()`, `_debe_girar()`	PU-22	CP-07	PU aprobadas (42/42); CP aprobados (v1.0.0)
RF-08	HU-10 / CU-03	DE-02; máquina de estados	`murcielago.tscn`	`murcielago.gd` → `evaluar_estado()`	PU-25, PU-26, PU-27	CP-08	PU aprobadas (42/42); CP aprobados (v1.0.0)
RF-09	HU-11	Capa 7; diagrama de clases	`roca.tscn`	`roca.gd` → `_physics_process()`, `_lo_esta_empujando()`	—	CP-09	CP aprobados (v1.0.0)
RF-10	HU-12	Capa 8; diagrama de clases	`viento.tscn`	`viento.gd` → `_on_body_entered()`; `caballero.gd` → `aplicar_viento()`	PU-24	CP-10	PU aprobadas (42/42); CP aprobados (v1.0.0)
RF-11	HU-13 / CU-04	DS-03; observador	`fragmento.tscn`	`fragmento.gd` → `_al_recoger()`; `estado_juego.gd` → `recoger_fragmento()`	PU-08	CP-11	PU aprobadas (42/42); CP aprobados (v1.0.0)
RF-12	HU-14	Clase base `Coleccionable`	`moneda.tscn`	`moneda.gd` → `_al_recoger()`; `estado_juego.gd` → `registrar_moneda()`	PU-08	CP-12	PU aprobadas (42/42); CP aprobados (v1.0.0)
RF-13	HU-18 / CU-04	DS-02, DS-04, DE-03	`santuario.tscn`, `victoria.tscn`, `derrota.tscn`	`santuario.gd` → `intentar_restaurar()`; `estado_juego.gd` → `completar_nivel()`	PU-09, PU-10, PU-11, PU-28, PU-29, PU-34	CP-05, CP-13	PU aprobadas (42/42); CP aprobados (v1.0.0)
RF-14	HU-17	Observador (señales)	`hud.tscn`	`hud.gd` → `_on_corazones_cambiados()`, `_on_fragmentos_cambiados()`	PU-31, PU-32	CP-14	PU aprobadas (42/42); CP aprobados (v1.0.0)
RF-15	HU-19 / CU-06	DE-03	`menu_pausa.tscn`	`menu_pausa.gd` → `pausar()`, `reanudar()`; `estado_juego.gd` → `reiniciar()`	PU-13, PU-33	CP-15	PU aprobadas (42/42); CP aprobados (v1.0.0)
RF-16	HU-15 / CU-01	Flujo de pantallas	`intro.tscn`, `final.tscn`, `cartel.tscn`	`pantalla_narrativa.gd` → `avanzar()`; `cartel.gd` → `mostrar()`	PU-30	CP-16	PU aprobadas (42/42); CP aprobados (v1.0.0)
RF-17	HU-16	C4 componentes	Todas (autoload)	`gestor_audio.gd` → `reproducir_musica()`, `reproducir_efecto()`	—	CP-17	CP aprobados (v1.0.0)
RF-18	HU-06, HU-20, HU-22 / CU-05, CU-07	Flujo de pantallas; composición	`menu_principal.tscn`, `opciones.tscn`, `creditos.tscn`	`opciones.gd` → `_on_slider_musica_value_changed()`; `creditos.gd`	PU-35, PU-36, PU-37	CP-18	PU aprobadas (42/42); CP aprobados (v1.0.0)
RF-19	HU-21 / CU-04, CU-05	ADR-002; DS-04	`victoria.tscn`, `opciones.tscn`	`gestor_guardado.gd` → `cargar()`, `guardar()`, `registrar_puntaje()`	PU-38, PU-41, PU-42	CP-19	PU aprobadas (42/42); CP aprobados (v1.0.0)

Requisito	HU / CU	Diseño (diagrama, patrón, ADR)	Escena (.tscn)	Script y función (.gd)	Prueba unitaria	Caso funcional	Resultado y versión
RNF-01	—	ADR-003	`nivel_bosque.tscn`	Escena completa (rendimiento)	—	Rendimiento, escenarios 1 y 2	Cumple con observación: zona 3 promedio 59,9 FPS (mín. 51); zona 5 60,0 FPS (mín. 58) (v1.0.0, 21 oct.)
RNF-02	CU-01	Flujo de pantallas	`intro.tscn`, `nivel_bosque.tscn`	`gestor_escenas.gd` → `cambiar_escena()`	—	Rendimiento, escenario 3	Aprobado: carga promedio 0,083 s (máx. 0,134 s) (v1.0.0, 21 oct.)
RNF-03	HU-15	Carteles (GDD 2.2)	`cartel.tscn`	`cartel.gd`	—	Pruebas de juego (6.5)	Aprobado: 4 de 4 personas superaron las zonas 1 y 2 sin ayuda (sesión 5 no realizada; 21 oct.)
RNF-04	—	GDD 2.6	Todas	—	—	Cuestionario SUS (6.5)	Aprobado: SUS promedio 68,75 con 4 personas (60 a 77,5); indicativo (21 oct.)
RNF-05	HU-23	C4 contenedores	—	`export_presets.cfg`	—	CP-20	CP aprobados (v1.0.0)
RNF-06	—	Estándar (5.1)	Todas	Todos los scripts	—	Inspección en el editor (0 advertencias)	Aprobado: 0 errores y 0 advertencias en el editor tras corregir DF-05 (v1.0.0, 21 oct.)
RNF-07	HU-07	ADR-004	—	`test/unit/*.gd`	—	Reporte GUT (42 pruebas)	Aprobado: 42/42 pruebas en `main` final; reporte `docs/reporte_gut.xml` (#46, 22 oct.)
RNF-08	HU-21	ADR-002	—	`gestor_guardado.gd` → `cargar()`	PU-38, PU-39, PU-40	Partida completa sin errores en el Depurador	PU aprobadas (42/42)

Nota. Elaboración propia.

Resumen de cobertura: 27 requisitos (19 funcionales y 8 no funcionales); 27 con al menos una prueba asignada (100 %); 42 pruebas unitarias, todas aprobadas; casos funcionales aprobados: 20 de 20; requisitos con resultado registrado: 27 de 27 (100 %).

7. Despliegue y cierre

7.1 Manual de usuario

Este manual es para quien va a jugar. No necesitas saber nada de programación.

7.1.1 Qué necesitas

- Un computador con Windows 10 u 11 de 64 bits.
- Una tarjeta de video compatible con OpenGL 3.3 (casi cualquier computador de los últimos diez años, incluidos los que tienen gráficos integrados).
- Unos 150 MB libres en el disco.
- Teclado. El ratón es opcional y sirve para los menús y para atacar.

7.1.2 Descargar y abrir el juego

1. Entra a https://github.com/jaider23100/FragmentosDeLuz/releases/tag/v1.0.0 y descarga el archivo `FragmentosDeLuz-v1.0.0-windows.zip`.
2. Haz clic derecho sobre el archivo y elige Extraer todo. Puedes guardarlo en cualquier carpeta.
3. Abre la carpeta y haz doble clic en `FragmentosDeLuz.exe`. No hace falta instalar nada. Deja el archivo `FragmentosDeLuz.pck` en la misma carpeta: ahí están los gráficos y los sonidos.

7.1.3 Controles

Tabla 69

Controles del juego

Acción	Teclas
Caminar	A y D, o las flechas izquierda y derecha
Saltar (y doble salto en el aire)	Espacio, W o flecha arriba
Impulso	Mayúsculas o K
Atacar con la espada	J o clic izquierdo
Pausa	Esc
Avanzar textos y confirmar en los menús	Enter o Espacio

Nota. Elaboración propia.

7.1.4 Objetivo y reglas

Eres Sir Aldric y debes recuperar los **cinco fragmentos** del Cristal de Luz y llevarlos al **santuario** que está al final del bosque.

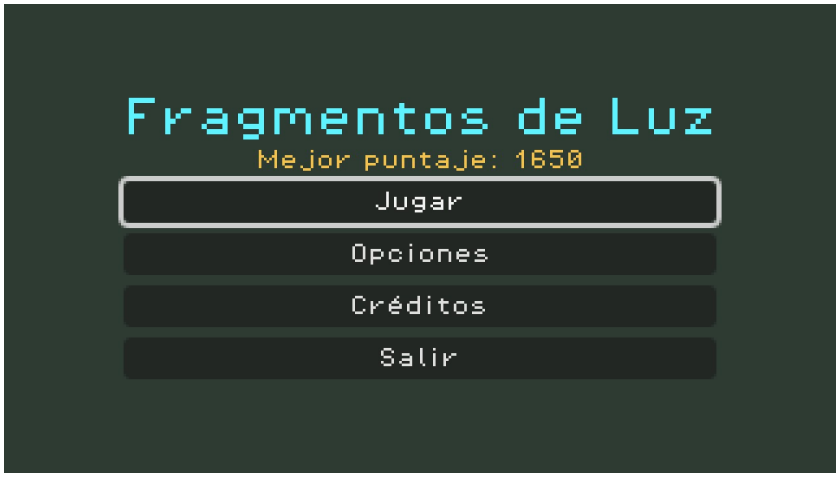
- Empiezas con **3 corazones**. Cada vez que te toca un enemigo o unas púas pierdes uno, y durante un segundo no puedes perder otro (el caballero parpadea).
- Si caes al vacío pierdes un corazón y vuelves al último **estandarte** que tocaste, o al inicio si no tocaste ninguno.
- El slime muere de un golpe y el murciélago de dos. Cada enemigo vale **50 puntos**, cada fragmento **100** y cada moneda **10**.
- Si llegas al santuario sin los cinco fragmentos, te dice cuántos te faltan.
- Al ganar recibes **100 puntos extra por cada corazón** que te quede. Si te quedas sin corazones, pierdes.
- Los carteles de piedra te enseñan cada movimiento nuevo: léelos.

7.1.5 Pantallas

Las Figuras 43 a 50 muestran cada pantalla del juego.

Figura 43

Pantalla: menú principal



Nota. Captura del juego, versión 1.0.0.

Muestra el título, tu mejor puntaje y cuatro botones: Jugar empieza una partida; Opciones abre el volumen; Créditos muestra quién hizo el juego; Salir lo cierra. Se maneja con el ratón o con las flechas y Enter.

Figura 44

Pantalla: opciones



Nota. Captura del juego, versión 1.0.0.

Dos barras para el volumen de la música y de los efectos. Al mover la de efectos suena un ejemplo. Volver guarda los valores, que se conservan aunque cierres el juego.

Figura 45

Pantalla: introducción



Nota. Captura del juego, versión 1.0.0.

Cuatro pantallas cuentan la historia. El texto aparece letra por letra: Enter lo completa o pasa a la siguiente pantalla, y Esc salta toda la introducción.

Figura 46

Pantalla: juego y HUD

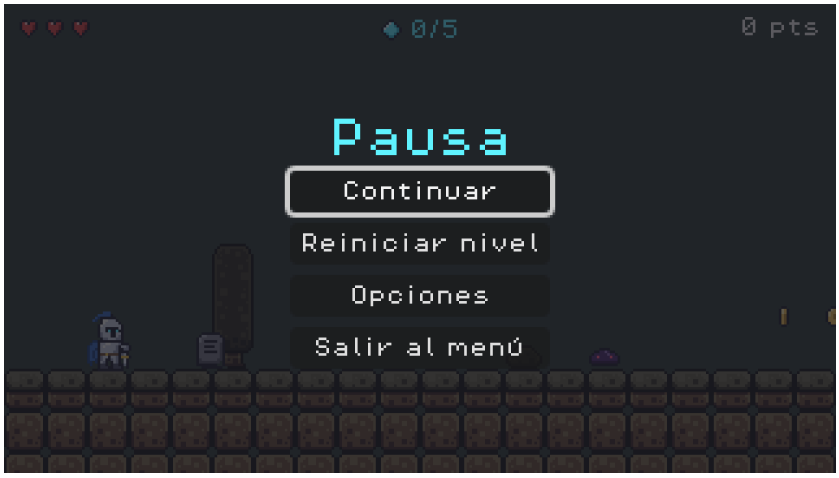


Nota. Captura del juego, versión 1.0.0.

Arriba a la izquierda están tus corazones; en el centro, los fragmentos recogidos (de 0/5 a 5/5); a la derecha, los puntos. Cuando te acercas a un cartel de piedra, su mensaje aparece bajo el HUD.

Figura 47

Pantalla: pausa



Nota. Captura del juego, versión 1.0.0.

Con Esc el juego se detiene. Continuar (o Esc) vuelve al juego; Reiniciar nivel empieza de cero; Opciones abre el volumen; Salir al menú vuelve al menú principal.

Figura 48

Pantalla: victoria



Nota. Captura del juego, versión 1.0.0.

Aparece después de la cinemática final. Muestra tus puntos, el bono de 100 por cada corazón que te quedó y el total. Si superaste tu mejor puntaje, lo indica y lo guarda.

Figura 49

Pantalla: derrota

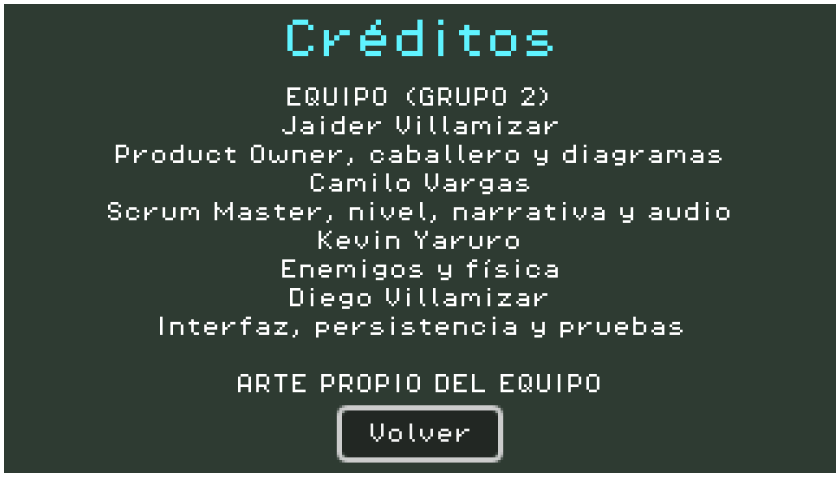


Nota. Captura del juego, versión 1.0.0.

Aparece si te quedas sin corazones. Muestra cuántos fragmentos y puntos alcanzaste. Reintentar empieza el nivel de nuevo.

Figura 50

Pantalla: créditos



Nota. Captura del juego, versión 1.0.0.

El texto sube solo; con las flechas puedes moverlo. Volver o Esc regresan al menú.

7.1.6 Guardado

El juego guarda solo tu **mejor puntaje** y el **volumen** de la música y de los efectos, en el archivo `%APPDATA%\Godot\app_userdata\Fragmentos de Luz\config.cfg`. La partida no se guarda a mitad del nivel: si cierras el juego, la próxima vez empiezas desde el principio. Para borrar tu récord, elimina ese archivo.

7.1.7 Solución de problemas

Tabla 70

Problemas frecuentes

Problema	Solución
Windows muestra «Windows protegió su PC» al abrir el juego	Haz clic en Más información y luego en Ejecutar de todas formas. Aparece porque el ejecutable no tiene firma digital (DT-08); el juego no modifica el equipo.
El juego no abre o se cierra enseguida	Verifica que `FragmentosDeLuz.pck` esté en la misma carpeta que el `.exe` y que el `.zip` esté descomprimido (no lo abras desde dentro del `.zip`). Actualiza el controlador de video.
No se oye nada	Revisa el volumen de Windows y, en el juego, Opciones: si una barra está en 0 %, ese sonido está apagado.
El juego no reconoce el mando	El juego solo funciona con teclado y ratón.
Quiero borrar mi mejor puntaje	Elimina el archivo `config.cfg` indicado en la sección 7.1.6.

Nota. Elaboración propia.

7.1.8 Créditos

Fragmentos de Luz fue hecho por Jaider Villamizar, Camilo Vargas, Kevin Yaruro y Diego Villamizar (grupo 2). Los recursos de terceros y sus licencias están en la pantalla de créditos del juego y en la sección 2.7.3. Hecho con Godot Engine (licencia MIT).

7.2 Manual técnico

Este manual es para un desarrollador que recibe el proyecto. Siguiéndolo se obtiene el mismo ejecutable de la versión 1.0.0. Los nombres de los menús son los del editor de Godot en español.

7.2.1 Obtener el proyecto

Clonar el repositorio y ubicarse en la versión entregada

```
git clone https://github.com/jaider23100/FragmentosDeLuz.git
cd FragmentosDeLuz
git switch --detach v1.0.0
```

7.2.2 Instalar Godot y las plantillas de exportación

1. Descargar **Godot 4.5-stable** (versión estándar, no la de .NET) desde https://godotengine.org/download/archive/4.5-stable/. No requiere instalación: se descomprime y se abre el ejecutable.

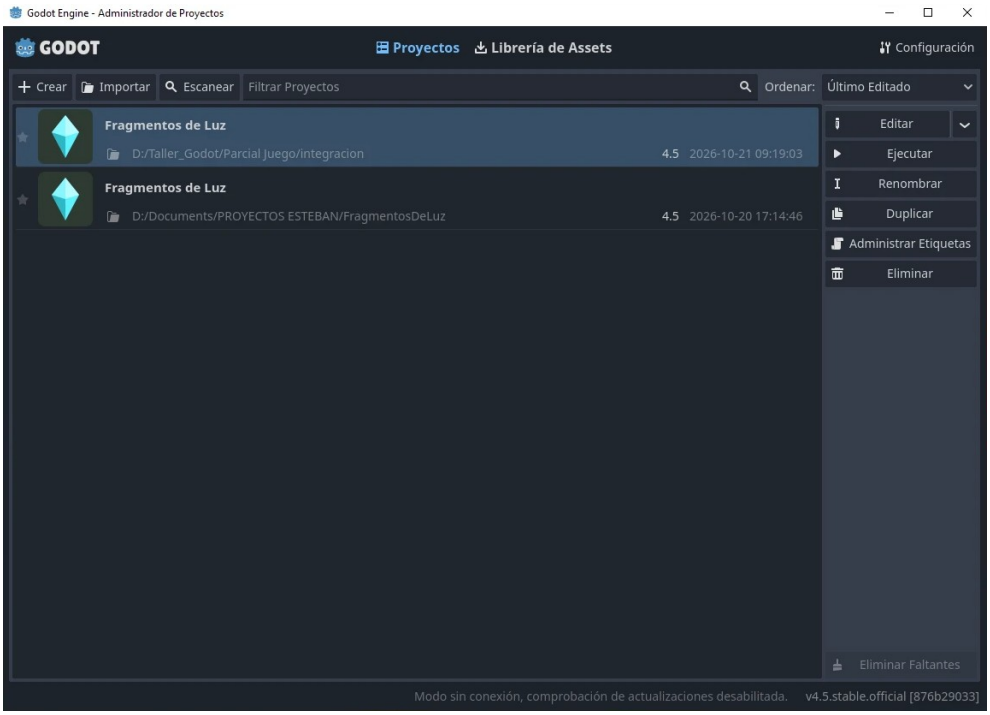
2. En el editor, abrir Editor › Administrar Plantillas de Exportación… y elegir Descargar e instalar. La versión de las plantillas debe ser exactamente **4.5.stable**, igual a la del editor (Godot Engine, s. f.-c). Sin conexión, se puede descargar `Godot_v4.5-stable_export_templates.tpz` de la misma página y usar Instalar desde un archivo.

7.2.3 Abrir el proyecto

1. En el administrador de proyectos de Godot, elegir Importar y seleccionar el archivo `project.godot` de la carpeta clonada.
2. Esperar a que Godot importe los recursos (la primera vez tarda unos segundos).
3. Presionar F5: el juego debe abrir en el menú principal sin errores en el panel Depurador.

Figura 51

Administrador de proyectos de Godot con el botón Importar y el proyecto abierto



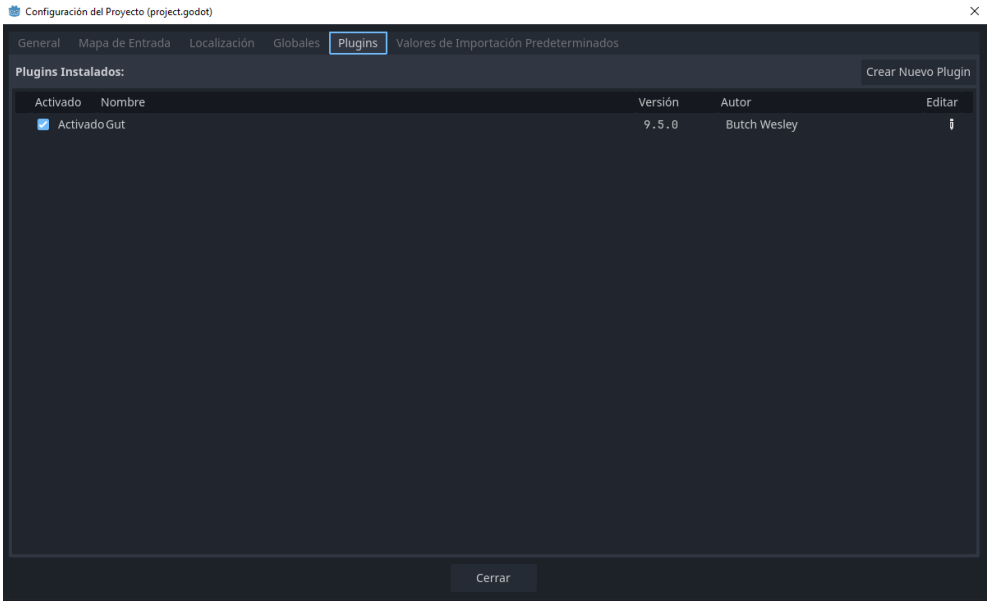
Nota. Captura de Godot 4.5.

7.2.4 Complementos

El único complemento es **GUT 9.5.0**, incluido en `addons/gut/`. Ya viene activado; si no lo está, se activa en Proyecto › Configuración del Proyecto › pestaña Plugins, marcando Gut como habilitado. No hay que descargar nada más.

Figura 52

Complemento GUT 9.5.0 activado en la pestaña Plugins



Nota. Captura del editor Godot 4.5, versión 1.0.0.

7.2.5 Ejecutar las pruebas

Desde el editor: panel inferior GUT › Run All. Por línea de comandos, desde la carpeta del proyecto:

Ejecución de las 42 pruebas con reporte JUnit XML

```
"<ruta>\Godot_v4.5-stable_win64_console.exe" --headless --path . -s addons/gut/gut_cmdln.gd
-d
    -gdir=res://test/unit -gexit -gjunit_xml_file=res://docs/reporte_gut.xml
```

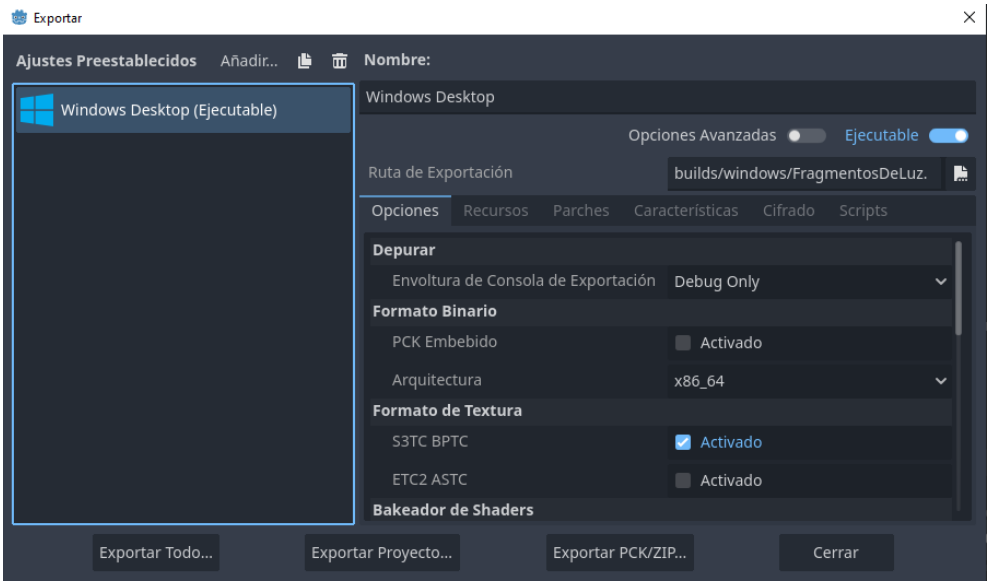
El resultado esperado es «All tests passed!» con 42 pruebas aprobadas.

7.2.6 Exportar el ejecutable

1. Abrir Proyecto › Exportar…. La configuración «Windows Desktop» ya está definida en `export_presets.cfg` (x86_64, versión 1.0.0.0, sin `test/`, `herramientas/`, `docs/` ni `addons/gut/`).
2. Elegir Exportar Proyecto…, desmarcar Exportar con depuración y guardar como `builds/windows/FragmentosDeLuz.exe`. Se generan `FragmentosDeLuz.exe` y `FragmentosDeLuz.pck`. La carpeta `builds/` está excluida del repositorio.
3. Comprimir la carpeta `builds/windows` como `FragmentosDeLuz-v1.0.0-windows.zip` y probar el `.exe` en un equipo sin Godot.

Figura 53

Diálogo Exportar con la configuración «Windows Desktop»



Nota. Captura del editor Godot 4.5, versión 1.0.0.

También se puede exportar sin abrir el editor:

Exportación por línea de comandos

```
"<ruta>\Godot_v4.5-stable_win64_console.exe" --headless --path . --export-release "Windows
Desktop"
    builds/windows/FragmentosDeLuz.exe
```

7.2.7 Publicar la versión

1. Fusionar en `main` la solicitud de integración de la versión (con `export_presets.cfg` y `CHANGELOG.md` actualizados).
2. Crear y subir la etiqueta anotada: `git tag -a v1.0.0 -m "v1.0.0: entrega final"` y `git push origin v1.0.0`.
3. En GitHub, Releases › Draft a new release: elegir la etiqueta, poner como título la versión, pegar la sección correspondiente de `CHANGELOG.md`, adjuntar el `.zip` y publicar (GitHub, s. f.-b).

7.2.8 Regenerar el arte y el nivel (opcional)

Las imágenes y sonidos propios se generan con los scripts de `herramientas/arte/` (requieren Python 3 y Pillow; por ejemplo, `python herramientas/arte/generar_personajes.py`). El nivel se construye ejecutando la escena `herramientas/nivel/construir_nivel.tscn` con F6; esto reescribe `nivel_bosque.tscn`, así que los cambios hechos a mano en el nivel se pierden (DT-05). Después de regenerar recursos, Godot los vuelve a importar al abrir el proyecto.

7.3 Notas de versión

El archivo `CHANGELOG.md` del repositorio sigue el formato de Keep a Changelog (Keep a Changelog, s. f.), con una sección por versión, su fecha y su número según el versionado semántico, y los cambios agrupados en Añadido, Cambiado, Corregido y Eliminado. Estas son las notas de la versión 1.0.0:

CHANGELOG.md, sección de la versión 1.0.0

```
## [1.0.0] - 2026-10-23

### Añadido
- Guardado del mejor puntaje y del volumen en `user://config.cfg` con el autoload
GestorGuardado; tolera archivos ausentes o dañados (HU-21).
- El menú principal muestra el mejor puntaje y la victoria avisa "¡Nuevo mejor puntaje!"
(RN-09).
- Pantalla de créditos con el equipo, los recursos de terceros, sus licencias y el aviso de
Godot (HU-22).
- Ejecutable de Windows que funciona sin instalar Godot (HU-23).

### Cambiado
- El volumen inicial se toma de la configuración guardada y se guarda al presionar Volver en
Opciones.
```

Limitaciones conocidas de la versión 1.0.0:

- Un solo nivel; no hay jefe final ni guardado a mitad del nivel.
- Solo teclado y ratón; las teclas no se pueden cambiar.
- Solo para Windows de 64 bits; no hay versiones web, Linux ni móviles.
- El ejecutable no está firmado digitalmente, por lo que Windows puede mostrar una advertencia la primera vez (DT-08).
- Las demás limitaciones técnicas están en el registro de deuda técnica (sección 5.5).

7.4 Métricas de proceso

7.4.1 Velocidad por iteración

La Tabla 71 compara los puntos comprometidos con los terminados en cada sprint.

Tabla 71

Velocidad por iteración

Sprint	Puntos comprometidos	Puntos terminados	Historias terminadas	Observaciones
Sprint 1	25	25	7 de 7	Las siete historias se fusionaron en `main` el 20 oct. y se publicó v0.1.0
Sprint 2	37	37	12 de 12	Incluye la corrección de DF-06 (#39); v0.2.0 se publicó el 22 oct. a las 4:58 p. m., unas horas después del mediodía previsto
Sprint 3	10	10	4 de 4	HU-20, HU-21 y HU-22 entraron el 22 oct. (#40, #44, #45); HU-23 con la versión 1.0.0 el 23 oct. (#47)
Total	72	72	23 de 23	—

Nota. Elaboración propia.

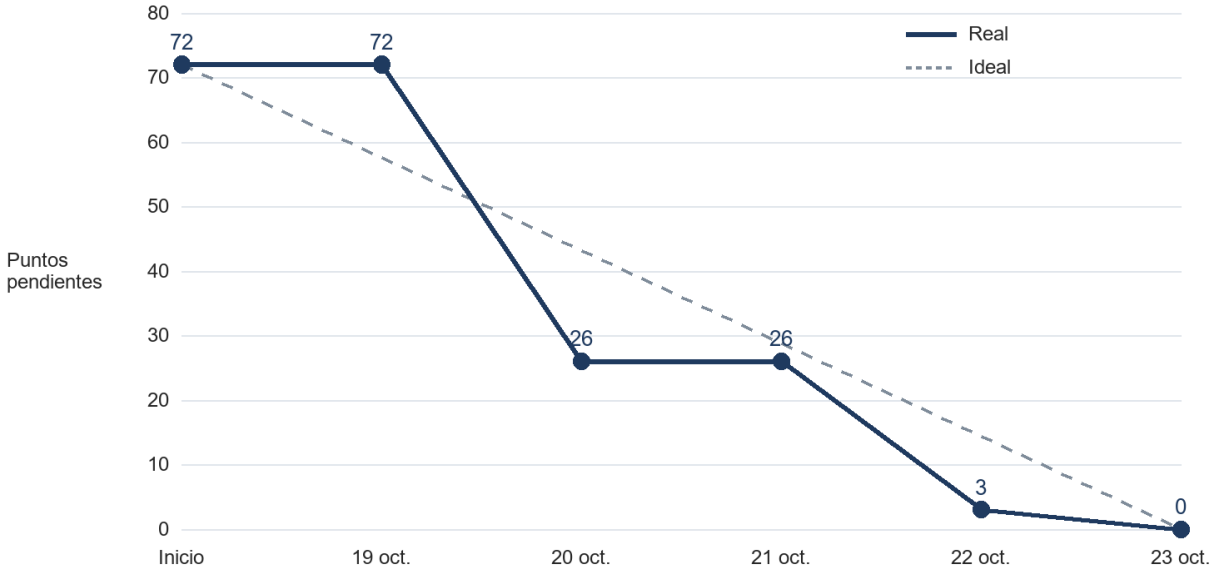
7.4.2 Gráfico de trabajo pendiente

El gráfico de trabajo pendiente muestra los puntos que faltan por terminar al final de cada día, frente a la línea ideal que va de 72 a 0 en cinco días. Se construye con las fechas en que se fusionó cada historia en `main` (Figura 54): el 20 oct. por la noche entraron 46 puntos (el Sprint 1 completo y parte del Sprint 2), el 22 oct. otros 23 y el 23 oct. los 3 de la HU-23. La línea real

queda por encima de la ideal el primer día, porque nada se integró antes de tener GUT y el caballero (orden de dependencias del Sprint 1), y por debajo desde el segundo día.

Figura 54

Gráfico de trabajo pendiente del proyecto



Nota. Elaboración propia con las fechas de fusión de GitHub (hora de Colombia).

7.4.3 Contribuciones por integrante

Las confirmaciones se cuentan con `git shortlog -sn --no-merges` sobre la etiqueta v1.0.0 (Git, s. f.); como cada solicitud se fusiona con «Squash and merge», cada una deja una confirmación en `main` a nombre de su autor. Las solicitudes creadas y revisadas salen de GitHub y las historias terminadas, del tablero. La Tabla 72 las resume.

Confirmaciones por integrante en main (git shortlog -sn --no-merges)

```
9  DiegoVillamizar
6  jaider23100          (incluye la estructura inicial del 19 oct.)
5  Camilo160605
5  TheYaru              (Kevin Yaruro)
```

Tabla 72

Contribuciones por integrante

Integrante	Rol	Confirmaciones	Solicitudes creadas	Solicitudes revisadas	HU terminadas (puntos)
Jaider Villamizar	Product Owner y Developer	6	5 (#24, #25, #33, #37, #48)	6	5 (17)
Camilo Vargas	Scrum Master y Developer	5	6 (una cerrada sin fusionar, #42)	6	6 (16)
Kevin Yaruro	Developer	5	5	9	4 (16)
Diego Villamizar	Developer	9	9	4	8 (23)
Total	—	25	25	25	23 (72)

Nota. Elaboración propia con datos de git y GitHub al 23 oct. 2026. Revisiones contadas como aprobaciones; la #45 la aprobaron dos integrantes.

Diferencias de participación: Diego tiene más historias y solicitudes porque le tocaron la interfaz completa, la persistencia, las pruebas automatizadas y las tres versiones publicadas (HU-23); Kevin revisó más solicitudes porque la rotación lo asignó como revisor de Diego y de Camilo en dos sprints. Los cuatro integrantes crearon y revisaron solicitudes y terminaron historias en los tres sprints.

7.5 Retrospectiva y lecciones aprendidas

Qué funcionó: los sprints cortos con un objetivo claro; la revisión cruzada rotativa de las solicitudes de integración; las 42 pruebas unitarias, que permitieron integrar sin regresiones; la composición de escenas, que evitó conflictos sobre el mismo archivo; y exportar desde el primer sprint, que adelantó el defecto DF-01.

Qué no funcionó (retrospectiva final con los cuatro integrantes, 22 oct.): el empuje de la roca, que no movía la roca y llegó a la versión 1.0.0 (DF-06, encontrado en las pruebas de juego), y el impulso, que hubo que terminar de ajustar y que al principio no se entendió (PT-01).

Qué se haría distinto: mejorar mucho la parte visual y la jugabilidad del proyecto, en línea con lo que pidieron los participantes de las pruebas de juego (PT-03 y PT-05).

Riesgos: el equipo considera que los riesgos se manejaron bien. Los datos lo respaldan en parte: el Sprint 1 terminó el 100 % de lo comprometido, no hubo conflictos de fusión (R-04) y nadie faltó a las reuniones (R-10); en cambio, R-05 se materializó en parte (4 de las 5 pruebas de juego previstas) y R-06 (errores de física) se presentó con DF-06.

La Tabla 73 registra las lecciones aprendidas.

Tabla 73

Lecciones aprendidas

ID	Área	Situación (qué ocurrió)	Impacto en el proyecto	Recomendación
LA-01	Documentación y código	Al pasar el borrador al documento final se encontraron datos que ya no coincidían con el código: la fuerza del viento, el número de púas, la forma de empujar la roca y quién guarda el récord (DF-04)	Riesgo de entrega inválida por incoherencia (R-02)	Generar las tablas y fragmentos del documento a partir del código y revisar la matriz de trazabilidad en cada revisión del sprint
LA-02	Exportación	El aviso de versión inválida (DF-01) solo apareció al exportar por primera vez	Corrección antes de publicar la versión 0.1.0, sin afectar al usuario	Exportar desde el primer sprint, como se planeó en R-03
LA-03	Pruebas	Una prueba de la pantalla de victoria habría escrito en el guardado real del jugador (DF-02)	Se detectó en revisión, antes de integrar	Hacer que todo lo que escribe en disco reciba la ruta como dato, para que las pruebas usen archivos temporales
LA-04	Organización del equipo	Los usuarios de GitHub de dos integrantes llegaron después del primer día, lo que retrasó sus invitaciones al repositorio	Esos integrantes no pudieron subir su trabajo hasta aceptar la invitación	Crear el repositorio e invitar a todos en la primera reunión, antes de repartir tareas
LA-05	Pruebas de juego	Conseguir personas externas para las pruebas de juego requiere agendarlas con anticipación (R-05)	Se realizaron 4 de las 5 sesiones previstas: la quinta persona no pudo asistir	Contactar a los participantes desde el primer sprint y tener listo el kit de pruebas antes del último sprint
LA-06	Rendimiento	El umbral «promedio ≥ 60 FPS» coincide con el tope de una pantalla de 60 Hz: una sola caída aislada (51 FPS) dejó el promedio de la zona 3 en 59,9	RNF-01 quedó cumplido con observación aunque el juego no presentó problemas visibles	Fijar umbrales por debajo del tope de la pantalla (por ejemplo, promedio ≥ 58 FPS y percentil 1 % ≥ 30) o medir con la sincronización vertical desactivada
LA-07	Pruebas	La primera ejecución de CP-09 se marcó aprobada con una captura que no mostraba el resultado esperado; la roca no se movía (DF-06) y lo revelaron las pruebas de juego	Un defecto de severidad alta llegó a la versión 1.0.0 y hubo que volver a exportar los ejecutables	Aprobar un caso solo cuando la evidencia muestre el resultado esperado completo, no un paso intermedio

Nota. Elaboración propia.

7.5.1 Trabajo futuro

Incluye los cambios que pidieron o mostraron las pruebas de juego (sección 6.5):

- Combate con Morvak como jefe final y más niveles con progresión (pedido en las sesiones 1 y 3, PT-05).
- Nombrar la acción «impulso» en el cartel de la zona 2 y mostrar la tecla al acercarse (PT-01).
- Revisar el daño y la velocidad del murciélago de la zona 4, que hizo abandonar a dos participantes (PT-03), y probar el cambio con nuevas sesiones.
- Nuevos diseños visuales y variedad de enemigos (PT-05).

- Guardado de la partida a mitad del nivel.
- Soporte de mando y reasignación de teclas.
- Versiones web y Linux a partir del mismo proyecto (el renderizador Compatibility ya lo permite, ADR-003).
- Pagar la deuda técnica registrada en la sección 5.5, empezando por DT-03 y DT-04.

Referencias

American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7.ª ed.). https://doi.org/10.1037/0000165-000

Bitwes. (s. f.). *GUT: Godot Unit Test* (versión 9.5.0) [Software]. GitHub. https://github.com/bitwes/Gut

Brooke, J. (1996). SUS: A «quick and dirty» usability scale. En P. W. Jordan, B. Thomas, B. A. Weerdmeester y I. L. McClelland (Eds.), *Usability evaluation in industry* (pp. 189-194). Taylor & Francis.

Brown, S. (s. f.). *The C4 model for visualising software architecture*. https://c4model.com/

Cohn, M. (2004). *User stories applied: For agile software development*. Addison-Wesley.

Conventional Commits. (s. f.). *Conventional Commits 1.0.0*. https://www.conventionalcommits.org/es/v1.0.0/

Cucumber. (s. f.). *Gherkin reference*. https://cucumber.io/docs/gherkin/reference/

Entertainment Software Rating Board. (s. f.). *Ratings guide*. https://www.esrb.org/ratings-guide/

Git. (s. f.). *git-shortlog documentation*. https://git-scm.com/docs/git-shortlog

GitHub. (s. f.-a). *Acerca de las solicitudes de incorporación de cambios*. https://docs.github.com/es/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-pull-requests

GitHub. (s. f.-b). *Acerca de los lanzamientos*. https://docs.github.com/es/repositories/releasing-projects-on-github/about-releases

Godot Engine. (s. f.-a). *Complying with licenses*. Godot Engine 4.5 documentation. https://docs.godotengine.org/en/4.5/about/complying_with_licenses.html

Godot Engine. (s. f.-b). *Debugger panel*. Godot Engine 4.5 documentation. https://docs.godotengine.org/en/4.5/tutorials/scripting/debug/debugger_panel.html

Godot Engine. (s. f.-c). *Exporting projects*. Godot Engine 4.5 documentation. https://docs.godotengine.org/en/4.5/tutorials/export/exporting_projects.html

Godot Engine. (s. f.-d). *GDScript documentation comments*. Godot Engine 4.5 documentation. https://docs.godotengine.org/en/4.5/tutorials/scripting/gdscript/gdscript_documentation_comments.html

Godot Engine. (s. f.-e). *GDScript style guide*. Godot Engine 4.5 documentation. https://docs.godotengine.org/en/4.5/tutorials/scripting/gdscript/gdscript_styleguide.html

Godot Engine. (s. f.-f). *Project organization*. Godot Engine 4.5 documentation. https://docs.godotengine.org/en/4.5/tutorials/best_practices/project_organization.html

Godot Engine. (s. f.-g). *Saving games*. Godot Engine 4.5 documentation. https://docs.godotengine.org/en/4.5/tutorials/io/saving_games.html

Godot Engine. (s. f.-h). *Creating instances*. Godot Engine 4.5 documentation. https://docs.godotengine.org/en/4.5/getting_started/step_by_step/instancing.html

Godot Engine. (s. f.-i). *Singletons (Autoload)*. Godot Engine 4.5 documentation. https://docs.godotengine.org/en/4.5/tutorials/scripting/singletons_autoload.html

Godot Engine. (s. f.-j). *Static typing in GDScript*. Godot Engine 4.5 documentation. https://docs.godotengine.org/en/4.5/tutorials/scripting/gdscript/static_typing.html

Godot Engine. (s. f.-k). *The Profiler*. Godot Engine 4.5 documentation. https://docs.godotengine.org/en/4.5/tutorials/scripting/debug/the_profiler.html

Godot Engine. (s. f.-l). *Using signals*. Godot Engine 4.5 documentation. https://docs.godotengine.org/en/4.5/getting_started/step_by_step/signals.html

Godot Engine. (s. f.-m). *Version control systems*. Godot Engine 4.5 documentation. https://docs.godotengine.org/en/4.5/tutorials/best_practices/version_control_systems.html

Institute of Electrical and Electronics Engineers. (1998). *IEEE recommended practice for software requirements specifications* (IEEE Std 830-1998). https://doi.org/10.1109/IEEESTD.1998.88286

International Organization for Standardization. (2018). *Systems and software engineering — Life cycle processes — Requirements engineering* (ISO/IEC/IEEE 29148:2018). https://www.iso.org/standard/72089.html

International Organization for Standardization. (2021). *Software and systems engineering — Software testing — Part 3: Test documentation* (ISO/IEC/IEEE 29119-3:2021). https://www.iso.org/standard/79429.html

International Organization for Standardization. (2023). *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model* (ISO/IEC 25010:2023). https://www.iso.org/standard/78176.html

Keep a Changelog. (s. f.). *Mantenga un registro de cambios* (versión 1.1.0). https://keepachangelog.com/es-ES/1.1.0/

Maddy Makes Games. (2018). *Celeste* [Videojuego]. Maddy Makes Games.

Nintendo. (1985). *Super Mario Bros.* [Videojuego]. Nintendo.

Nygard, M. (2011, 15 de noviembre). *Documenting architecture decisions*. Cognitect. https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions

Nystrom, R. (2014). *Game programming patterns*. Genever Benning. https://gameprogrammingpatterns.com/

Object Management Group. (2017). *OMG Unified Modeling Language (OMG UML)* (versión 2.5.1). https://www.omg.org/spec/UML/2.5.1/

PEGI. (s. f.). *Pan European Game Information*. https://pegi.info/es

Preston-Werner, T. (s. f.). *Versionado semántico 2.0.0*. https://semver.org/lang/es/

Schwaber, K. y Sutherland, J. (2020). *La Guía Scrum: la guía definitiva de Scrum, las reglas del juego*. https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-Spanish-Latin-South-American.pdf

Team Cherry. (2017). *Hollow Knight* [Videojuego]. Team Cherry.

Yacht Club Games. (2014). *Shovel Knight* [Videojuego]. Yacht Club Games.

Anexo A. Historias de usuario

Cada historia de la pila del producto (sección 3.5) con la plantilla completa. Los casos de prueba son los de la sección 6.3 y el Anexo B.

Tabla 74

Historia de usuario HU-01: Moverse y saltar

Campo	Contenido
ID y título	HU-01 Moverse y saltar
Historia	Como jugador, quiero mover al caballero y saltar, para recorrer el bosque y superar obstáculos.
Criterios de aceptación	Escenario 1: Dado que el caballero está en el suelo, cuando mantengo D o la flecha derecha, entonces avanza a la derecha a 100 px/s. Escenario 2: Dado que el caballero está en el suelo, cuando presiono Espacio, entonces salta unos 42 px de altura.
Estimación y prioridad	Puntos: 3 Prioridad (MoSCoW): Obligatoria
Iteración y estado	Iteración: Sprint 1 Estado: Terminada Responsable: Jaider
Requisitos y pruebas	RF/RNF: RF-01 Casos de prueba: CP-01

Nota. Elaboración propia.

Tabla 75

Historia de usuario HU-02: Doble salto

Campo	Contenido
ID y título	HU-02 Doble salto
Historia	Como jugador, quiero saltar otra vez en el aire, para alcanzar plataformas altas.
Criterios de aceptación	Escenario 1: Dado que el caballero está en el aire y no ha usado el doble salto, cuando presiono Espacio, entonces vuelve a impulsarse hacia arriba. Escenario 2: Dado que ya usó el doble salto, cuando presiono Espacio de nuevo en el aire, entonces no ocurre nada hasta tocar el suelo.
Estimación y prioridad	Puntos: 3 Prioridad (MoSCoW): Obligatoria
Iteración y estado	Iteración: Sprint 1 Estado: Terminada Responsable: Jaider
Requisitos y pruebas	RF/RNF: RF-02 Casos de prueba: CP-02

Nota. Elaboración propia.

Tabla 76

Historia de usuario HU-03: Impulso

Campo	Contenido
ID y título	HU-03 Impulso
Historia	Como jugador, quiero hacer una embestida rápida, para cruzar huecos y esquivar enemigos.
Criterios de aceptación	Escenario 1: Dado que el impulso está disponible, cuando presiono Mayúsculas, entonces el caballero avanza unos 39 px en 0,15 s sin caer. Escenario 2: Dado que acabo de usar el impulso, cuando presiono Mayúsculas antes de 0,5 s, entonces no se ejecuta.
Estimación y prioridad	Puntos: 5 Prioridad (MoSCoW): Obligatoria
Iteración y estado	Iteración: Sprint 1 Estado: Terminada Responsable: Jaider
Requisitos y pruebas	RF/RNF: RF-03 Casos de prueba: CP-03

Nota. Elaboración propia.

Tabla 77

Historia de usuario HU-04: Slime patrullero

Campo	Contenido
ID y título	HU-04 Slime patrullero
Historia	Como jugador, quiero enfrentar slimes que patrullan, para que el bosque tenga peligros.
Criterios de aceptación	Escenario 1: Dado un slime caminando, cuando llega a una pared o a un borde, entonces da la vuelta. Escenario 2: Dado un slime activo, cuando el caballero lo toca, entonces el caballero pierde 1 corazón.
Estimación y prioridad	Puntos: 5 Prioridad (MoSCoW): Obligatoria
Iteración y estado	Iteración: Sprint 1 Estado: Terminada Responsable: Kevin
Requisitos y pruebas	RF/RNF: RF-07 Casos de prueba: CP-07

Nota. Elaboración propia.

Tabla 78

Historia de usuario HU-05: Puntos de control y caída al vacío

Campo	Contenido
ID y título	HU-05 Puntos de control y caída al vacío
Historia	Como jugador, quiero reaparecer en el último estandarte al caer, para no repetir todo el nivel.
Criterios de aceptación	Escenario 1: Dado que toqué el punto de control 1, cuando caigo al vacío, entonces reaparezco en el punto de control 1 con un corazón menos. Escenario 2: Dado que no activé ningún punto de control, cuando caigo al vacío, entonces reaparezco en el inicio del nivel.
Estimación y prioridad	Puntos: 3 Prioridad (MoSCoW): Obligatoria
Iteración y estado	Iteración: Sprint 1 Estado: Terminada Responsable: Camilo
Requisitos y pruebas	RF/RNF: RF-06 Casos de prueba: CP-06

Nota. Elaboración propia.

Tabla 79

Historia de usuario HU-06: Menú principal

Campo	Contenido
ID y título	HU-06 Menú principal
Historia	Como jugador, quiero un menú al abrir el juego, para empezar a jugar o salir.
Criterios de aceptación	Escenario 1: Dado que abro el juego, cuando carga, entonces veo el menú con Jugar, Opciones, Créditos y Salir. Escenario 2: Dado el menú principal, cuando presiono Jugar, entonces se muestra la introducción.
Estimación y prioridad	Puntos: 3 Prioridad (MoSCoW): Obligatoria
Iteración y estado	Iteración: Sprint 1 Estado: Terminada Responsable: Diego
Requisitos y pruebas	RF/RNF: RF-18 Casos de prueba: CP-18

Nota. Elaboración propia.

Tabla 80

Historia de usuario HU-07: Pruebas automatizadas

Campo	Contenido
ID y título	HU-07 Pruebas automatizadas
Historia	Como desarrollador, quiero ejecutar las pruebas unitarias por línea de comandos, para verificar la lógica antes de integrar.
Criterios de aceptación	Escenario 1: Dado el proyecto con GUT 9.5.0, cuando ejecuto `gut_cmdln.gd`, entonces se ejecutan las pruebas de `test/unit` y se genera el reporte XML. Escenario 2: Dada una prueba que falla, cuando se ejecutan las pruebas, entonces el reporte la marca como fallida.
Estimación y prioridad	Puntos: 3 Prioridad (MoSCoW): Obligatoria
Iteración y estado	Iteración: Sprint 1 Estado: Terminada Responsable: Diego
Requisitos y pruebas	RF/RNF: RNF-07 Casos de prueba: Pruebas unitarias (reporte GUT)

Nota. Elaboración propia.

Tabla 81

Historia de usuario HU-08: Atacar con la espada

Campo	Contenido
ID y título	HU-08 Atacar con la espada
Historia	Como jugador, quiero atacar con mi espada, para derrotar enemigos y ganar puntos.
Criterios de aceptación	Escenario 1: Dado un slime justo delante del caballero, cuando presiono J, entonces el slime es derrotado y sumo 50 puntos. Escenario 2: Dado que acabo de atacar, cuando presiono J antes de 0,35 s, entonces no se ejecuta otro ataque.
Estimación y prioridad	Puntos: 3 Prioridad (MoSCoW): Obligatoria
Iteración y estado	Iteración: Sprint 2 Estado: Terminada Responsable: Jaider
Requisitos y pruebas	RF/RNF: RF-04 Casos de prueba: CP-04

Nota. Elaboración propia.

Tabla 82

Historia de usuario HU-09: Vida, daño e invulnerabilidad

Campo	Contenido
ID y título	HU-09 Vida, daño e invulnerabilidad
Historia	Como jugador, quiero tener 3 corazones y un instante de protección tras un golpe, para que un error no me elimine de inmediato.
Criterios de aceptación	Escenario 1: Dado que tengo 3 corazones, cuando toco púas, entonces quedan 2 y el caballero parpadea. Escenario 2: Dado que acabo de recibir daño, cuando toco otro peligro antes de 1,0 s, entonces no pierdo otro corazón.
Estimación y prioridad	Puntos: 3 Prioridad (MoSCoW): Obligatoria
Iteración y estado	Iteración: Sprint 2 Estado: Terminada Responsable: Jaider
Requisitos y pruebas	RF/RNF: RF-05 Casos de prueba: CP-05

Nota. Elaboración propia.

Tabla 83

Historia de usuario HU-10: Murciélago perseguidor

Campo	Contenido
ID y título	HU-10 Murciélago perseguidor
Historia	Como jugador, quiero enfrentar murciélagos que me persiguen, para tener un reto aéreo.
Criterios de aceptación	Escenario 1: Dado un murciélago dormido, cuando el caballero se acerca a menos de 80 px, entonces despierta y lo persigue. Escenario 2: Dado un murciélago que persigue, cuando el caballero se aleja a más de 140 px, entonces regresa a su punto de origen.
Estimación y prioridad	Puntos: 5 Prioridad (MoSCoW): Importante
Iteración y estado	Iteración: Sprint 2 Estado: Terminada Responsable: Kevin
Requisitos y pruebas	RF/RNF: RF-08 Casos de prueba: CP-08

Nota. Elaboración propia.

Tabla 84

Historia de usuario HU-11: Roca empujable

Campo	Contenido
ID y título	HU-11 Roca empujable
Historia	Como jugador, quiero empujar una roca, para usarla de escalón y llegar a un fragmento alto.
Criterios de aceptación	Escenario 1: Dado el caballero junto a la roca, cuando camino hacia ella, entonces la roca se desplaza. Escenario 2: Dada la roca bajo la cornisa, cuando salto sobre ella y luego a la cornisa, entonces alcanzo el fragmento F4.
Estimación y prioridad	Puntos: 3 Prioridad (MoSCoW): Obligatoria
Iteración y estado	Iteración: Sprint 2 Estado: Terminada Responsable: Kevin
Requisitos y pruebas	RF/RNF: RF-09 Casos de prueba: CP-09

Nota. Elaboración propia.

Tabla 85

Historia de usuario HU-12: Corriente de viento

Campo	Contenido
ID y título	HU-12 Corriente de viento
Historia	Como jugador, quiero que el viento me eleve, para llegar a zonas altas.
Criterios de aceptación	Escenario 1: Dado el caballero dentro de la corriente, cuando no presiono nada, entonces sube. Escenario 2: Dado que salgo de la corriente, cuando estoy en el aire, entonces vuelvo a caer con la gravedad normal.
Estimación y prioridad	Puntos: 3 Prioridad (MoSCoW): Obligatoria
Iteración y estado	Iteración: Sprint 2 Estado: Terminada Responsable: Kevin
Requisitos y pruebas	RF/RNF: RF-10 Casos de prueba: CP-10

Nota. Elaboración propia.

Tabla 86

Historia de usuario HU-13: Fragmentos y puntaje

Campo	Contenido
ID y título	HU-13 Fragmentos y puntaje
Historia	Como jugador, quiero recoger los fragmentos del Cristal, para avanzar en la historia y sumar puntos.
Criterios de aceptación	Escenario 1: Dado que tengo 2/5 y 200 puntos, cuando toco un fragmento, entonces tengo 3/5 y 300 puntos. Escenario 2: Dado un fragmento recogido, cuando sigo jugando, entonces ese fragmento ya no aparece en el nivel.
Estimación y prioridad	Puntos: 3 Prioridad (MoSCoW): Obligatoria
Iteración y estado	Iteración: Sprint 2 Estado: Terminada Responsable: Camilo
Requisitos y pruebas	RF/RNF: RF-11 Casos de prueba: CP-11

Nota. Elaboración propia.

Tabla 87

Historia de usuario HU-14: Monedas

Campo	Contenido
ID y título	HU-14 Monedas
Historia	Como jugador, quiero recoger monedas opcionales, para mejorar mi puntaje si exploro.
Criterios de aceptación	Escenario 1: Dado que tengo 100 puntos, cuando toco una moneda, entonces tengo 110 y la moneda desaparece.
Estimación y prioridad	Puntos: 1 Prioridad (MoSCoW): Deseable
Iteración y estado	Iteración: Sprint 2 Estado: Terminada Responsable: Camilo
Requisitos y pruebas	RF/RNF: RF-12 Casos de prueba: CP-12

Nota. Elaboración propia.

Tabla 88

Historia de usuario HU-15: Historia en el juego

Campo	Contenido
ID y título	HU-15 Historia en el juego
Historia	Como jugador, quiero conocer la historia mientras juego, para entender por qué busco los fragmentos.
Criterios de aceptación	Escenario 1: Dado que presiono Jugar, cuando empieza la partida, entonces veo las 4 pantallas de la introducción, que puedo avanzar con Enter o saltar con Esc. Escenario 2: Dado el caballero cerca de un cartel, cuando entra en su área, entonces aparece el texto del cartel, y al alejarse desaparece. Escenario 3: Dado que gano el nivel, cuando termina, entonces veo las 3 pantallas de la cinemática final.
Estimación y prioridad	Puntos: 5 Prioridad (MoSCoW): Obligatoria
Iteración y estado	Iteración: Sprint 2 Estado: Terminada Responsable: Camilo
Requisitos y pruebas	RF/RNF: RF-16, RNF-03 Casos de prueba: CP-16

Nota. Elaboración propia.

Tabla 89

Historia de usuario HU-16: Música y efectos de sonido

Campo	Contenido
ID y título	HU-16 Música y efectos de sonido
Historia	Como jugador, quiero escuchar música y sonidos en cada acción, para sentir la respuesta del juego.
Criterios de aceptación	Escenario 1: Dado el menú, cuando se muestra, entonces suena la música del menú; al entrar al nivel cambia a la música del nivel. Escenario 2: Dado el nivel, cuando salto, hago el impulso, ataco, recibo daño o recojo un objeto, entonces suena el efecto correspondiente.
Estimación y prioridad	Puntos: 3 Prioridad (MoSCoW): Obligatoria
Iteración y estado	Iteración: Sprint 2 Estado: Terminada Responsable: Camilo
Requisitos y pruebas	RF/RNF: RF-17 Casos de prueba: CP-17

Nota. Elaboración propia.

Tabla 90

Historia de usuario HU-17: HUD

Campo	Contenido
ID y título	HU-17 HUD
Historia	Como jugador, quiero ver mis corazones, fragmentos y puntaje, para saber cómo voy.
Criterios de aceptación	Escenario 1: Dado el nivel en curso, cuando pierdo un corazón o recojo un fragmento, entonces el HUD se actualiza de inmediato.
Estimación y prioridad	Puntos: 2 Prioridad (MoSCoW): Obligatoria
Iteración y estado	Iteración: Sprint 2 Estado: Terminada Responsable: Diego
Requisitos y pruebas	RF/RNF: RF-14 Casos de prueba: CP-14

Nota. Elaboración propia.

Tabla 91

Historia de usuario HU-18: Victoria y derrota

Campo	Contenido
ID y título	HU-18 Victoria y derrota
Historia	Como jugador, quiero saber cuándo gané o perdí, para conocer mi resultado y decidir si reintento.
Criterios de aceptación	Escenario 1: Dado que tengo 5/5, cuando toco el santuario, entonces empieza la cinemática final y luego la pantalla de victoria con el puntaje final. Escenario 2: Dado que tengo 3/5, cuando toco el santuario, entonces aparece «Faltan 2 fragmentos» y sigo jugando. Escenario 3: Dado que tengo 1 corazón, cuando recibo daño, entonces aparece la pantalla de derrota con Reintentar y Menú principal.
Estimación y prioridad	Puntos: 3 Prioridad (MoSCoW): Obligatoria
Iteración y estado	Iteración: Sprint 2 Estado: Terminada Responsable: Diego
Requisitos y pruebas	RF/RNF: RF-13 Casos de prueba: CP-05, CP-13

Nota. Elaboración propia.

Tabla 92

Historia de usuario HU-19: Pausa y reinicio

Campo	Contenido
ID y título	HU-19 Pausa y reinicio
Historia	Como jugador, quiero pausar y reiniciar el nivel, para descansar o empezar de nuevo.
Criterios de aceptación	Escenario 1: Dado el nivel en curso, cuando presiono Esc, entonces el juego se detiene y aparece el menú de pausa. Escenario 2: Dado el menú de pausa, cuando elijo Reiniciar nivel, entonces empiezo con 3 corazones, 0/5 y 0 puntos.
Estimación y prioridad	Puntos: 3 Prioridad (MoSCoW): Obligatoria
Iteración y estado	Iteración: Sprint 2 Estado: Terminada Responsable: Diego
Requisitos y pruebas	RF/RNF: RF-15 Casos de prueba: CP-15

Nota. Elaboración propia.

Tabla 93

Historia de usuario HU-20: Opciones de volumen

Campo	Contenido
ID y título	HU-20 Opciones de volumen
Historia	Como jugador, quiero ajustar el volumen de la música y de los efectos, para jugar a mi gusto.
Criterios de aceptación	Escenario 1: Dado el panel de opciones, cuando muevo el deslizador de Música al 0 %, entonces la música deja de oírse pero los efectos siguen.
Estimación y prioridad	Puntos: 3 Prioridad (MoSCoW): Obligatoria
Iteración y estado	Iteración: Sprint 3 Estado: Terminada Responsable: Diego
Requisitos y pruebas	RF/RNF: RF-18 Casos de prueba: CP-18

Nota. Elaboración propia.

Tabla 94

Historia de usuario HU-21: Guardar mejor puntaje y volumen

Campo	Contenido
ID y título	HU-21 Guardar mejor puntaje y volumen
Historia	Como jugador, quiero que el juego recuerde mi mejor puntaje y mi volumen, para no perderlos al cerrar.
Criterios de aceptación	Escenario 1: Dado que logré 1700 puntos y el mejor era 1650, cuando cierro y vuelvo a abrir el juego, entonces el menú muestra 1700. Escenario 2: Dado que el archivo de guardado no existe o está dañado, cuando abro el juego, entonces inicia con mejor puntaje 0 y volumen por defecto, sin errores.
Estimación y prioridad	Puntos: 3 Prioridad (MoSCoW): Obligatoria
Iteración y estado	Iteración: Sprint 3 Estado: Terminada Responsable: Diego
Requisitos y pruebas	RF/RNF: RF-19, RNF-08 Casos de prueba: CP-19

Nota. Elaboración propia.

Tabla 95

Historia de usuario HU-22: Créditos

Campo	Contenido
ID y título	HU-22 Créditos
Historia	Como jugador, quiero ver los créditos, para conocer a los autores y las licencias.
Criterios de aceptación	Escenario 1: Dado el menú principal, cuando presiono Créditos, entonces veo el equipo, los recursos de terceros con su licencia y el aviso de Godot, y puedo volver.
Estimación y prioridad	Puntos: 1 Prioridad (MoSCoW): Obligatoria
Iteración y estado	Iteración: Sprint 3 Estado: Terminada Responsable: Camilo
Requisitos y pruebas	RF/RNF: RF-18 Casos de prueba: CP-18

Nota. Elaboración propia.

Tabla 96

Historia de usuario HU-23: Jugar sin instalar Godot

Campo	Contenido
ID y título	HU-23 Jugar sin instalar Godot
Historia	Como jugador, quiero descargar y abrir el juego directamente, para jugar sin instalar nada más.
Criterios de aceptación	Escenario 1: Dado un equipo con Windows 10 u 11 sin Godot, cuando descomprimo el `.zip` de la versión publicada y abro el `.exe`, entonces el juego inicia en el menú principal sin errores.
Estimación y prioridad	Puntos: 3 Prioridad (MoSCoW): Obligatoria
Iteración y estado	Iteración: Sprint 3 Estado: Terminada Responsable: Diego
Requisitos y pruebas	RF/RNF: RNF-05 Casos de prueba: CP-20

Nota. Elaboración propia.

Anexo B. Casos de prueba funcionales

Cada caso de la sección 6.3 con la plantilla completa. El resultado obtenido, el estado, la fecha y quién lo ejecutó se registran al ejecutarlo sobre el ejecutable v1.0.0.

Tabla 97

Caso de prueba CP-01: Moverse y saltar

Campo	Contenido
ID y nombre	CP-01 Moverse y saltar
Requisito y HU que verifica	RF-01 / HU-01
Tipo y técnica	Funcional / partición de equivalencia (en el suelo y en el aire)
Precondiciones y datos	Nivel iniciado; caballero en el suelo de la zona 1
Pasos	1. Mantener D durante 2 s. 2. Soltar D y mantener A durante 2 s. 3. Presionar Espacio una vez y soltarlo enseguida. 4. Mantener Espacio en un segundo salto.
Resultado esperado	El caballero camina a la derecha y a la izquierda; el salto corto es más bajo que el salto con la tecla mantenida (unos 42 px, 2,6 casillas).
Resultado obtenido	Igual al esperado.
Estado, versión y fecha	Aprobado Ejecutable: v1.0.0 Fecha: 21 oct. 2026 Ejecutó: Jaider Villamizar
Evidencia y defecto asociado	`E-01.png` Defecto: ninguno

Nota. Elaboración propia.

Figura 55

Evidencia del caso CP-01 (E-01.png)



Nota. Captura del ejecutable v1.0.0.

Tabla 98

Caso de prueba CP-02: Doble salto

Campo	Contenido
ID y nombre	CP-02 Doble salto
Requisito y HU que verifica	RF-02 / HU-02
Tipo y técnica	Funcional / valores límite (segundo y tercer salto en el aire)
Precondiciones y datos	Nivel iniciado; caballero en el suelo
Pasos	1. Presionar Espacio para saltar. 2. En el aire, presionar Espacio de nuevo. 3. Aún en el aire, presionar Espacio una tercera vez. 4. Aterrizar y repetir los pasos 1 y 2.
Resultado esperado	El segundo salto impulsa hacia arriba y suena `doble_salto.wav`; el tercero no tiene efecto; al aterrizar el doble salto vuelve a estar disponible.
Resultado obtenido	Igual al esperado.
Estado, versión y fecha	Aprobado Ejecutable: v1.0.0 Fecha: 21 oct. 2026 Ejecutó: Jaider Villamizar
Evidencia y defecto asociado	`E-02.png` Defecto: ninguno

Nota. Elaboración propia.

Figura 56

Evidencia del caso CP-02 (E-02.png)



Nota. Captura del ejecutable v1.0.0.

Tabla 99

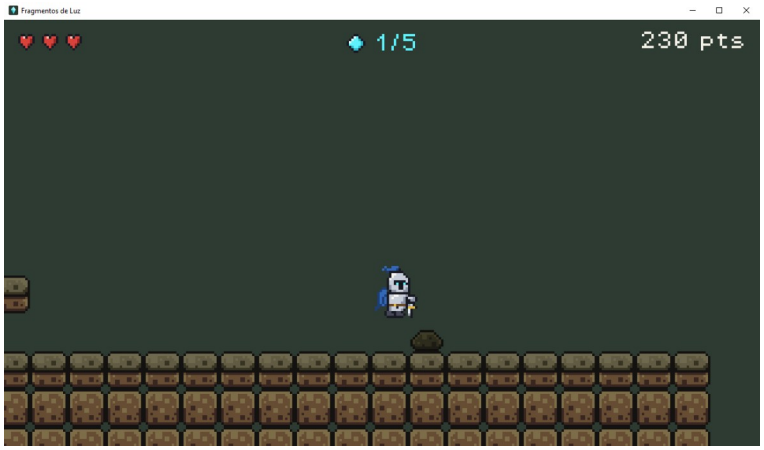
Caso de prueba CP-03: Impulso con enfriamiento

Campo	Contenido
ID y nombre	CP-03 Impulso con enfriamiento
Requisito y HU que verifica	RF-03 / HU-03
Tipo y técnica	Funcional / valores límite (antes y después de 0,5 s)
Precondiciones y datos	Nivel iniciado; caballero en el suelo mirando a la derecha
Pasos	1. Presionar Mayúsculas. 2. Presionar Mayúsculas de nuevo antes de medio segundo. 3. Esperar 1 s y presionar Mayúsculas. 4. Saltar y, en el aire, presionar Mayúsculas dos veces.
Resultado esperado	El caballero avanza unos 39 px sin caer y suena `dash.wav`; el segundo intento inmediato no hace nada; tras el enfriamiento vuelve a funcionar; en el aire solo funciona una vez.
Resultado obtenido	Igual al esperado.
Estado, versión y fecha	Aprobado Ejecutable: v1.0.0 Fecha: 21 oct. 2026 Ejecutó: Jaider Villamizar
Evidencia y defecto asociado	`E-03.png` Defecto: ninguno

Nota. Elaboración propia.

Figura 57

Evidencia del caso CP-03 (E-03.png)



Nota. Captura del ejecutable v1.0.0.

Tabla 100

Caso de prueba CP-04: Atacar con la espada

Campo	Contenido
ID y nombre	CP-04 Atacar con la espada
Requisito y HU que verifica	RF-04 / HU-08
Tipo y técnica	Funcional / partición de equivalencia (slime y murciélago)
Precondiciones y datos	Nivel iniciado; caballero frente al primer slime de la zona 1
Pasos	1. Acercarse al slime mirándolo y presionar J. 2. Observar el puntaje. 3. En la zona 3, golpear una vez al murciélago. 4. Golpearlo una segunda vez.
Resultado esperado	El slime desaparece con un golpe y el puntaje suma 50; el murciélago sigue activo tras el primer golpe y desaparece con el segundo (+50).
Resultado obtenido	Igual al esperado.
Estado, versión y fecha	Aprobado Ejecutable: v1.0.0 Fecha: 21 oct. 2026 Ejecutó: Jaider Villamizar
Evidencia y defecto asociado	`E-04_E-05.png` Defecto: ninguno

Nota. Elaboración propia.

Figura 58

Evidencia del caso CP-04 (E-04_E-05.png)



Nota. Captura del ejecutable v1.0.0.

Tabla 101

Caso de prueba CP-05: Daño, invulnerabilidad y derrota

Campo	Contenido
ID y nombre	CP-05 Daño, invulnerabilidad y derrota
Requisito y HU que verifica	RF-05, RF-13 / HU-09, HU-18
Tipo y técnica	Funcional / transición de estados
Precondiciones y datos	Nivel iniciado con 3 corazones
Pasos	1. Tocar un slime. 2. Antes de 1 s, tocar otro peligro. 3. Tocar púas dos veces más, esperando más de 1 s entre cada vez. 4. En la pantalla de derrota, presionar Reintentar.
Resultado esperado	El primer contacto deja 2 corazones con parpadeo y retroceso; el segundo, dentro del segundo, no quita corazones; al llegar a 0 aparece «La sombra te venció» con fragmentos y puntos; Reintentar empieza con 3 corazones, 0/5 y 0 puntos.
Resultado obtenido	Igual al esperado. La evidencia muestra el daño (1 corazón) y el caballero derrotado con 0 corazones.
Estado, versión y fecha	Aprobado Ejecutable: v1.0.0 Fecha: 21 oct. 2026 Ejecutó: Jaider Villamizar
Evidencia y defecto asociado	`E-04_E-05.png`, `E-05_derrota.png` Defecto: ninguno

Nota. Elaboración propia.

Figura 59

Evidencia del caso CP-05 (E-04_E-05.png)



Nota. Captura del ejecutable v1.0.0.

Figura 60

Evidencia del caso CP-05 (E-05_derrota.png)



Nota. Captura del ejecutable v1.0.0.

Tabla 102

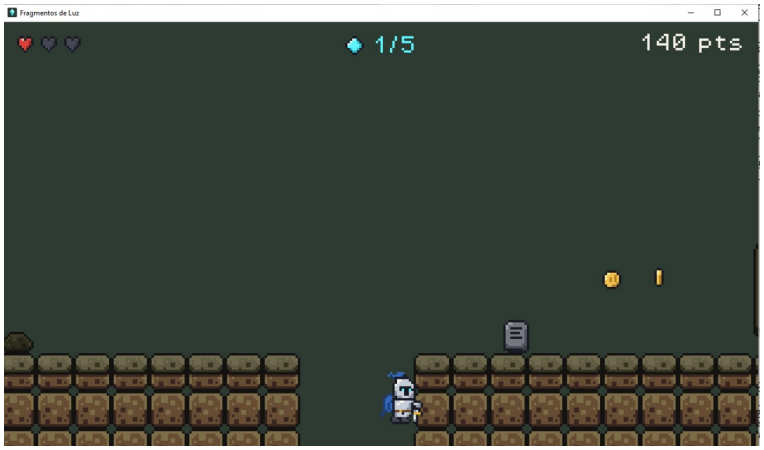
Caso de prueba CP-06: Caída al vacío y punto de control

Campo	Contenido
ID y nombre	CP-06 Caída al vacío y punto de control
Requisito y HU que verifica	RF-06 / HU-05
Tipo y técnica	Funcional / partición de equivalencia (con y sin punto de control)
Precondiciones y datos	Nivel iniciado con 3 corazones
Pasos	1. Sin tocar ningún estandarte, caer al foso de la zona 2. 2. Avanzar y tocar el estandarte del punto de control 1. 3. Caer al vacío de nuevo.
Resultado esperado	Tras la primera caída reaparece en el inicio con 2 corazones; el estandarte cambia de color al tocarlo; tras la segunda caída reaparece en el punto de control 1 con 1 corazón.
Resultado obtenido	Igual al esperado.
Estado, versión y fecha	Aprobado Ejecutable: v1.0.0 Fecha: 21 oct. 2026 Ejecutó: Jaider Villamizar
Evidencia y defecto asociado	`E-06.png` Defecto: ninguno

Nota. Elaboración propia.

Figura 61

Evidencia del caso CP-06 (E-06.png)



Nota. Captura del ejecutable v1.0.0.

Tabla 103

Caso de prueba CP-07: Slime patrullero

Campo	Contenido
ID y nombre	CP-07 Slime patrullero
Requisito y HU que verifica	RF-07 / HU-04
Tipo y técnica	Funcional / observación de comportamiento
Precondiciones y datos	Nivel iniciado; slime de la zona 1 a la vista
Pasos	1. Observar al slime durante 10 s sin acercarse. 2. Observar qué hace al llegar al borde de una plataforma o a una pared.
Resultado esperado	El slime camina de lado a lado y da la vuelta en paredes y bordes, sin caerse.
Resultado obtenido	Igual al esperado.
Estado, versión y fecha	Aprobado Ejecutable: v1.0.0 Fecha: 21 oct. 2026 Ejecutó: Jaider Villamizar
Evidencia y defecto asociado	`E-07.png` Defecto: ninguno

Nota. Elaboración propia.

Figura 62

Evidencia del caso CP-07 (E-07.png)



Nota. Captura del ejecutable v1.0.0.

Tabla 104

Caso de prueba CP-08: Murciélago perseguidor

Campo	Contenido
ID y nombre	CP-08 Murciélago perseguidor
Requisito y HU que verifica	RF-08 / HU-10
Tipo y técnica	Funcional / valores límite (80 px y 140 px)
Precondiciones y datos	Nivel en la zona 3; murciélago dormido
Pasos	1. Acercarse al murciélago lentamente. 2. Cuando persiga, alejarse rápidamente hacia atrás. 3. Esperar a que vuelva a su sitio.
Resultado esperado	Despierta y persigue al caballero cuando está cerca (menos de 80 px); al alejarse (más de 140 px) regresa a su origen y vuelve a dormir.
Resultado obtenido	Igual al esperado.
Estado, versión y fecha	Aprobado Ejecutable: v1.0.0 Fecha: 21 oct. 2026 Ejecutó: Jaider Villamizar
Evidencia y defecto asociado	`E-08.png` Defecto: ninguno

Nota. Elaboración propia.

Figura 63

Evidencia del caso CP-08 (E-08.png)



Nota. Captura del ejecutable v1.0.0.

Tabla 105

Caso de prueba CP-09: Roca empujable

Campo	Contenido
ID y nombre	CP-09 Roca empujable
Requisito y HU que verifica	RF-09 / HU-11
Tipo y técnica	Funcional / caso de uso del acertijo
Precondiciones y datos	Nivel en la zona 4 (ruinas)
Pasos	1. Caminar contra la roca. 2. Empujarla hasta debajo de la cornisa. 3. Saltar sobre la roca y luego a la cornisa.
Resultado esperado	La roca se desplaza al empujarla; el caballero puede pararse sobre ella y alcanzar el fragmento F4.
Resultado obtenido	Repetido sobre el ejecutable corregido (DF-06): la roca se desplaza al empujarla, el caballero se sube a ella, alcanza la cornisa y toma el fragmento F4. El contador marca 2/5 porque en esa partida no se tomó otro fragmento anterior.
Estado, versión y fecha	Aprobado Ejecutable: v1.0.0 Fecha: 21 oct. 2026 Ejecutó: Jaider Villamizar
Evidencia y defecto asociado	`E-09_reintento_empujando.png`, `E-09_sobre_la_roca.png`, `E-09_en_la_cornisa.png` Defecto: DF-06 (corregido y verificado)

Nota. Elaboración propia.

Figura 64

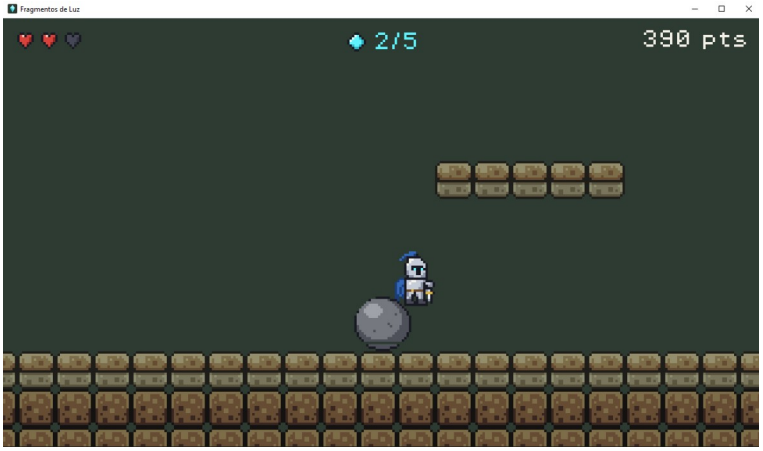
Evidencia del caso CP-09 (E-09_reintento_empujando.png)



Nota. Captura del ejecutable v1.0.0.

Figura 65

Evidencia del caso CP-09 (E-09_sobre_la_roca.png)



Nota. Captura del ejecutable v1.0.0.

Figura 66

Evidencia del caso CP-09 (E-09_en_la_cornisa.png)



Nota. Captura del ejecutable v1.0.0.

Tabla 106

Caso de prueba CP-10: Corriente de viento

Campo	Contenido
ID y nombre	CP-10 Corriente de viento
Requisito y HU que verifica	RF-10 / HU-12
Tipo y técnica	Funcional / partición de equivalencia (dentro y fuera de la corriente)
Precondiciones y datos	Nivel en la zona 3
Pasos	1. Entrar en la columna de viento sin presionar nada. 2. Subir hasta la plataforma alta. 3. Salir de la corriente en el aire.
Resultado esperado	El caballero sube solo dentro de la corriente hasta la plataforma de F3; al salir cae con la gravedad normal.
Resultado obtenido	Igual al esperado.
Estado, versión y fecha	Aprobado Ejecutable: v1.0.0 Fecha: 21 oct. 2026 Ejecutó: Jaider Villamizar
Evidencia y defecto asociado	`E-10.png` Defecto: ninguno

Nota. Elaboración propia.

Figura 67

Evidencia del caso CP-10 (E-10.png)



Nota. Captura del ejecutable v1.0.0.

Tabla 107

Caso de prueba CP-11: Recoger fragmentos

Campo	Contenido
ID y nombre	CP-11 Recoger fragmentos
Requisito y HU que verifica	RF-11 / HU-13
Tipo y técnica	Funcional
Precondiciones y datos	Nivel iniciado con 0/5 y 0 puntos
Pasos	1. Recoger el fragmento F1. 2. Observar el HUD. 3. Volver a pasar por el lugar de F1.
Resultado esperado	El contador pasa a 1/5 con una animación, el puntaje suma 100 y suena `fragmento.wav`; F1 ya no aparece.
Resultado obtenido	Igual al esperado. El contador pasa a 1/5.
Estado, versión y fecha	Aprobado Ejecutable: v1.0.0 Fecha: 21 oct. 2026 Ejecutó: Jaider Villamizar
Evidencia y defecto asociado	`E-11.png` Defecto: ninguno

Nota. Elaboración propia.

Figura 68

Evidencia del caso CP-11 (E-11.png)



Nota. Captura del ejecutable v1.0.0.

Tabla 108

Caso de prueba CP-12: Recoger monedas

Campo	Contenido
ID y nombre	CP-12 Recoger monedas
Requisito y HU que verifica	RF-12 / HU-14
Tipo y técnica	Funcional
Precondiciones y datos	Nivel iniciado; anotar el puntaje
Pasos	1. Recoger una moneda de la zona 1.
Resultado esperado	El puntaje suma 10, la moneda desaparece y suena `moneda.wav`.
Resultado obtenido	Igual al esperado.
Estado, versión y fecha	Aprobado Ejecutable: v1.0.0 Fecha: 21 oct. 2026 Ejecutó: Jaider Villamizar
Evidencia y defecto asociado	`E-12.png` Defecto: ninguno

Nota. Elaboración propia.

Figura 69

Evidencia del caso CP-12 (E-12.png)



Nota. Captura del ejecutable v1.0.0.

Tabla 109

Caso de prueba CP-13: Santuario y victoria

Campo	Contenido
ID y nombre	CP-13 Santuario y victoria
Requisito y HU que verifica	RF-13 / HU-18
Tipo y técnica	Funcional / partición de equivalencia (3/5, 4/5 y 5/5)
Precondiciones y datos	Partida con 3/5 fragmentos junto al santuario (y luego con 5/5)
Pasos	1. Tocar el santuario con 3/5. 2. Recoger un fragmento más y tocarlo con 4/5. 3. Con 5/5, tocar el santuario. 4. Avanzar la cinemática final.
Resultado esperado	Con 3/5 aparece «Faltan 2 fragmentos»; con 4/5, «Falta 1 fragmento»; con 5/5 se restaura el Cristal, se muestran las 3 pantallas finales y la victoria con puntos, bono por corazones y total.
Resultado obtenido	Igual al esperado: con 3/5 aparece «Faltan 2 fragmentos»; con 5/5 se muestran las pantallas finales y la victoria con 750 puntos, bono por corazones +100, total 850 y «¡Nuevo mejor puntaje!».
Estado, versión y fecha	Aprobado Ejecutable: v1.0.0 Fecha: 21 oct. 2026 Ejecutó: Jaider Villamizar
Evidencia y defecto asociado	`E-13_3de5.png`, `E-13_final.png`, `E-13_victoria.png` Defecto: ninguno

Nota. Elaboración propia.

Figura 70

Evidencia del caso CP-13 (E-13_3de5.png)



Nota. Captura del ejecutable v1.0.0.

Figura 71

Evidencia del caso CP-13 (E-13_final.png)



Nota. Captura del ejecutable v1.0.0.

Figura 72

Evidencia del caso CP-13 (E-13_victoria.png)



Nota. Captura del ejecutable v1.0.0.

Tabla 110

Caso de prueba CP-14: HUD

Campo	Contenido
ID y nombre	CP-14 HUD
Requisito y HU que verifica	RF-14 / HU-17
Tipo y técnica	Funcional
Precondiciones y datos	Nivel iniciado
Pasos	1. Recibir un daño. 2. Recoger un fragmento. 3. Derrotar un enemigo.
Resultado esperado	El corazón perdido pasa a vacío, el contador de fragmentos y el puntaje cambian en el mismo instante.
Resultado obtenido	Igual al esperado. Evidencia compartida con CP-04 y CP-05 (HUD con corazones, fragmentos y puntaje).
Estado, versión y fecha	Aprobado Ejecutable: v1.0.0 Fecha: 21 oct. 2026 Ejecutó: Jaider Villamizar
Evidencia y defecto asociado	`E-04_E-05.png` Defecto: ninguno

Nota. Elaboración propia.

Figura 73

Evidencia del caso CP-14 (E-04_E-05.png)



Nota. Captura del ejecutable v1.0.0.

Tabla 111

Caso de prueba CP-15: Pausa y reinicio

Campo	Contenido
ID y nombre	CP-15 Pausa y reinicio
Requisito y HU que verifica	RF-15 / HU-19
Tipo y técnica	Funcional
Precondiciones y datos	Nivel con 1/5 fragmentos y puntos acumulados
Pasos	1. Presionar Esc. 2. Observar los enemigos durante 3 s. 3. Presionar Continuar. 4. Presionar Esc y elegir Reiniciar nivel.
Resultado esperado	El juego se detiene, los enemigos quedan quietos y la música baja; Continuar reanuda; Reiniciar nivel empieza con 3 corazones, 0/5 y 0 puntos en el inicio.
Resultado obtenido	Igual al esperado.
Estado, versión y fecha	Aprobado Ejecutable: v1.0.0 Fecha: 21 oct. 2026 Ejecutó: Jaider Villamizar
Evidencia y defecto asociado	`E-15_pausa.png`, `E-15_reinicio.png` Defecto: ninguno

Nota. Elaboración propia.

Figura 74

Evidencia del caso CP-15 (E-15_pausa.png)



Nota. Captura del ejecutable v1.0.0.

Figura 75

Evidencia del caso CP-15 (E-15_reinicio.png)



Nota. Captura del ejecutable v1.0.0.

Tabla 112

Caso de prueba CP-16: Historia en el juego

Campo	Contenido
ID y nombre	CP-16 Historia en el juego
Requisito y HU que verifica	RF-16 / HU-15
Tipo y técnica	Funcional
Precondiciones y datos	Menú principal
Pasos	1. Presionar Jugar y avanzar con Enter las 4 pantallas. 2. En el nivel, acercarse y alejarse del primer cartel. 3. Repetir desde el menú y presionar Esc en la introducción.
Resultado esperado	Se muestran las 4 pantallas con el texto letra por letra y luego el nivel; el texto del cartel aparece al acercarse y desaparece al alejarse; Esc salta la introducción.
Resultado obtenido	Igual al esperado.
Estado, versión y fecha	Aprobado Ejecutable: v1.0.0 Fecha: 21 oct. 2026 Ejecutó: Jaider Villamizar
Evidencia y defecto asociado	`E-16_intro.png`, `E-07.png` Defecto: ninguno

Nota. Elaboración propia.

Figura 76

Evidencia del caso CP-16 (E-16_intro.png)



Nota. Captura del ejecutable v1.0.0.

Figura 77

Evidencia del caso CP-16 (E-07.png)



Nota. Captura del ejecutable v1.0.0.

Tabla 113

Caso de prueba CP-17: Música y efectos

Campo	Contenido
ID y nombre	CP-17 Música y efectos
Requisito y HU que verifica	RF-17 / HU-16
Tipo y técnica	Funcional
Precondiciones y datos	Volumen del equipo encendido
Pasos	1. Escuchar el menú principal. 2. Entrar al nivel. 3. Saltar, hacer doble salto, impulso y ataque, recibir daño y recoger objetos. 4. Presionar cualquier botón de un menú.
Resultado esperado	Suena la música del menú y cambia a la del nivel; cada acción reproduce su efecto; los botones suenan.
Resultado obtenido	Igual al esperado. Verificado al oído; el sonido no se registra en una captura.
Estado, versión y fecha	Aprobado Ejecutable: v1.0.0 Fecha: 21 oct. 2026 Ejecutó: Jaider Villamizar
Evidencia y defecto asociado	Sin captura (verificado al oído) Defecto: ninguno

Nota. Elaboración propia.

Tabla 114

Caso de prueba CP-18: Menú, opciones y créditos

Campo	Contenido
ID y nombre	CP-18 Menú, opciones y créditos
Requisito y HU que verifica	RF-18 / HU-06, HU-20, HU-22
Tipo y técnica	Funcional / valores límite (0 % y 100 %)
Precondiciones y datos	Menú principal
Pasos	1. Presionar Opciones y bajar Música a 0 %. 2. Mover Efectos. 3. Presionar Volver y luego Créditos. 4. Presionar Volver.
Resultado esperado	Con Música en 0 % la música deja de oírse pero los efectos siguen; el deslizador de Efectos reproduce un sonido de muestra; los créditos muestran equipo, licencias y el aviso de Godot y suben solos; Volver regresa al menú.
Resultado obtenido	Igual al esperado.
Estado, versión y fecha	Aprobado Ejecutable: v1.0.0 Fecha: 21 oct. 2026 Ejecutó: Jaider Villamizar
Evidencia y defecto asociado	`E-18_opciones.png`, `E-18_creditos.png` Defecto: ninguno

Nota. Elaboración propia.

Figura 78

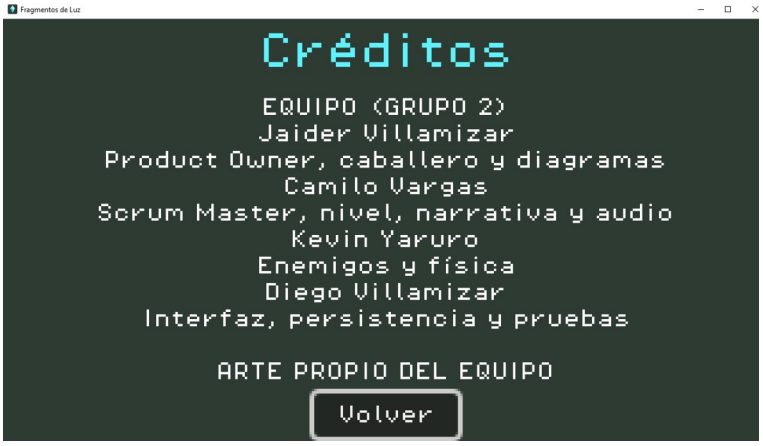
Evidencia del caso CP-18 (E-18_opciones.png)



Nota. Captura del ejecutable v1.0.0.

Figura 79

Evidencia del caso CP-18 (E-18_creditos.png)



Nota. Captura del ejecutable v1.0.0.

Tabla 115

Caso de prueba CP-19: Persistencia del récord y del volumen

Campo	Contenido
ID y nombre	CP-19 Persistencia del récord y del volumen
Requisito y HU que verifica	RF-19 / HU-21
Tipo y técnica	Funcional / regresión entre sesiones
Precondiciones y datos	Sin archivo de guardado (primera ejecución)
Pasos	1. Verificar que el menú dice «Mejor puntaje: 0». 2. Poner Música al 30 % y presionar Volver. 3. Ganar una partida. 4. Cerrar el juego y abrirlo de nuevo.
Resultado esperado	La victoria muestra «¡Nuevo mejor puntaje!»; al reabrir, el menú muestra ese puntaje y Opciones sigue en 30 %.
Resultado obtenido	Igual al esperado: tras ganar (850, «¡Nuevo mejor puntaje!») y volver a abrir el juego, el menú muestra «Mejor puntaje: 850»; `user://config.cfg` guarda `mejor_puntaje=850` y `volumen_musica=0.45` (la prueba se hizo con Música en 45 % en lugar de 30 %).
Estado, versión y fecha	Aprobado Ejecutable: v1.0.0 Fecha: 21 oct. 2026 Ejecutó: Jaider Villamizar
Evidencia y defecto asociado	`E-13_victoria.png`, `E-19_menu_record.png` Defecto: ninguno

Nota. Elaboración propia.

Figura 80

Evidencia del caso CP-19 (E-13_victoria.png)



Nota. Captura del ejecutable v1.0.0.

Figura 81

Evidencia del caso CP-19 (E-19_menu_record.png)



Nota. Captura del ejecutable v1.0.0.

Tabla 116

Caso de prueba CP-20: Ejecutable sin Godot

Campo	Contenido
ID y nombre	CP-20 Ejecutable sin Godot
Requisito y HU que verifica	RNF-05 / HU-23
Tipo y técnica	Aceptación / instalación
Precondiciones y datos	Equipo con Windows 10 u 11 sin Godot instalado
Pasos	1. Descargar el `.zip` de la versión publicada v1.0.0. 2. Descomprimirlo y abrir `FragmentosDeLuz.exe`. 3. Jugar hasta la zona 2 y salir.
Resultado esperado	El juego abre en el menú principal sin pedir nada adicional y funciona sin errores.
Resultado obtenido	Igual al esperado. Ejecutado en el equipo de referencia desde el `.zip` de v1.0.0 extraído en una carpeta aparte (`Escritorio\Prueba CP-20`, solo `FragmentosDeLuz.exe` y `.pck`), con el editor de Godot cerrado; el administrador de tareas confirmó que el proceso salía de esa carpeta y que no había ningún proceso de Godot. Abrió en el menú y se jugó hasta la zona 2. Limitación: no se probó en un segundo equipo sin Godot.
Estado, versión y fecha	Aprobado Ejecutable: v1.0.0 Fecha: 21 oct. 2026 Ejecutó: Jaider Villamizar
Evidencia y defecto asociado	`E-20_carpeta_aparte.png` Defecto: ninguno

Nota. Elaboración propia.

Figura 82

Evidencia del caso CP-20 (E-20_carpeta_aparte.png)



Nota. Captura del ejecutable v1.0.0.

Anexo C. Datos de las sesiones de pruebas de juego

Datos de cada sesión, tomados del registro en vivo del 21 de octubre de 2026 (el observador dictó los datos durante la partida y la persona respondió el cuestionario). Ninguna persona autorizó su nombre: aparecen como «Usuario N». La sesión 5 no se realizó porque la persona no pudo asistir.

Tabla 117

Sesión 1 de pruebas de juego

Dato	Valor
Participante	Usuario 1 (no autorizó su nombre; aceptó de palabra)
Fecha, observador y ejecutable	21 oct. 2026, 11:38 a. m.; Jaider Villamizar; v1.0.0 sin la corrección de DF-06
Rango de edad y experiencia con videojuegos	18 a 25; juega a veces
Zonas 1 y 2 sin ayuda	Sí / Sí (zona 2 en el tercer intento)
Zona a los 2 y 5 minutos	Zona 2 a los ~6 min (perdió dos veces antes); a los 5 min no se anotó
¿Terminó el nivel? (tiempo y puntaje)	No; paró a los ~8 min en la zona 3
Caídas o daños y ayudas	3 derrotas; ayudas: ninguna
Respuestas SUS (frases 1 a 10)	5, 1, 3, 3, 3, 3, 5, 2, 5, 1
Puntuación SUS	77,5
Observaciones	Al principio el observador creyó que el impulso no funcionaba; luego cruzó el barranco solo. Su primera respuesta del SUS («5 en todas») se descartó y se le leyeron las frases una por una.
Respuestas abiertas	Le gustó el juego. Lo más difícil: el barranco. Cambiaría: agregar un jefe final.

Nota. Elaboración propia.

Tabla 118

Sesión 2 de pruebas de juego

Dato	Valor
Participante	Usuario 2 (no autorizó su nombre; aceptó de palabra)
Fecha, observador y ejecutable	21 oct. 2026, 11:54 a. m.; Jaider Villamizar; v1.0.0 sin la corrección de DF-06
Rango de edad y experiencia con videojuegos	18 a 25; juega con frecuencia
Zonas 1 y 2 sin ayuda	Sí / Sí (impulso con Shift)
Zona a los 2 y 5 minutos	A los 2 min no se confirmó (estaba entre las zonas 3 y 4); terminó antes de los 5 min
¿Terminó el nivel? (tiempo y puntaje)	No; paró a los ~3 min en la zona 4
Caídas o daños y ayudas	1 derrota (murciélago, zona 4); ayudas: ninguna
Respuestas SUS (frases 1 a 10)	4, 3, 3, 2, 3, 2, 5, 1, 4, 2
Puntuación SUS	72,5
Observaciones	Entendió sola el viento y empujó la roca sin lograr moverla. Al repetirlo, el observador confirmó que la roca no se movía (DF-06).
Respuestas abiertas	Le gustó la ambientación, el barranco y el diseño. Lo más difícil y lo que cambiaría: «el barranco y el diseño» (lo dijo así, sin separar).

Nota. Elaboración propia.

Tabla 119

Sesión 3 de pruebas de juego

Dato	Valor
Participante	Usuario 3 (no autorizó su nombre; aceptó de palabra)
Fecha, observador y ejecutable	21 oct. 2026, 12:53 p. m.; Jaider Villamizar; v1.0.0 con la corrección de DF-06
Rango de edad y experiencia con videojuegos	26 a 40; juega con frecuencia
Zonas 1 y 2 sin ayuda	Sí / Sí
Zona a los 2 y 5 minutos	Zona 4 a los ~2 min; terminó antes de los 5 min
¿Terminó el nivel? (tiempo y puntaje)	No; paró a los ~2 min en la zona 4
Caídas o daños y ayudas	1 derrota (murciélago, zona 4); ayudas: ninguna
Respuestas SUS (frases 1 a 10)	3, 3, 3, 3, 3, 4, 5, 2, 5, 1
Puntuación SUS	65
Observaciones	Empujó la roca con la corrección de DF-06.
Respuestas abiertas	Le gustó la historia. Lo más difícil: lo de la roca. Cambiaría: los enemigos.

Nota. Elaboración propia.

Tabla 120

Sesión 4 de pruebas de juego

Dato	Valor
Participante	Usuario 4 (no autorizó su nombre; aceptó de palabra)
Fecha, observador y ejecutable	21 oct. 2026, 12:59 p. m.; Jaider Villamizar; v1.0.0 con la corrección de DF-06
Rango de edad y experiencia con videojuegos	26 a 40; juega con frecuencia
Zonas 1 y 2 sin ayuda	Sí / Sí
Zona a los 2 y 5 minutos	Zonas 1 y 2 superadas antes de los 2 min; terminó hacia los 5 min
¿Terminó el nivel? (tiempo y puntaje)	Sí, en ~5 min; 890 puntos + 100 de bono = 990 («¡Nuevo mejor puntaje!», guardado a la 1:04 p. m.)
Caídas o daños y ayudas	No se anotaron; ayudas: ninguna
Respuestas SUS (frases 1 a 10)	3, 2, 2, 3, 3, 4, 4, 2, 4, 1
Puntuación SUS	60
Observaciones	Única persona que terminó el nivel.
Respuestas abiertas	Le gustó el juego. Lo más difícil: el barranco. Cambiaría: los diseños del juego.

Nota. Elaboración propia.

Anexo D. Evidencias de los eventos Scrum

Cada evento con su fecha, medio, asistentes y decisiones. Se marca «sin registro» lo que no quedó documentado y «sin captura» lo que tiene acta escrita pero no imagen.

Tabla 121

Eventos Scrum realizados

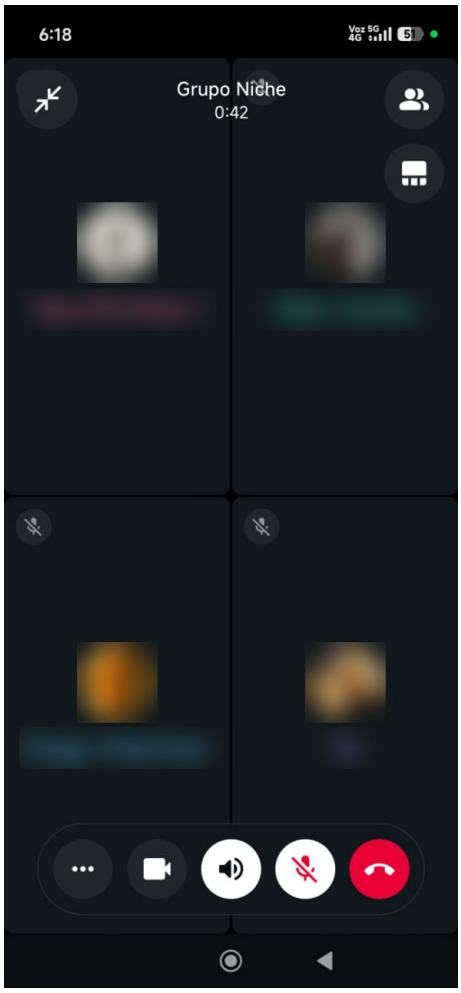
Evento	Fecha	Medio	Asistentes	Decisiones	Evidencia
Planificación del Sprint 1	19 oct. 2026	No registrado	Los cuatro integrantes	Se comprometieron HU-01 a HU-07 (25 puntos); orden de integración por dependencias (HU-07 primero); cada integrante sube su parte desde su cuenta con una solicitud revisada por el compañero de la rotación	Acta escrita (sin captura)
Revisión y retrospectiva del Sprint 1	20 oct. 2026	—	—	—	Sin registro
Planificación del Sprint 2 y reunión diaria	21 oct. 2026	Presencial	Los cuatro integrantes (nadie faltó)	Adelantar trabajo en la noche	Acta escrita (sin captura)
Retrospectiva de los sprints 1 y 2	22 oct. 2026, 6:18 p. m.	Llamada grupal de WhatsApp	Los cuatro integrantes	No funcionaron el empuje de la roca (DF-06) ni, al principio, el impulso; se haría distinto: mejorar mucho la parte visual y la jugabilidad (sección 7.5)	Figura (captura de la llamada)

Nota. Elaboración propia.

La figura siguiente es la captura de la llamada del 22 de octubre: cuatro participantes en la llamada «Grupo Niche». Las fotos de perfil y los nombres de contacto se difuminaron por privacidad.

Figura 83

Llamada de la retrospectiva del 22 de octubre



Nota. Captura del celular de un integrante, 22 oct. 2026.